

Agentic Hybrid RAG on SAP BTP

Sriram Rokkam

June 2026

Contents

Front Matter	8
Agentic Hybrid RAG on SAP BTP	8
A Hands-On Guide with LangGraph, HANA Cloud, and Google Vertex AI	8
Title Page	8
Copyright	8
Dedication	9
Foreword	9
Preface	9
Who This Book Is For	10
Who This Book Is NOT For	10
Prerequisites	11
Technical Knowledge	11
NOT Required	12
Tools and Accounts (All Free)	12
How to Use This Book	12
Reading Cover to Cover (Recommended)	12
As a Reference	13
Chapter Structure	13
The Code	13
Conventions Used in This Book	13
Code Blocks	13
Callout Boxes	14
File Paths	14
Screenshots	14
Acknowledgements	14
About the Author	15
Chapter 1: Welcome to the Agentic Era	16
1.1 From Models That Answer to Systems That Act	16
1.2 A Spectrum, Not a Switch	17
1.3 The Trouble With General-Purpose Assistants	19
1.4 Why Hybrid Retrieval Is the Right Engine	21
1.5 What We Will Build	22
1.6 Why This Moment, and Why for SAP Developers	23
1.7 Summary	24

Chapter 2: What Is Agentic AI — And Why SAP Developers Should Care	26
2.1 Beyond the Chatbot	26
2.2 The Four Capabilities That Define an Agent	26
2.3 Why This Matters for SAP Developers	27
SAP's Strategic Direction: The Autonomous Enterprise	27
SAP's AI Partnerships and Infrastructure Investments	28
What This Means for You as a Developer	30
2.4 What We Build in This Book	30
2.5 Scope of This Book	31
What This Book Covers	32
What This Book Does NOT Cover	32
Local Development is the Primary Mode	33
2.6 The Platform Choice: SAP BTP + Google Vertex AI	34
2.7 What You Need to Follow Along	34
2.8 Summary	35
Chapter 3: Platform Setup — SAP BTP Trial + Google Vertex AI	36
3.1 The Goal of This Chapter	36
3.2 Understanding What We Are Setting Up	36
3.3 SAP BTP Trial Account	38
Signing Up	38
Navigating to Your Trial Subaccount	39
3.4 Provisioning SAP HANA Cloud	40
Creating the HANA Cloud Instance	40
Enabling the Vector Engine	42
Getting Your HANA Connection Details	43
3.5 Setting Up Google Cloud + Vertex AI	43
Creating a GCP Account	43
Creating a New Project	45
Enabling the Vertex AI API	45
Creating a Service Account	45
Downloading the Service Account Key	46
3.6 Configuring the BTP Destination Service	47
Creating the Destination	47
3.7 Local Development Environment	49
Python 3.11+	49
Node.js 20+	50
Cloud Foundry CLI	50

SAP CDS CLI	51
VS Code Extensions	51
3.8 Project Setup	51
3.9 Verifying the Setup: Your First Gemini API Call.	53
3.10 Verifying the HANA Cloud Connection	55
3.11 What We Have Built.	57
3.12 Understanding the Credential Flow	57
3.13 Summary	58
Chapter 3: Vector Search on SAP HANA Cloud	59
3.1 What Are Embeddings? Intuition Without the Math	59
3.2 The HANA Cloud REAL_VECTOR Column Type	60
3.3 Creating the Vector Table: The Lazy Initialization Pattern	63
3.4 Calling text-embedding-004 from Python	65
3.5 Storing Embeddings: INSERT with REAL_VECTOR	66
3.6 Cosine Similarity Search: The Query Explained	68
3.7 Chunking Strategy for MSDS Documents.	70
3.8 Building vector_srv.py — The Vector Service Layer	71
3.9 Testing: One MSDS, Two Questions	75
3.10 What Vector Search Gets Right — and What It Gets Wrong.	78
3.11 Summary	80
3.12 Checkpoint.	80
Chapter 4: Knowledge Graphs on SAP HANA Cloud	82
4.1 What is a knowledge graph?	82
Why call it a graph?	83
What does this give us that a relational schema does not?	84
4.2 RDF and SPARQL — the standards explained plainly	85
4.3 Why HANA Cloud supports SPARQL.	86
Why a stored procedure and not an HTTP endpoint?	87
4.4 Designing the MSDS ontology — what facts matter?	87
4.5 The OWL ontology file — MSDS_Ontology.ttl	88
4.6 Extracting triples from MSDS text using Gemini	90
4.7 Storing triples in HANA — named graphs per document	92
4.8 Writing SPARQL queries — from natural language to graph traversal	94
4.9 Building kg_srv.py — the knowledge graph service layer	96
Validation: the KG equivalent of SQL injection.	100
What delete_graph and count_triples give us	101

4.10 Testing — store triples for acetone, query for GHS codes	101
4.11 What the KG gets right that vector search cannot	104
4.12 Summary	106
4.13 Checkpoint.	107
Chapter 6: The PDF Ingestion Pipeline	108
6.1 The dual-pipeline architecture	108
6.1.1 Why process both in parallel?	109
6.1.2 The fire-and-forget pattern	110
6.2 PDF text extraction with PyMuPDF	111
6.2.1 Extracting with metadata	111
6.2.2 Handling upload files in FastAPI	112
6.3 Chunking strategy for vector search	113
6.3.1 Our chunking parameters	113
6.3.2 The chunker function	114
6.3.3 Why the KG pipeline does not chunk	115
6.4 Thread-local HANA connections	116
6.4.1 Why connections cannot be shared across threads	116
6.4.2 The <code>threading.local()</code> solution	117
6.5 The ingestion flow in detail	118
6.5.1 The upload endpoint	118
6.5.2 The dual-pipeline orchestrator	119
6.5.3 Thread 1 — the vector pipeline	120
6.5.4 Thread 2 — the knowledge graph pipeline.	121
6.6 Status tracking	122
6.6.1 The status fields.	122
6.6.2 Status helper functions	123
6.6.3 The status endpoint	124
6.7 Error handling — what happens when one pipeline fails	125
6.8 Testing — upload five MSDS documents	126
6.8.1 Start the service.	126
6.8.2 Upload the first document	126
6.8.3 Poll for completion.	126
6.8.4 Verify the vector store	126
6.8.5 Verify the knowledge graph	127
6.9 Summary	128
6.10 Checkpoint.	129
Chapter 7: LangGraph Fundamentals.	130
7.1 Why LangChain chains are not enough	130

7.2 Core concepts	131
7.2.1 State	131
7.2.2 Nodes	132
7.2.3 Edges	132
7.2.4 Conditional edges	132
7.3 Installation and project layout	133
7.4 Building the state and nodes	133
7.5 Wiring the graph	135
7.6 Adding a conditional retry edge	136
7.7 The StateGraph visualised	138
7.8 The complete simple_qa_agent.py	139
7.9 Tool use in LangGraph	141
7.10 Streaming responses	142
7.11 Debugging with LangSmith	142
7.12 Why stateless works on BTP Cloud Foundry	143
7.13 Summary	144
7.14 Checkpoint	145
Chapter 8: The Parallel Hybrid RAG Agent	146
8.1 Why parallel beats sequential	146
8.2 The agent state	147
8.3 The vector chain	148
8.4 The KG chain	151
8.5 The merge function	155
8.6 The orchestrator	157
8.7 The FastAPI /query endpoint	160
8.8 The full request/response contract	162
8.9 Conversation history	163
8.10 Testing: three scenarios that prove hybrid wins	164
8.10.1 The KG wins: precise structured facts	164
8.10.2 The vector wins: narrative safety procedures	164
8.10.3 Both contribute: a complex combined question	165
8.11 Monitoring chain performance	165
8.12 Summary	166
8.13 Checkpoint	167

Chapter 9: The Multi-Agent Supervisor Pattern	168
9.1 The complexity ceiling	168
9.2 The supervisor pattern	169
9.3 The four specialist agents for MSDS	170
9.3.1 HazardAgent	170
9.3.2 ComplianceAgent	171
9.3.3 SafetyAgent	171
9.3.4 SummaryAgent	171
9.4 Shared state for the supervisor graph	171
9.5 The supervisor node	172
9.6 The specialist agents	174
9.7 The summary agent	176
9.8 Building the supervisor graph	178
9.9 When to use the supervisor vs the direct agent	179
9.10 The /query-advanced endpoint	180
9.11 Testing: a question that requires all three specialists	181
9.12 Summary	182
9.13 Checkpoint	183

Front Matter

Agentic Hybrid RAG on SAP BTP

A Hands-On Guide with LangGraph, HANA Cloud, and Google Vertex AI

Title Page

Agentic Hybrid RAG on SAP BTP *A Hands-On Guide with LangGraph, HANA Cloud, and Google Vertex AI*

Author: Sriram Rokkam

First Edition

Published on Amazon KDP ISBN: [To be assigned by Amazon KDP]

Copyright

Copyright © 2026 Sriram Rokkam. All rights reserved.

No part of this publication may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the author, except in the case of brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law.

The code samples in this book are released under the MIT License and are freely available at: <https://github.com/sriramrokkam/agent-ai-sap-btp-book>

SAP, SAP BTP, SAP HANA, SAP CAP, and Joule are trademarks or registered trademarks of SAP SE. Google Cloud, Vertex AI, and Gemini are trademarks of Google LLC. All other trademarks are the property of their respective owners. The author is not affiliated with SAP SE or Google LLC.

Dedication

[To be added by Sriram Rokkam]

Foreword

[To be written by a colleague, SAP contact, or industry peer after the book is complete. 300–500 words. Suggested topics: why Agentic AI matters for SAP customers today, why the BTP + Vertex AI approach is relevant, and why the author is credible on this subject.]

— [Name], [Title], [Company]

Preface

[To be written by Sriram Rokkam in his own voice. Suggested content below — edit freely, this is your personal story.]

I have spent years working at the intersection of SAP and cloud platforms, watching enterprises struggle with the same problem in different forms: they have extraordinary data — decades of business transactions, master data, documents, and domain knowledge — but they cannot easily ask questions of it.

The arrival of large language models changed what was possible. Suddenly, the gap between a business question and a data answer felt bridgeable. But every proof of concept I saw had the same flaw: it worked for demos and broke for production. The LLM hallucinated facts. The retrieval missed precise structured answers. The architecture did not fit the SAP ecosystem that customers already had invested in.

This book is my answer to that problem.

It builds a real system — not a toy — that combines two retrieval strategies that together handle what neither handles alone. It runs on platforms that SAP developers already know: SAP BTP, SAP HANA Cloud, and CAP Node.js. It uses Google Vertex AI not because it is the only option, but because it is excellent, well-documented, and free to get started

with.

Every line of code in this book has been written and tested. Every architecture decision has a reason. Where I made a tradeoff, I say so.

My hope is that you finish this book with a working system deployed on SAP BTP, a clear mental model of how agentic AI actually works, and the confidence to take these patterns into your own projects.

Let us build something real.

Sriram Rokkam *May 2026*

Who This Book Is For

This book is written for:

- **SAP developers and architects** familiar with SAP BTP, CAP, or ABAP who want to build AI-powered applications on the platform they already know
- **AI and ML engineers** working in SAP customer environments who need to understand the SAP data layer and how to integrate with it
- **Technical leads and solution architects** evaluating agentic AI patterns for SAP projects and looking for a production-ready reference architecture
- **Full-stack developers** who have experimented with LLMs and chatbots and are ready to go beyond simple Q&A into multi-step agentic systems
- **BTP developers** who want hands-on experience with SAP HANA Cloud's vector engine and SPARQL knowledge graph capabilities

If you work with SAP systems and want to build AI agents that reason over your enterprise data — this book is for you.

Who This Book Is NOT For

This book is not the right fit if you are:

- **Looking for a general machine learning or deep learning introduction.** We do not cover neural networks, training models, or ML theory. This book is about

applying LLMs to SAP problems, not building LLMs.

- **An ABAP developer with no Python experience.** The agent layer is written in Python. You do not need to be an expert, but you should be comfortable reading and writing basic Python functions, classes, and pip packages before starting Chapter 3.
- **Looking for a no-code or low-code AI solution.** We write real code throughout. Every component is built from first principles so you understand exactly what it does and why.
- **Expecting SAP AI Core as the primary AI backend.** We use Google Vertex AI (Gemini + text-embedding-004) because it is available on the free trial. SAP AI Core is covered in Appendix E for readers who have an enterprise subscription.
- **New to cloud development entirely.** You should be comfortable with the concept of cloud services, REST APIs, and deploying applications. Chapter 2 walks through all the setup steps, but it assumes you have used a terminal before.

Prerequisites

Technical Knowledge

The following knowledge is assumed. You do not need to be an expert — a working familiarity is enough.

Topic	Level Required	Where to Learn if Needed
Python	Basic — functions, classes, pip, venv	python.org/about/gettingstarted
JavaScript / Node.js	Basic — enough to read CAP handlers	nodejs.dev/learn
SAP BTP concepts	Familiar — subaccounts, CF, services	learning.sap.com
REST APIs	Basic — HTTP methods, JSON	Any REST API tutorial
SQL	Basic — SELECT, INSERT	W3Schools SQL

NOT Required

You do not need prior knowledge of: - Machine learning theory or mathematics - LangChain or LangGraph (we introduce it from scratch in Chapter 6) - RDF, SPARQL, or knowledge graphs (introduced from scratch in Chapter 4) - SAP AI Core (covered optionally in Appendix E) - ABAP development - Docker or Kubernetes

Tools and Accounts (All Free)

All tools and accounts are free. Chapter 2 walks through every setup step.

Tool / Account	Purpose	Cost
SAP BTP Trial	Runtime, HANA Cloud, CAP deployment	Free
GCP Account	Vertex AI API (Gemini + embeddings)	Free (\$300 credit)
Python 3.11+	Agent development	Free
Node.js 20+	CAP frontend development	Free
CF CLI	Deploy to BTP Cloud Foundry	Free
CDS CLI (@sap/cds-dk)	CAP development	Free
VS Code	Code editor	Free

How to Use This Book

Reading Cover to Cover (Recommended)

Each chapter builds directly on the previous one. The system grows chapter by chapter — by Chapter 10 you have a fully deployed, working Hybrid RAG Agent on SAP BTP. Reading linearly gives you the full progression from concept to production.

As a Reference

If you already have some of the pieces (a HANA Cloud instance, some LangGraph experience), jump to the chapter you need. Each chapter opens with a summary of what it assumes you already have from previous chapters.

Chapter Structure

Every chapter in this book follows the same pattern:

1. **Opening question** — the single question this chapter answers
2. **Concept** — the idea explained in plain language before any code
3. **Code** — we build the component step by step
4. **Explanation** — what each part does and why it is designed that way
5. **Test** — how to verify it works before moving on
6. **Summary** — what we built and what comes next
7. **Checkpoint** — a checklist before starting the next chapter

The Code

All code is in the companion GitHub repository:

<https://github.com/sriramrokkam/agent-ai-sap-btp-book>

Each chapter has a corresponding folder in the repo. If you get stuck, the repo has the complete working code for reference.

Conventions Used in This Book

Code Blocks

```
1  # Python code appears in blocks like this
2  def example():
3      return "this is a code example"
```

```
1  // JavaScript/Node.js code looks like this
2  const example = () => "this is a code example";
```

```
1 # Terminal commands look like this
2 cf push my-app
```

```
1 -- SQL queries look like this
2 SELECT * FROM MY_TABLE;
```

Callout Boxes

Note: Additional context or clarification that is helpful but not critical to follow along.

Important: Something you must do or understand before continuing. Skipping this will cause problems later.

Warning: A common mistake or pitfall. Pay attention here.

Tip: A shortcut, best practice, or time-saving suggestion.

File Paths

File paths are shown relative to the project root: - `agents/srv/hdb_srv.py` — the HANA connection service in the agents module - `cap-srv/db/schema.cds` — the CDS data model

Screenshots

Screenshots show the state of the UI at the time of writing. SAP BTP and GCP update their interfaces regularly. If a screen looks slightly different, the underlying action is the same — use the search or navigation to find the equivalent option.

[SCREENSHOT: description] markers in this draft will be replaced with actual screenshots in the final published version.

Acknowledgements

[To be added by Sriram Rokkam.]

Suggested: colleagues who reviewed chapters, the SAP and Google Cloud developer

communities, anyone who provided feedback or inspiration.]

About the Author

Sriram Rokkam is a [title] with [X] years of experience building enterprise applications on SAP BTP and Google Cloud Platform.

[Complete this section in your own words — your current role, notable projects, SAP community involvement, LinkedIn/GitHub handles.]

End of Front Matter — Chapter 1 begins on the next page.

Chapter 1: Welcome to the Agentic Era

There is a particular moment in every technology cycle when the language we use to describe the work changes. We stop talking about “models” and start talking about “agents.” We stop asking “what can it generate?” and start asking “what can it do?” We stop measuring tokens per second and start measuring tasks per hour. That moment, for enterprise AI, has arrived.

If you are reading this book, you have probably already noticed the shift. Your inbox is full of vendor pitches for “agentic platforms.” Your colleagues are forwarding LinkedIn posts about autonomous workflows. Your CTO has asked, in a meeting that ran fifteen minutes too long, whether the team has a strategy for “AI agents.” And underneath all of that noise, you have a quieter, more pragmatic question: *What does this actually mean for the systems I build, and how do I build one that works?*

This book is an answer to that question. Over the next several chapters, we will build, end to end, a working agentic system that solves a real enterprise problem on SAP Business Technology Platform (SAP BTP) and Google Cloud. But before we touch a single line of code, we need to agree on what we are building and why. That is the work of this chapter.

1.1 From Models That Answer to Systems That Act

A useful way to understand the agentic era is to remember what it replaced.

For most of the last decade, building anything intelligent into an enterprise application meant assembling a small army. You needed data engineers to clean and pipe the training data, data scientists to choose and tune a model, ML engineers to package the artifact, and platform engineers to deploy it behind an API. Each model solved exactly one problem — classifying invoices, forecasting demand, recommending products — and each new problem demanded a new project, a new dataset, and a new round of training. The barrier to entry was high, the time to value was long, and the resulting models were brittle in the ways that custom ML is always brittle: a small shift in input distribution and the predictions quietly drifted off the rails.

Foundation models changed the economics. A single API call to a large language model could now classify, summarize, extract, translate, draft, and explain — without any train-

ing, without any infrastructure, without any of the traditional ML lifecycle. For application developers, this was liberating. For the first time, a feature that would once have required a six-month project could be prototyped in an afternoon.

But there was a catch, and anyone who built a serious feature on top of a raw LLM eventually ran into it. A foundation model, however capable, is fundamentally *passive*. It receives a prompt and returns a completion. It does not look up data it does not already have. It does not call your APIs. It does not check whether its answer is true. It does not stop and think when it is uncertain. It does not remember what you told it ten minutes ago. It produces fluent text on demand, and that is both its strength and its limit.

The agentic leap is the move from passive completion to active reasoning. An agent is what you get when you take a foundation model and surround it with the missing pieces: tools it can call, memory it can read and write, plans it can construct and revise, and a loop that keeps running until a goal is met. The model becomes the reasoning engine; the surrounding system gives it hands, eyes, and a sense of purpose.

The clearest test of whether a system is genuinely agentic is the one Michael Albada offers in *Building Applications with AI Agents*: does it demonstrate real decision making, or is it following a static script? A workflow that always retrieves three documents, always passes them to the same prompt, and always returns the model's first response is not an agent. It is a deterministic pipeline with an LLM somewhere in the middle. An agent, by contrast, decides — at runtime, based on what it sees — whether to retrieve more, whether to ask a clarifying question, whether to invoke a different tool, or whether to stop and admit it does not know.

That distinction matters because it changes what we have to design for. With a deterministic pipeline, the engineering problem is mostly about plumbing. With an agent, the engineering problem is about *behavior under uncertainty*: how the system reasons, when it acts, what it can be trusted to do without supervision, and how we observe and correct it when it goes wrong.

1.2 A Spectrum, Not a Switch

It is tempting to treat “agent” as a binary label — either a system is one or it is not. In practice, the more honest picture is a spectrum, and most enterprise systems live somewhere in the middle of it.

At one end is traditional code: deterministic, explainable, fast, and incapable of handling anything its author did not anticipate. A SQL query is at this end of the spectrum.

A step further along is the deterministic LLM workflow. The structure is fixed — retrieve, prompt, return — but the model contributes flexibility within each step. A summarization service that always reads the same field from the same record and always asks the model for the same kind of summary is a deterministic workflow.

Further still is retrieval-augmented generation, or RAG. Here the system grounds the model in external knowledge: the user asks a question, the system retrieves relevant documents, the model answers using those documents as context. RAG is where most production “AI assistants” live today. It is more flexible than a static workflow because the retrieval step adapts to the question, but the control flow is still essentially fixed.

At the far end of the spectrum is the autonomous agent: a system that decides which tools to call, in what order, and when to stop, often invoking the model multiple times in a single turn to plan, act, observe, and replan. Autonomous agents are the most adaptable systems we know how to build, but they pay for that adaptability in latency, cost, and explainability. Every additional reasoning step is another LLM call, another opportunity for the model to wander off course, another thing the auditor will want to see traced.

The system we build in this book sits deliberately in the middle of this spectrum. It is more than a fixed RAG pipeline — it makes runtime decisions about which retrieval strategy to use and how to combine the results — but it is less than a fully autonomous agent. We will call it an *agentic RAG* system, and the choice to stay in the middle is intentional. For the kind of enterprise problem we are solving, the middle of the spectrum is where the trust, the cost, and the capability all balance.

The table below sketches how these four points compare on the dimensions that matter for enterprise deployment.

	Traditional Code	Deterministic LLM Workflow	RAG System	Agentic RAG (this book)	Autonomous Agent
Input structure	Strict schema	Mostly structured	Natural language	Natural language	Natural language
Adaptability	None	Low	Medium	Medium-high	High
Explainability	High	High	High	High	Lower
Latency	Milliseconds	Seconds	Seconds	Seconds	Tens of seconds

	Traditional	Deterministic	RAG	Agentic RAG	Autonomous
Dimension	Code	LLM Workflow	System	(this book)	Agent
Failure mode	Hard error	Wrong but consistent	Plausible but stale	Honest “I don’t know”	Confidently off-task

The last row is the one that matters most for the enterprise. A system that fails loudly, predictably, and honestly is far more useful than one that fails in confident, fluent prose.

1.3 The Trouble With General-Purpose Assistants

Most of the AI assistants the public has met so far are general-purpose. They are designed to answer almost anything, to anyone, in any context. That generality is a remarkable engineering achievement, and it makes for an excellent demo. It also makes them the wrong shape for most enterprise problems.

Consider a question that any chemical safety officer might reasonably ask of an internal assistant: “*What is the GHS hazard classification for acetone?*”

A general-purpose assistant will produce a fluent, confident answer. The answer will probably be correct, because acetone is well-documented in the public corpus the model was trained on. But the assistant cannot tell you *which version* of the GHS revision its answer reflects, *which jurisdiction* it applies to, or *whether your company’s own safety data sheet for acetone* — the document that actually governs your operations — agrees with what it just told you. If the public corpus is six months out of date, or if your supplier has issued a revised data sheet, the assistant will not know, and it will not say so. It will simply produce its best guess, indistinguishable in tone from a verified fact.

In a casual context, this is fine. In a regulated context, it is the kind of problem that ends careers. “Plausible-sounding” is not the standard a manufacturing safety system can be held to. The standard is *verifiable*, *structured*, and *traceable to a source*.

This is the gap that domain-specific agents exist to close. Albada categorizes agents into several types in *Building Applications with AI Agents* — business-task agents, conversational agents, research agents, analytics agents, developer agents, browser-using agents — but the one we care about most in this book is the domain-specific agent. In his words, these are agents “tuned for specialized professional domains, such as le-

gal (Harvey), medical (Hippocratic AI), or finance agents,” combining “domain-specific knowledge with structured workflows to deliver targeted, expert-level assistance.”

The defining characteristic of a domain-specific agent is not that it knows more than a general assistant. It is that it knows *less, but better*. It operates within a bounded knowledge domain — your documents, your ontology, your facts — and it refuses to step outside that boundary. When asked about acetone, a well-built domain-specific MSDS agent will not lean on a half-remembered training-data tidbit. It will retrieve the actual safety data sheet your company maintains for acetone, cite the section it drew from, and tell you when the document was last revised. If the document does not contain the answer, the agent says so. That last part — the willingness to say “I don’t have that information” — is the single feature that most distinguishes a trustworthy enterprise agent from a hallucinating chatbot.

There are three properties that make domain-specific agents the right shape for enterprise work, and each will reappear throughout this book.

The first is **bounded knowledge**. The agent’s universe is the corpus you give it, plus the structured facts you have curated. It does not invent codes, classifications, or part numbers because it has no source for invented ones. It can be wrong, but only in ways that trace back to a real document.

The second is **structured plus semantic retrieval**. Some questions are best answered by a precise lookup against structured data — “what is the GHS code for acetone?” is one of them. Other questions are best answered by finding semantically similar passages in unstructured prose — “what should I do if a worker spills acetone on bare skin?” is one of those. A domain-specific agent uses both.

The third is **auditability**. Every answer the agent produces can be traced to a specific document chunk or a specific knowledge-graph triple. The auditor can ask “where did this come from?” and get an honest, specific answer. In a regulated industry, this is not a nice-to-have. It is the price of admission.

Note: Domain-specific does not mean small. Harvey serves global law firms; Hippocratic AI is deployed across major hospital systems. The boundary is not on capacity but on scope: these agents are deep in one domain rather than shallow across many. Depth is what makes them trustworthy.

1.4 Why Hybrid Retrieval Is the Right Engine

Once you accept that a domain-specific agent should retrieve before it generates, the next question is *how* it should retrieve. This is where most architectures take a wrong turn.

The default answer in 2024 and early 2025 was: vector search, full stop. Embed your documents, store the vectors, and at query time embed the question and find the nearest chunks. Vector search is genuinely powerful. It is forgiving — a question phrased one way will retrieve a passage phrased another way, because both map to similar regions of embedding space. It handles natural language gracefully. It is fast at scale. For a long stretch, it looked like the universal retrieval primitive.

But anyone who has tried to build a real domain-specific assistant on vector search alone has run into the same wall: vector search is bad at facts. Asked for the GHS hazard classification of acetone, a vector search will return a chunk that *talks about* acetone and hazard classifications, but the agent then has to read prose and extract a code. Sometimes the model extracts correctly. Sometimes it mixes up acetone with a similarly-named solvent that appeared in the same document. Sometimes the relevant chunk did not get retrieved at all because the question phrasing was too terse to embed well. For narrative reasoning, vector search is excellent. For symbolic facts, it is the wrong tool.

The complementary tool is the knowledge graph. A knowledge graph stores facts as triples — (*acetone*, *hasGHSClassification*, *H225*) — and answers questions by traversing relationships rather than by similarity. Asked the same hazard-classification question, a graph-backed retrieval returns the exact code, with a citation to the document that asserted it. The graph cannot hallucinate a classification it has never been told, because it does not generate; it looks up. For symbolic facts, this is exactly the property we want.

Neither retrieval strategy alone is enough. Vector search wins on fuzzy, narrative, semantic questions. Knowledge graphs win on precise, structured, symbolic questions. A real enterprise corpus contains both kinds of content, and a real user asks both kinds of questions, often in the same conversation.

Hybrid RAG runs both retrieval strategies in parallel and lets the agent decide what to do with the results. The decision logic is simpler than it sounds.

When both retrievals return relevant content, the agent synthesizes — using the graph

to anchor the structured facts and the vector results to flesh out the narrative context. The user gets an answer that is both precise and readable.

When only one retrieval returns relevant content, the agent uses that one directly. A pure factual question may need only the graph. A pure narrative question may need only the vectors.

When neither retrieval returns relevant content, the agent says so. This is the moment that separates trustworthy systems from theatrical ones. A general assistant will fall back on its training data and produce a confident guess. A well-built hybrid agent will say “I don’t have that in my sources” and stop.

This pattern — graph plus vectors, parallel retrieval, honest fallback — is the architectural spine of every system we build in this book. It is also, not coincidentally, the architecture that SAP itself is converging on for its agentic infrastructure, for the same reasons we have just walked through.

1.5 What We Will Build

The running example for the rest of the book is a domain-specific agent for **Material Safety Data Sheets**, or MSDS. If you have not encountered MSDS documents before, they are the regulatory artifacts that govern how chemical substances are handled, stored, transported, and disposed of. Every manufacturer, pharmaceutical company, chemical processor, and laboratory on Earth maintains a library of them. Most of those libraries live as PDFs in shared drives, with metadata in spreadsheets, and the people who actually need answers — operators on a shop floor, technicians in a warehouse, a nurse responding to a spill — typically cannot find the information they need fast enough to act on it.

MSDS documents are an almost unfair example for hybrid RAG, because they contain both kinds of content in equal measure. On one page, you will find dense narrative prose: how to ventilate a room after a spill, what the first-aid steps are for skin contact, how to dispose of contaminated absorbent material. On the next page, you will find precise structured data: GHS hazard codes, signal words, exposure limits, flash points, classification categories. The narrative pages are perfect for vector search. The structured pages are perfect for a knowledge graph. A real user — say, a safety officer responding to a question from the shop floor — will mix questions about both, often in a single sentence.

Over the course of this book we will build a system that does the following.

It ingests MSDS PDFs end to end, automatically extracting both the prose passages and the structured facts. The prose passages are chunked, embedded with a vector model, and stored in SAP HANA Cloud's vector engine. The structured facts are extracted into RDF triples and stored in HANA's graph engine. The same document populates both stores in a single ingestion run.

It answers questions using both retrieval strategies in parallel. A FastAPI service orchestrated with LangGraph runs the graph query and the vector search at the same time, gathers the results, and routes them through Gemini 2.5 Flash on Vertex AI for synthesis. The agent decides, per query, whether to use one retrieval, both, or neither.

It exposes the answers through a CAP Node.js OData V4 service with a Fiori Elements user interface, so a safety officer can ask a question from a familiar SAP-style screen and see a cited, structured response.

It runs entirely on the SAP BTP free trial and the Google Cloud Platform \$300 free credit. There is no paid step in the build. By the end of the book, you will have a working system, deployed to a real cloud, that you built without spending a dollar.

The architecture, the patterns, and the engineering choices we make along the way generalize far beyond MSDS. Substitute legal contracts for safety data sheets and you have the skeleton of a Harvey-style legal agent. Substitute clinical guidelines and you have something that looks a lot like Hippocratic AI. Substitute equipment maintenance manuals and you have a field-service agent. The domain changes. The pattern does not.

1.6 Why This Moment, and Why for SAP Developers

In May 2026, at SAP Sapphire, SAP announced more than two hundred specialized Joule agents shipping across the application portfolio over the following year. Read carefully, that is not an announcement of two hundred general-purpose chatbots. It is an announcement of two hundred *domain-specific agents* — for finance close, for procurement, for HR transactions, for supply-chain exceptions, for sustainability reporting. SAP has bet the next chapter of its product strategy on the same pattern this book teaches.

That is not a coincidence, and it is not marketing. It is what happens when a company with the deepest enterprise data graph in the world looks at the agentic landscape and asks which patterns will actually work inside its customers' regulated, audited, mission-

critical processes. The answer is the answer this book has been making the case for: bounded domains, hybrid retrieval, traceable answers, and a refusal to hallucinate.

For an SAP developer, this is the most interesting moment your career has handed you so far. The skills required to build these agents — vector search, knowledge graphs, LangGraph orchestration, CAP integration, BTP deployment, Document grounding — are not yet common. They are exactly the skills SAP itself is hiring for, that SAP partners are bidding on, and that SAP customers are about to need at scale. The patterns are early enough to learn, and important enough to be worth learning.

The chapters that follow are designed to make that learning concrete. We will not stay in the conceptual register much longer. By the end of Chapter 3, you will be running working code against a real HANA Cloud vector index. By the end of the book, you will have shipped a domain-specific MSDS agent with both retrieval engines, a real UI, and a real deployment.

1.7 Summary

The agentic era is not a marketing slogan; it is a real shift in what software can do. Foundation models gave us systems that could understand language. Tools, memory, and planning give us systems that can act on it. The transition from passive completion to active reasoning is the one engineering shift that matters most for enterprise applications over the next several years.

General-purpose assistants are impressive demos, but they are the wrong shape for regulated, high-stakes enterprise work. Domain-specific agents — bounded in scope, deep in knowledge, auditable in operation — are the shape that fits.

The retrieval engine that makes domain-specific agents trustworthy is hybrid. Vector search alone fails on facts. Knowledge graphs alone fail on narrative. Run both in parallel, synthesize when both return, defer to whichever returns when only one does, and admit ignorance when neither does. That is the architecture we will build.

The system we build to make this concrete is an MSDS agent — a domain-specific assistant for safety data sheets that runs on SAP HANA Cloud, LangGraph, and Vertex AI, exposed through a CAP service and a Fiori UI, deployed entirely on free tiers.

In Chapter 2, we examine why SAP developers are uniquely positioned to build these systems — and set up the platform we will use throughout the rest of the book.

Note: The framework for thinking about agent types in this chapter draws on Michael Albada’s *Building Applications with AI Agents* (O’Reilly, 2026), which provides an excellent taxonomy of agentic systems. Our focus on domain-specific agents builds directly on that foundation.

Chapter 2: What Is Agentic AI — And Why SAP Developers Should Care

2.1 Beyond the Chatbot

Most developers encounter AI for the first time as a chatbot. You type a question, the model replies. It is impressive, but it is fundamentally passive. The model waits for input, responds, and stops. It has no memory of what happened before, no ability to take action in the world, and no way to break a complex task into steps and execute them one by one.

Agentic AI changes this entirely.

An AI agent is a system where a large language model (LLM) does not just respond — it **reasons, plans, and acts**. It can call tools, retrieve information from databases, invoke APIs, hand work off to other agents, and loop back to verify its own output. The agent decides what to do next based on what it has learned so far. It is not following a fixed script. It is reasoning dynamically toward a goal.

This shift — from chatbot to agent — is the most important development in applied AI since the transformer architecture itself.

2.2 The Four Capabilities That Define an Agent

An AI agent is defined by four capabilities that a plain LLM call does not have:

- 1. Tool Use** The agent can call external functions — search a database, invoke an API, run a calculation, read a file. The LLM decides when and how to use each tool based on the question it is trying to answer.
- 2. Memory** The agent maintains state across steps. This can be short-term (what happened in this conversation) or long-term (facts stored in a vector database or knowledge graph that persist across sessions).
- 3. Planning** The agent can decompose a complex goal into sub-tasks, execute them in sequence or in parallel, and adapt the plan when intermediate results are unexpected.
- 4. Multi-Agent Collaboration** Complex tasks can be delegated. A supervisor agent

breaks work into pieces and assigns each piece to a specialist agent — one for database queries, one for calculations, one for document retrieval. The supervisor collects results and synthesizes a final answer.

2.3 Why This Matters for SAP Developers

SAP developers work in an ecosystem that is rich with structured data, complex business processes, and decades of accumulated knowledge locked in documents, systems, and workflows.

Consider what an agentic system can do in an SAP context:

- A logistics manager asks: *“Which of our open deliveries are at risk of missing the SLA, and what should we do about them?”* — The agent queries SAP S/4HANA OData APIs, retrieves delivery statuses, checks route data, reasons about delays, and proposes actions.
- A procurement analyst asks: *“Summarize the hazard information for all chemicals we ordered last quarter.”* — The agent retrieves material safety data sheets, searches a knowledge graph for GHS classifications, cross-references a vector store of regulatory documents, and synthesizes a compliance summary.
- A developer asks: *“What CAP annotations do I need to add to expose this entity as a Fiori list report?”* — The agent searches internal documentation, reads the CDS schema, and generates the exact annotations needed.

None of these tasks can be answered by a single database query or a single LLM prompt. They require reasoning, retrieval, and action — the three pillars of agentic AI.

SAP’s Strategic Direction: The Autonomous Enterprise

The timing of this book is not accidental. SAP has placed agentic AI at the centre of its entire product strategy.

At **SAP Sapphire 2026 in Orlando**, CEO **Christian Klein** unveiled the company’s most significant AI announcement to date: a unified vision called the **Autonomous Enterprise** — a model where humans and AI collaborate continuously to execute, adapt, and optimise critical business processes. In Klein’s own words:

“For the mission-critical processes of our customers, ‘almost right’ just isn’t

good enough. By uniting SAP Business AI Platform with SAP Autonomous Suite, we anchor AI agents in the business processes, data and governance so they can deliver accurate, compliant and secure outcomes, unlocking new sources of revenue and meaningful cost savings.”

The scale of SAP’s AI buildout is notable:

Milestone	Detail
50+ Joule Assistants	Domain-specific AI assistants across Finance, Supply Chain, Procurement, HR, and Customer Experience
200+ specialised agents	Available for orchestration of precise business tasks across the SAP portfolio
7 Industry AI solutions	Vertical-specific autonomous solutions through the Industry AI initiative
€100 million partner fund	Launched to accelerate partner deployments of AI assistants and agents
SAP Autonomous Suite	A new product suite embedding agents directly into S/4HANA, Ariba, SuccessFactors, and more

SAP also introduced **Joule Studio** — a no-code and pro-code environment for building custom agents on BTP — and **Joule Work**, a conversational interface that gives every business user access to the agent layer. These are precisely the tools this book’s architecture is designed to extend.

SAP’s AI Partnerships and Infrastructure Investments

SAP is not building its AI strategy alone. The company announced a broad coalition of AI partnerships at Sapphire 2026, each serving a distinct role in the stack:

Partner	Role
NVIDIA	Provides OpenShell — the trusted, secure runtime environment for Joule Studio’s agent execution
Anthropic	Foundation model partnership for Claude integration within SAP Business AI Platform
Google Cloud	Strategic AI infrastructure and Gemini model integration
Microsoft	Copilot integration and Azure AI services
Mistral AI & Cohere	European AI model options within the platform
Palantir	Enterprise data and AI operations

The NVIDIA partnership deserves particular attention. NVIDIA’s strength is not just GPUs — it is the full stack from hardware through software frameworks (CUDA, NIM microservices) to the runtime environment for deploying and securing agents at scale. By embedding NVIDIA OpenShell into Joule Studio, SAP gives enterprise customers a hardened, auditable execution environment for agents that touch mission-critical processes. This is not a marketing partnership — it is a direct response to enterprise concerns about agent reliability and security, themes that Albada covers extensively in *Building Applications with AI Agents* (Chapter 12: Protecting Agentic Systems).

SAP also announced acquisitions that signal the depth of its AI commitment. In May 2026, SAP announced plans to acquire **Prior Labs** — a frontier AI research lab based in Europe — with the explicit goal of establishing a “globally leading frontier AI lab” within the company. This follows the acquisition of **Reltio**, a master data management platform, in the same period. Together, these moves signal that SAP intends to control the AI stack from the data layer (Reltio) through the model layer (Prior Labs) to the application layer (Autonomous Suite).

What This Means for You as a Developer

You are not building a side project. You are building on the platform and with the patterns that SAP itself is standardising for the next decade of enterprise software.

The architecture in this book — agents orchestrated with LangGraph, vector search and knowledge graphs on HANA Cloud, CAP Node.js as the API layer, exposed through Joule — maps directly to SAP’s publicly stated direction. The skills you build here translate immediately to the Joule Studio agent development environment, to the SAP Business AI Platform, and to the autonomous enterprise patterns that SAP’s largest customers are now adopting.

The 200+ agents SAP announced at Sapphire 2026 were not built by SAP alone. Partners, customers, and developers on BTP built them. This book shows you how to be one of those builders.

2.4 What We Build in This Book

This book builds a single, coherent, production-grade system called the **Hybrid RAG Agent**. It runs on SAP BTP (free trial) and uses Google Vertex AI for LLM and embedding capabilities.

The system answers questions about documents by combining two retrieval strategies in parallel:

Vector Search finds semantically similar content. When you ask “*What precautions are needed when handling acetone?*”, vector search finds all document chunks that are semantically close to that question — even if they do not use the exact same words.

Knowledge Graph Search finds structured facts. When you ask “*What is the GHS hazard code for acetone?*”, the knowledge graph returns a precise, structured answer via a SPARQL query — no ambiguity, no hallucination.

A LangGraph orchestrator runs both searches in parallel, merges the results, and synthesizes a final answer using Gemini 2.5 Flash. The system is exposed via a CAP Node.js OData V4 API with a Fiori Elements UI, and optionally extended to Joule A2A for enterprise SAP copilot integration.

Figure 2.1 — The Hybrid RAG Agent: user questions enter a FastAPI backend, which fans out to a Vector Chain (HANA REAL_VECTOR + cosine search) and a KG Chain (HANA

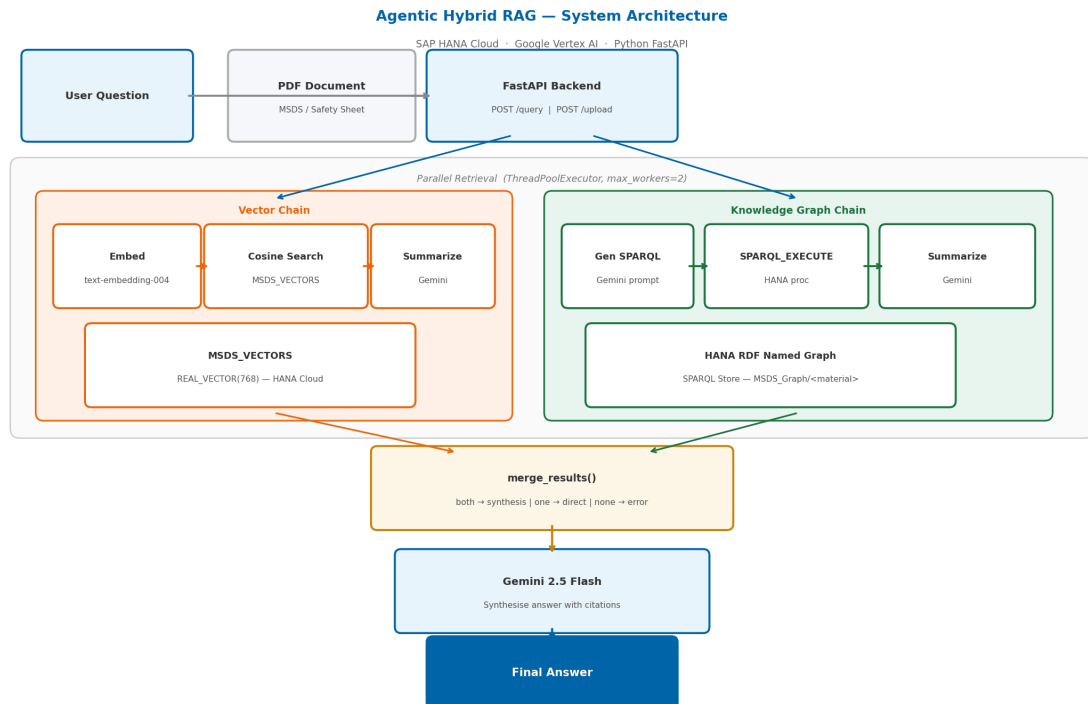


Figure 1: Agentic Hybrid RAG System Architecture

SPARQL/RDF) running in parallel. Both results feed into Gemini 2.5 Flash for synthesis into a final cited answer.

By the time you finish this book, you will have built a fully working system that:

- Ingests PDF documents and builds both a vector store and a knowledge graph automatically
- Answers questions using dual-retrieval hybrid RAG running locally
- Orchestrates multi-step reasoning with LangGraph
- Connects to SAP HANA Cloud (BTP trial) for vector and graph storage
- Calls Google Vertex AI for embeddings and LLM inference
- Exposes everything via CAP OData V4 with a Fiori Elements UI running locally

2.5 Scope of This Book

Before going further, it is important to be explicit about what this book covers and what it deliberately leaves out. Knowing the boundaries upfront will save confusion later.

What This Book Covers

Topic	Coverage
Agentic AI concepts — tool use, memory, planning, multi-agent	Full coverage, Ch 1–2
SAP BTP trial setup — HANA Cloud, CF space, Destination Service	Full coverage, Ch 3
Vector search on HANA Cloud (REAL_VECTOR)	Full coverage, Ch 4
Knowledge graphs on HANA Cloud (SPARQL/RDF)	Full coverage, Ch 5
PDF ingestion pipeline — dual-threaded KG + vector	Full coverage, Ch 6
LangGraph — state, nodes, edges, conditional routing	Full coverage, Ch 7
Hybrid RAG agent — parallel chains, merge, orchestrator	Full coverage, Ch 8
Multi-agent supervisor pattern — specialist agents	Full coverage, Ch 9
CAP Node.js OData V4 service + Fiori Elements UI	Full coverage, Ch 10
Deploying to SAP BTP Cloud Foundry	Full coverage, Ch 11
Joule A2A integration	Appendix D (enterprise only)
SAP AI Core as LLM alternative	Appendix E (enterprise only)

What This Book Does NOT Cover

Production-grade hardening. This book teaches the architecture and implementation patterns. It does not cover production concerns such as rate limiting, multi-tenant isolation, secrets rotation, disaster recovery, or SLA monitoring. Those topics belong in an operations guide, not a developer learning book.

Fine-tuning or training models. All LLM usage in this book is inference-only — calling

hosted models via API. We do not fine-tune Gemini, train embeddings, or modify model weights.

Multi-document cross-referencing. The hybrid RAG agent in this book answers questions about one MSDS document at a time (targeted by material number). Cross-document reasoning is a natural extension but is not covered here.

Streaming responses. The agent returns complete answers synchronously. Streaming token-by-token output to the Fiori UI is a worthwhile enhancement but adds significant frontend complexity that would dilute the core learning objectives.

CI/CD pipelines. Deployment in Chapter 11 uses manual `cf push` and `mbt build && cf deploy` commands. Automating these through GitHub Actions or SAP CI/CD Service is not covered.

Joule and SAP AI Core on free trial. Both require enterprise SAP subscriptions. They are covered in Appendices D and E specifically so readers with enterprise access can follow along, while the main chapters remain accessible to anyone on the free trial.

Local Development is the Primary Mode

Most chapters in this book develop and test everything on your **local machine**, using:

- `uvicorn main:app --reload` for the FastAPI agent service
- `cds serve` for the CAP Node.js service
- Your local HANA Cloud trial instance (which is cloud-hosted but accessed from your laptop)
- The Vertex AI API called directly from your local Python environment

Chapter 11 then shows you how to deploy this same code — unchanged — to BTP Cloud Foundry. The local and deployed architectures are identical; only the credential source and the runtime environment change.

Note: You do not need to complete Chapter 11 to have a working system. Chapters 1–10 produce a fully functional Hybrid RAG Agent running locally against your BTP HANA Cloud trial. Chapter 11 is the production deployment step.

2.6 The Platform Choice: SAP BTP + Google Vertex AI

A recurring question in enterprise AI projects is: which platform? This book makes a deliberate, practical choice.

SAP BTP is the data and integration platform. It provides: - SAP HANA Cloud — the only database that combines vector search and SPARQL knowledge graph storage in a single managed service - Cloud Foundry runtime — free for developers, the same runtime used in production - CAP Node.js — the standard for building SAP-native OData APIs - Destination Service — secure, centralized credential management for external APIs

Google Vertex AI is the AI inference layer. It provides: - Gemini 2.5 Flash — one of the best price-performance LLMs available today - text-embedding-004 — state-of-the-art embeddings for semantic search - \$300 free trial credit — enough to build and test everything in this book at zero cost

SAP AI Core is deliberately excluded from the main examples. It is a viable alternative for organizations that require all components on BTP, but it is a paid service that is not available on the free trial. We mention it where relevant and the architecture is designed so that swapping Vertex AI for AI Core requires changing only the LLM client — nothing else.

2.7 What You Need to Follow Along

To build everything in this book you need:

Requirement	Where to Get It	Cost
SAP BTP Trial account	account.hanatrial.ondemand.com	Free
SAP HANA Cloud instance	BTP Trial cockpit	Free (30 days)
GCP account	cloud.google.com/free	Free (\$300 credit)
Vertex AI API enabled	GCP Console	Free (within credit)
Python 3.11+	python.org	Free
Node.js 20+	nodejs.org	Free
CF CLI	CF documentation	Free
CDS CLI (@sap/cds)	npm	Free
VS Code	code.visualstudio.com	Free

Chapter 3 walks through every setup step with screenshots. If you already have a BTP trial and a GCP account, Chapter 3 will take about 30 minutes.

2.8 Summary

- Agentic AI extends LLMs with tool use, memory, planning, and multi-agent collaboration
- SAP developers have a unique opportunity — rich enterprise data, complex processes, and a platform (BTP) purpose-built for hybrid AI architectures
- SAP itself is betting on domain-specific agents: 200+ specialist agents announced at Sapphire 2026, the Autonomous Enterprise vision, and the Joule Studio development environment
- This book builds a Hybrid RAG Agent: vector search + knowledge graph + Lang-Graph orchestration on SAP BTP
- The full stack is free to build and test using BTP trial + GCP free credit
- SAP AI Core is not required — Vertex AI is called as an external API from BTP Cloud Foundry
- Local development is the primary mode — Chapter 11 handles deployment once the system is working

In Chapter 3, we set up the platform — BTP trial, HANA Cloud, GCP project, and Vertex AI credentials — so you have a working environment before we write a single line of code.

Chapter 3: Platform Setup — SAP BTP Trial + Google Vertex AI

3.1 The Goal of This Chapter

Before we write a single line of agent code, we need a working environment. This chapter gets you there.

By the end of this chapter, you will have:

- A running SAP BTP trial account with a HANA Cloud instance
- A GCP project with Vertex AI enabled and a service account ready
- A BTP Destination that securely connects your BTP applications to Vertex AI
- A local development environment with all required tools installed
- A confirmed, working API call to Gemini 2.5 Flash from your local machine

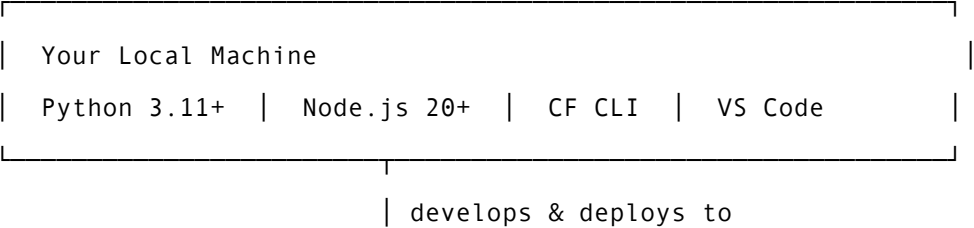
This is pure setup. There is no agent code here. But every chapter that follows depends on this foundation being solid. Rushing this chapter and skipping verification steps is the most common cause of confusing errors later. Take the 45 minutes. Do it properly.

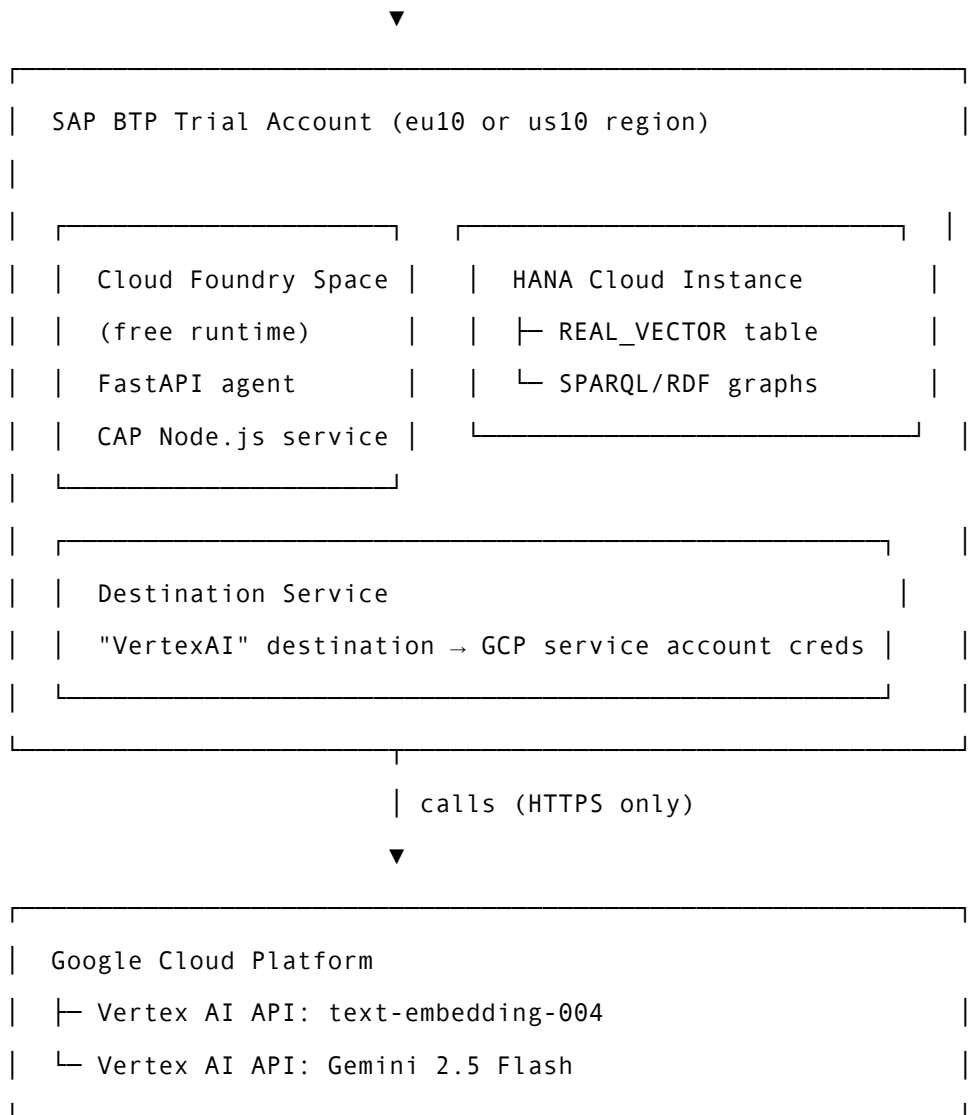
Cost reminder: Everything in this chapter is free. SAP BTP trial is free. GCP provides \$300 credit on signup. The Vertex AI API calls in this chapter will cost cents — comfortably within the trial credit.

3.2 Understanding What We Are Setting Up

Before clicking through consoles, it helps to understand the shape of what we are building.

Our system has three infrastructure components:





SAP HANA Cloud is the **knowledge layer** — it stores both our vector embeddings and our RDF knowledge graph in a single managed service. No other database on the market provides both `REAL_VECTOR` column types (for cosine similarity search) and a native SPARQL execution engine in one service. This is a significant architectural advantage — one connection, one credential, two retrieval strategies.

Google Vertex AI is the **AI inference layer** — it is an external HTTPS API that our BTP application calls like any other REST service. Gemini 2.5 Flash handles both our LLM tasks (SPARQL generation, answer synthesis) and we use `text-embedding-004` for generating embeddings. The BTP Destination Service stores the GCP credentials securely — the service account JSON key never lives in application code or environment variables directly.

With this picture in mind, let us set it up.

3.3 SAP BTP Trial Account

Signing Up

Navigate to `account.hanatrial.ondemand.com` and click **Try for Free**.

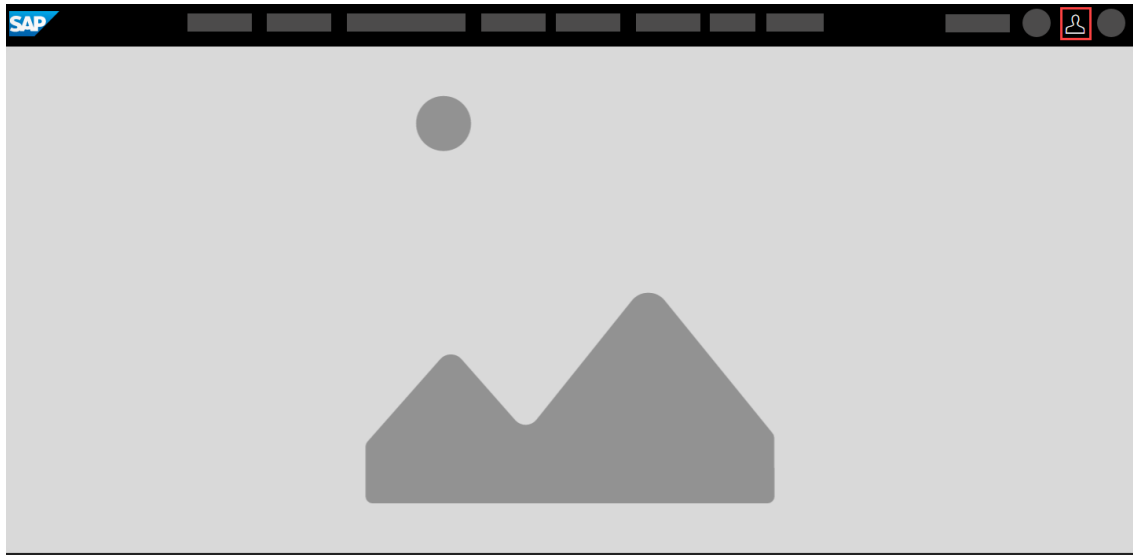


Figure: SAP BTP Trial — navigate to `account.hanatrial.ondemand.com` and click Try for Free

Fill in your details. Use a personal email address if possible — corporate email addresses sometimes cause issues with trial account activation. After verifying your email, BTP will automatically provision a trial account in either the EU10 (Frankfurt) or US10 (Virginia) region depending on your location.

Note: The region matters. Your HANA Cloud instance must be in the same region as your Cloud Foundry space. BTP assigns both automatically — do not change the region during signup.

Once logged in, you will land on the **BTP Cockpit** — the central control panel for all BTP services.

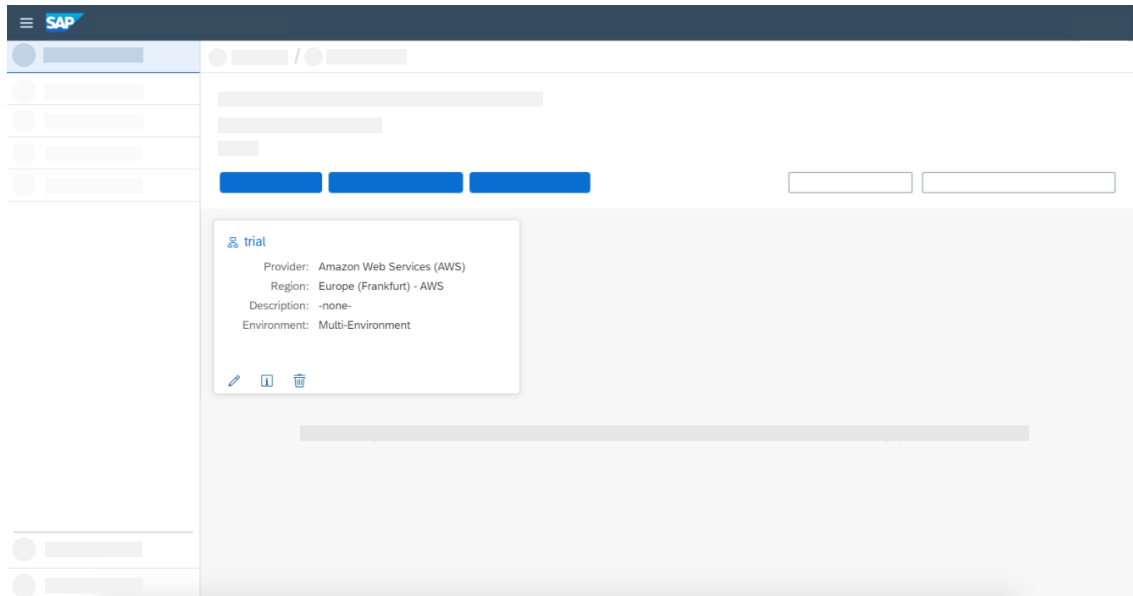


Figure: SAP BTP Cockpit home screen showing your Global Account and trial subaccount

Navigating to Your Trial Subaccount

The BTP Cockpit has a hierarchy: Global Account → Subaccount → Space. Your trial comes with one subaccount called “trial.” Click it.

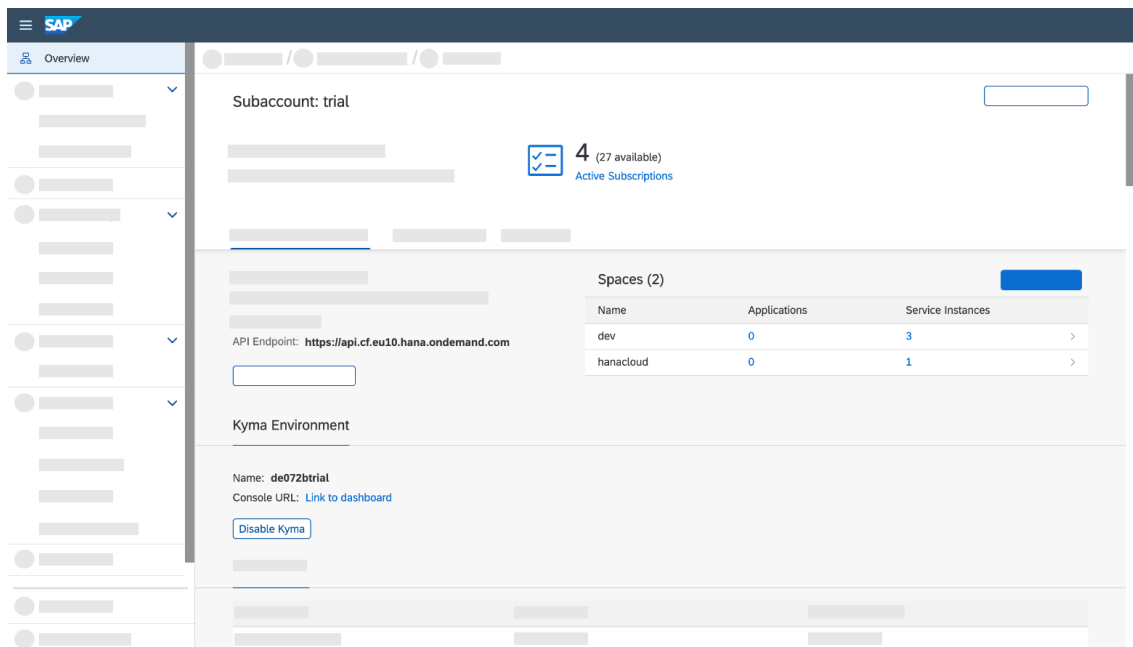


Figure: BTP trial subaccount overview page

Inside the subaccount, you will see your **Cloud Foundry environment**. Note your CF API endpoint — it will look like `https://api.cf.eu10.hana.ondemand.com`. You will need this later.

Click **Cloud Foundry** ▢ **Spaces**. You should see a space called “dev.” This is where you will deploy your applications.

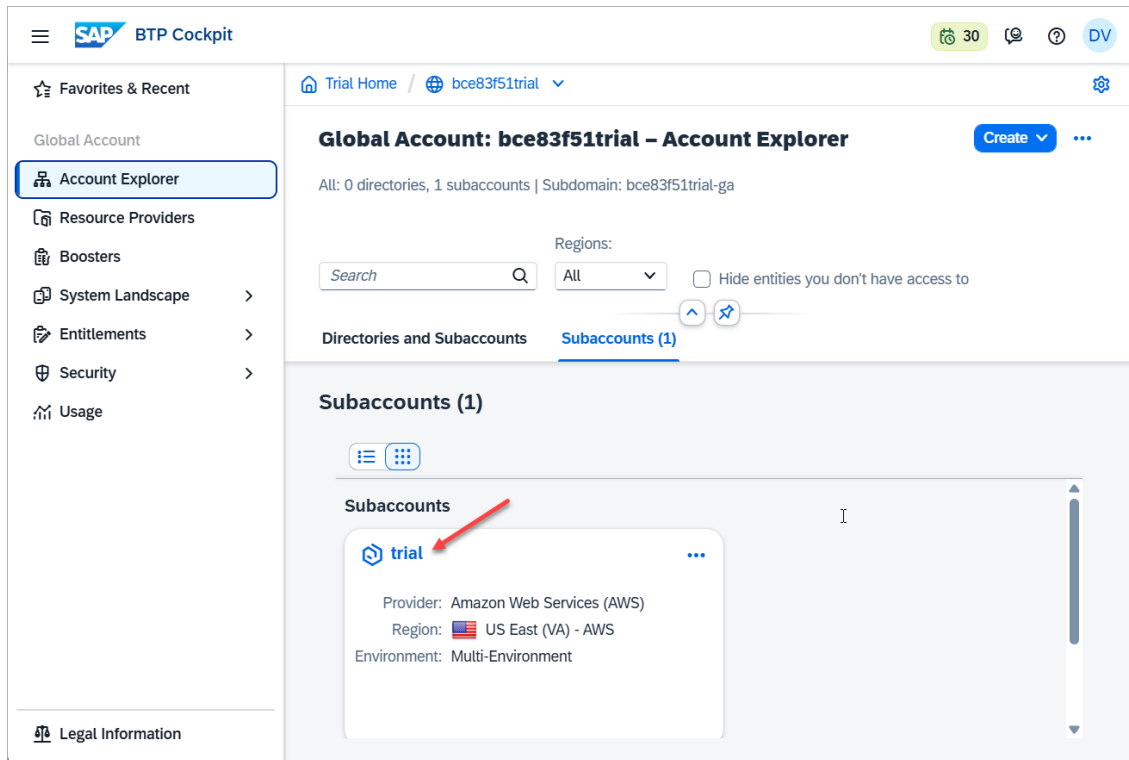


Figure: Cloud Foundry space view — note your org name and the “dev” space

Write these down now: - CF API Endpoint: `https://api.cf.<region>.hana.ondemand.com`
 - CF Org: `<your-trial-org>` - CF Space: `dev`

3.4 Provisioning SAP HANA Cloud

HANA Cloud is provisioned from the BTP Cockpit. This is a one-time setup that takes about 10–15 minutes.

Creating the HANA Cloud Instance

From your trial subaccount, go to **Services** ▢ **Service Marketplace**. Search for “HANA Cloud.”

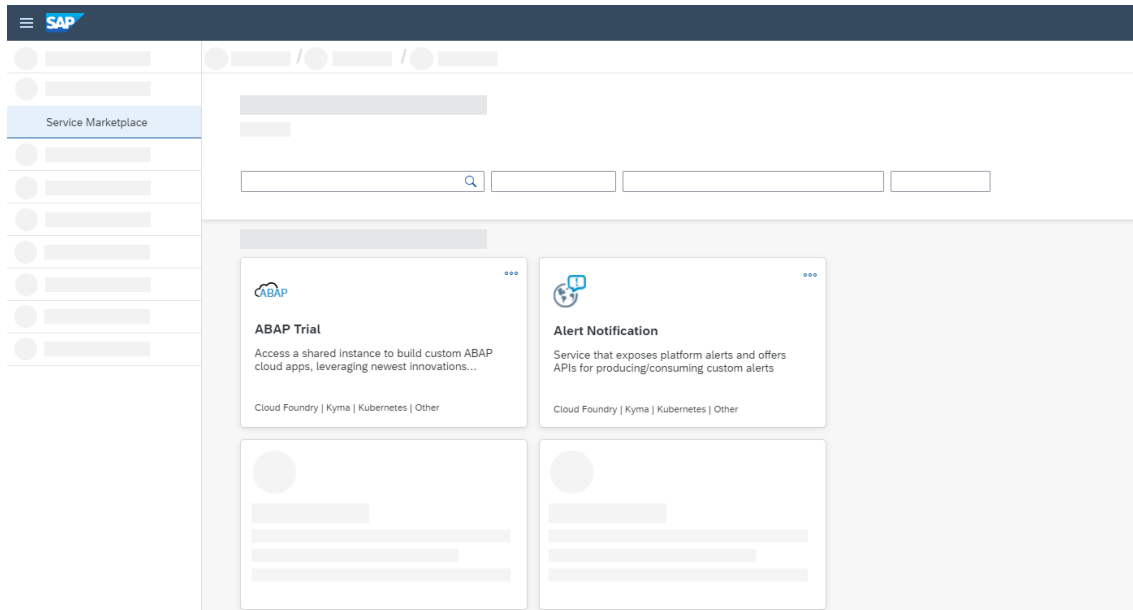


Figure: BTP Service Marketplace — search for “HANA Cloud” to find the tile

Click the HANA Cloud tile **Create**. You will be taken to the HANA Cloud provisioning wizard.

Fill in the configuration:

Field	Value
Instance Name	hana-trial
Administrator Password	Choose a strong password (save it!)
Memory	30 GB (the trial default — do not reduce)
Storage	120 GB (the trial default)

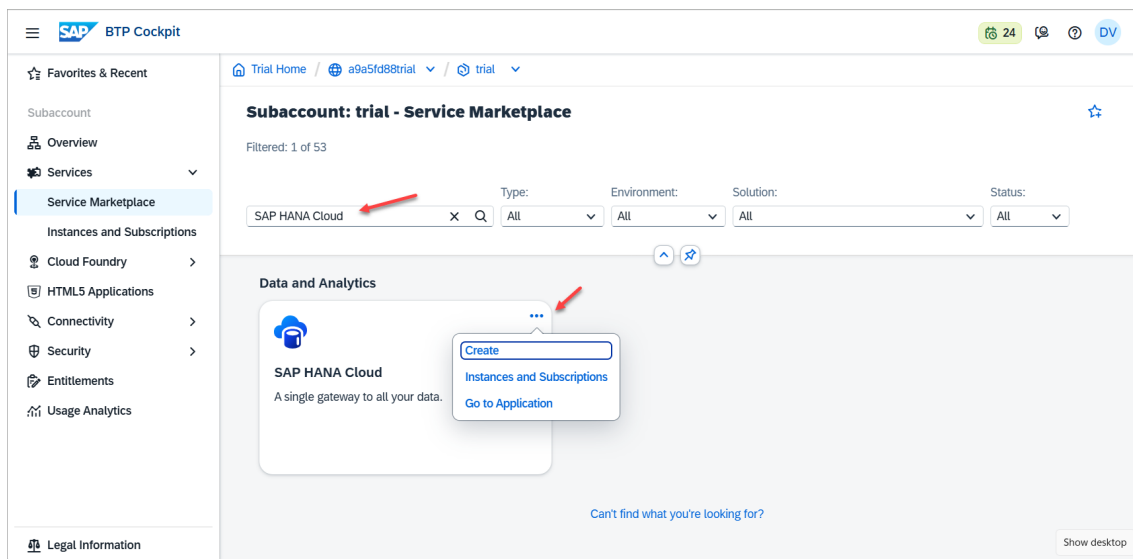


Figure: HANA Cloud instance creation wizard — set instance name and keep default

30GB memory

Click **Create**. Provisioning takes approximately 10–15 minutes. You will see the instance appear in the SAP HANA Cloud Central tool with a “Starting” status.

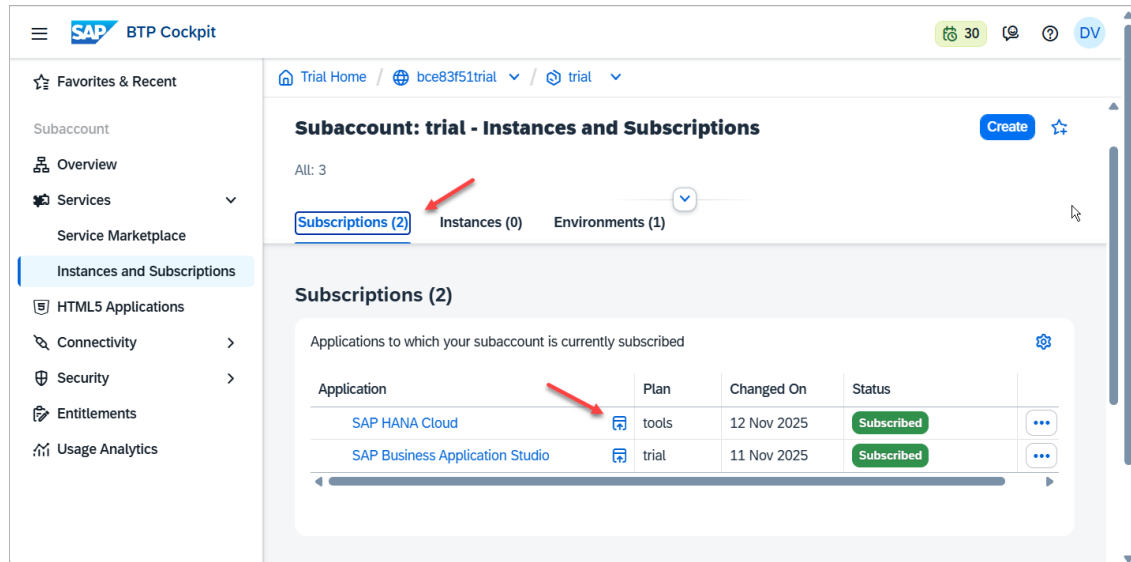


Figure: SAP HANA Cloud Central showing the instance provisioning status

Wait until the status shows **Running** before proceeding.

Enabling the Vector Engine

By default, HANA Cloud provisions with the vector engine enabled in trial. To confirm, open **SAP HANA Cloud Central** → click your instance → **Manage HANA Cloud** → **Edit**.

Under **Additional Features**, verify that **Script Server** and **Document Store** are enabled. The vector engine (REAL_VECTOR column type) is part of the core engine — no separate flag is needed.

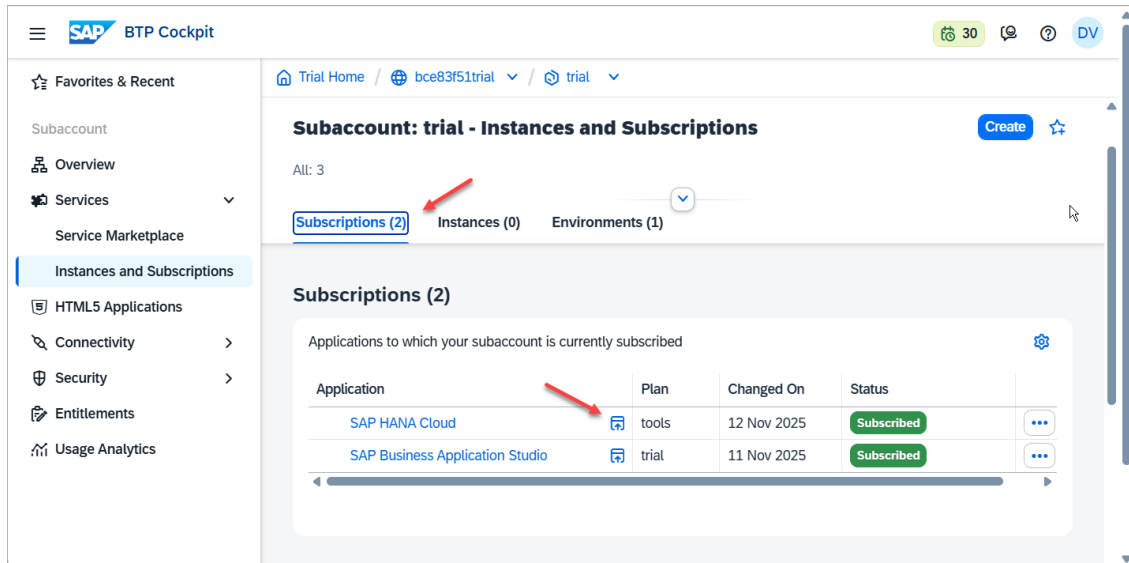


Figure: HANA Cloud Central — click your instance to manage settings and verify vector engine availability

Getting Your HANA Connection Details

From SAP HANA Cloud Central, click your instance ▢ **Actions** ▢ **Copy SQL Endpoint**.

This gives you a connection string like:

```
<instance-id>.hana.trial-us10.hanacloud.ondemand.com:443
```

Save these: - **Host:** <instance-id>.hana.trial-<region>.hanacloud.ondemand.com

- **Port:** 443 - **User:** DBADMIN - **Password:** The password you set during provisioning

Warning: The DBADMIN user has full database privileges. For production, create a dedicated application user with limited permissions. For this book's trial environment, DBADMIN is acceptable.

3.5 Setting Up Google Cloud + Vertex AI

Creating a GCP Account

Go to cloud.google.com/free and click **Get started for free**.



Figure: Google Cloud free tier page — click “Get started for free” to claim your \$300 credit

Sign in with your Google account. GCP will ask for a credit card for identity verification — **you will not be charged** during the free trial, and the \$300 credit covers everything in this book many times over.

After signup, you will land on the **GCP Console**.

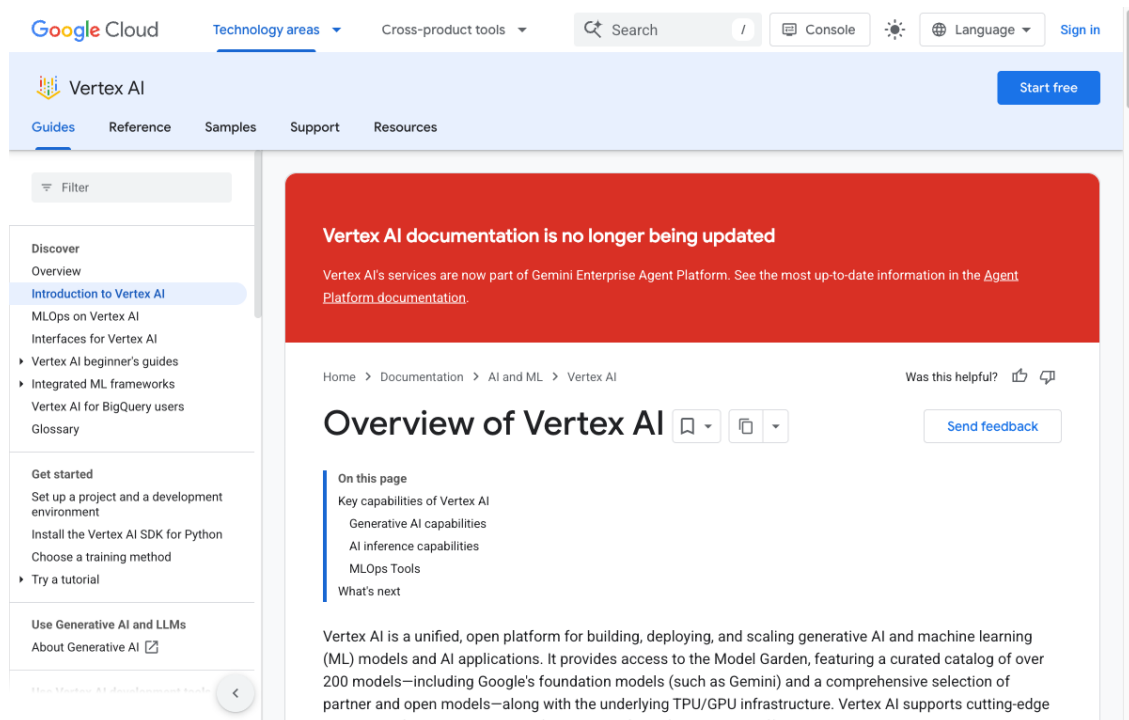


Figure: Google Cloud Console — your project selector appears in the top navigation bar

Creating a New Project

Click the project selector at the top of the page ▢ **New Project**.

Field	Value
Project Name	agentic-rag-btp
Project ID	Auto-generated (note this down)
Organization	No organization (personal account)

Note: In GCP Console, click the project selector dropdown ▢ “New Project” ▢ enter a project name (e.g. hybrid-rag-btp) ▢ Create.

Click **Create**. Wait a few seconds for the project to be created, then select it from the project selector.

Enabling the Vertex AI API

In the GCP Console, go to **APIs & Services** ▢ **Library**. Search for “Vertex AI API.”

Note: Navigate to APIs & Services ▢ Library ▢ search “Vertex AI API” ▢ click Enable.

Click the Vertex AI API tile ▢ **Enable**.

Note: After enabling, the Vertex AI API page will show a green checkmark and “API enabled” status.

Enabling the API takes about 30 seconds. Once enabled, you will see the Vertex AI API dashboard.

Note: Enabling Vertex AI automatically enables several dependent APIs (Cloud Storage, Cloud Resource Manager, etc.). This is expected.

Creating a Service Account

Your BTP application needs credentials to call the Vertex AI API. GCP uses **service accounts** for machine-to-machine authentication. This is the right tool — not user accounts, not OAuth flows.

Go to **IAM & Admin** ▢ **Service Accounts** ▢ **Create Service Account**.

Field	Value
Service Account Name	agentic-rag-btp-sa
Service Account ID	Auto-generated from agentic-rag-btp-sa
Description	Service account for Agentic Hybrid RAG on SAP BTP

Note: Navigate to IAM & Admin → Service Accounts → Create Service Account
→ enter name hybrid-rag-sa → Continue.

Click **Create and Continue**.

In Step 2 (Grant access), add the following role:

Role	Purpose
Vertex AI User	Allows calling Vertex AI APIs (LLM and embeddings)

Note: In the role assignment step, search for “Vertex AI User” and select it.
Click Continue → Done.

Click **Continue** → **Done**.

Downloading the Service Account Key

You will now download a JSON key file that contains the credentials your BTP application will use.

From the Service Accounts list, click the service account you just created → **Keys** tab → **Add Key** → **Create new key** → **JSON** → **Create**.

Note: Click your service account → Keys tab → Add Key → Create new key → JSON → Create. The key file downloads automatically.

A JSON file will download automatically. It looks like this:

```

1 {
2   "type": "service_account",
3   "project_id": "agentic-rag-btp",
4   "private_key_id": "abc123...",
5   "private_key": "-----BEGIN RSA PRIVATE KEY-----\n...\n-----END RSA
PRIVATE KEY-----\n",

```

```
6  "client_email":  
    "agentic-rag-btp-sa@agentic-rag-btp.iam.gserviceaccount.com",  
7  "client_id": "123456789",  
8  "auth_uri": "https://accounts.google.com/o/oauth2/auth",  
9  "token_uri": "https://oauth2.googleapis.com/token"  
10 }
```

Warning: This file contains your private key. Treat it like a password. Do not commit it to git. Do not share it. Store it somewhere safe on your machine — we will upload it to BTP Destination Service in the next section, and then you will not need it in plaintext again.

Save the file as `gcp-sa-key.json` in a secure location outside your project directory.

3.6 Configuring the BTP Destination Service

The BTP Destination Service is the secure credential store that connects your BTP applications to external services. Your FastAPI agent will read the Vertex AI credentials from this destination at runtime — the JSON key never lives in application code or in `manifest.yml`.

This is the SAP-native way to manage external service credentials, and it is an important pattern to understand. As Albada notes in *Building Applications with AI Agents*, modularity is essential in agent system design — your agent should be able to swap its LLM provider by changing a destination configuration, not by redeploying code.

Creating the Destination

In the BTP Cockpit, go to your trial subaccount ▢ **Connectivity** ▢ **Destinations** ▢ **New Destination**.

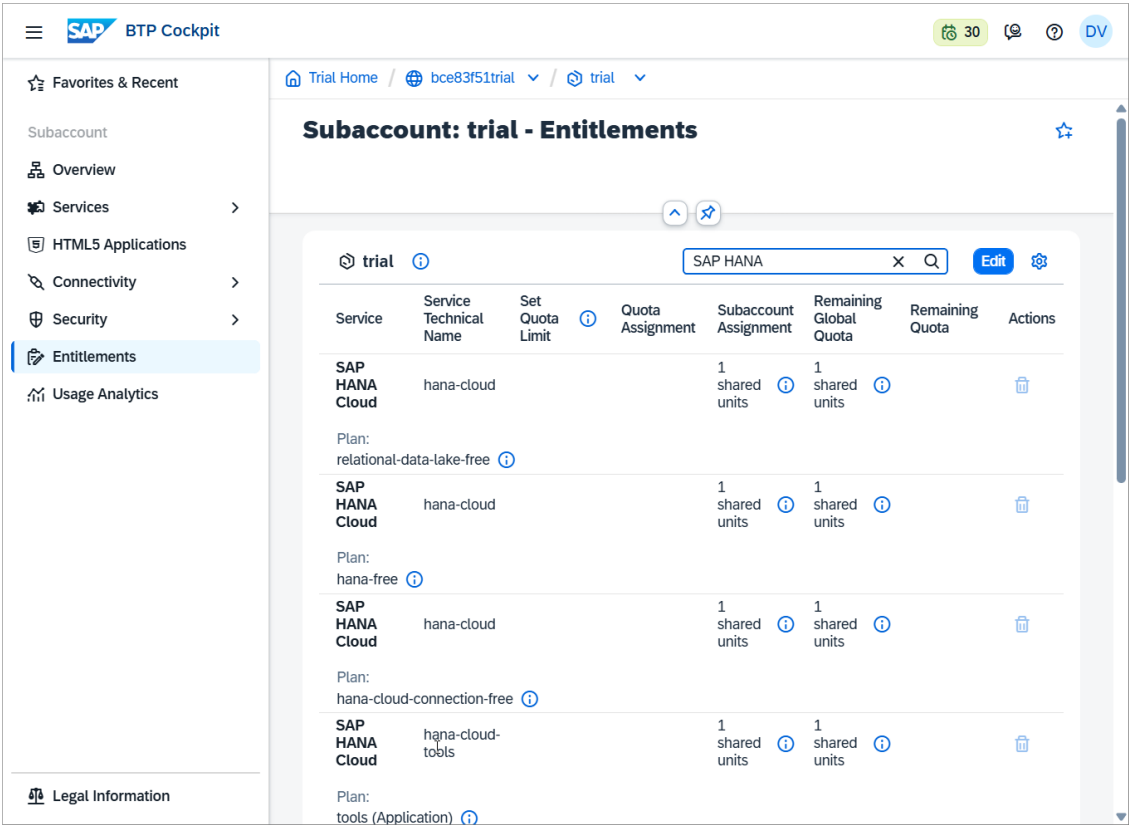


Figure: BTP Connectivity > Destinations — click “New Destination” to create the Vertex AI destination

Fill in the destination configuration:

Field	Value
Name	VertexAI
Type	HTTP
Description	Google Vertex AI API (Gemini + Embeddings)
URL	https://us-central1-aiplatform.googleapis.com
Proxy Type	Internet
Authentication	NoAuthentication

Click **New Property** to add these additional properties:

Property	Value
gcp.project_id	Your GCP project ID (e.g., agentic-rag-btp)
gcp.location	us-central1

Property	Value
<code>gcp.service_account_key</code>	The entire contents of your <code>gcp-sa-key.json</code> file, pasted as a single string

Note: Fill in all destination fields as shown in the table above. Paste your GCP service account JSON key into the token service URL credentials field.

Click **Save**, then click **Check Connection**. You should see a green “200 OK” response.

Tip: After saving, click “Check Connection” at the bottom of the destination form. A “200 OK” response confirms the credentials are valid.

Note: The `gcp.service_account_key` property stores the raw JSON — not a file path. Open `gcp-sa-key.json` in a text editor, select all, copy, and paste the entire JSON object into the property value field. BTP encrypts this value at rest.

3.7 Local Development Environment

Now set up your local machine. We need Python, Node.js, the CF CLI, the CDS CLI, and VS Code.

Python 3.11+

Verify your Python version:

```
1 python3 --version
2 # Should output: Python 3.11.x or 3.12.x
```

If you need to install Python, download it from python.org/downloads. On macOS with Homebrew:

```
1 brew install python@3.11
```

Node.js 20+

Verify:

```
1 node --version # Should output: v20.x.x or higher
2 npm --version  # Should output: 10.x.x or higher
```

Install from `nodejs.org` if needed. On macOS:

```
1 brew install node@20
```

Cloud Foundry CLI

The CF CLI is how you deploy your application to BTP Cloud Foundry.

```
1 # macOS (Homebrew)
2 brew install cloudfoundry/tap/cf-cli@8
3
4 # Verify
5 cf --version
6 # Cloud Foundry CLI version 8.x.x
```

For other platforms, download from the CF CLI releases page.

Log in to your BTP CF environment:

```
1 cf login -a https://api.cf.<your-region>.hana.ondemand.com
2 # Enter your BTP email and password
3 # Select the "trial" org and "dev" space
```

Note: After `cf login`, you will be prompted to select your org (your trial org name) and space (dev). Select the correct ones and press Enter.

Verify the login:

```
1 cf target
2 # Should show:
3 # API endpoint: https://api.cf.eu10.hana.ondemand.com
4 # API version: 3.x.x
5 # user: your@email.com
```

```

6 # org:          your-trial-org
7 # space:        dev

```

SAP CDS CLI

The CDS CLI is the development tool for CAP Node.js applications.

```

1 npm install -g @sap/cds-dk
2
3 # Verify
4 cds --version
5 # @sap/cds-dk: 8.x.x

```

VS Code Extensions

Open VS Code and install these extensions:

Extension	Publisher	Purpose
SAP CDS Language Support	SAP	CDS syntax highlighting and validation
Python	Microsoft	Python IntelliSense and debugging
REST Client	Humao	Test API endpoints without Postman

Tip: In VS Code, press `Cmd+Shift+X` to open Extensions. Search for each extension name and click Install.

3.8 Project Setup

Clone the companion repository and set up the Python environment for the agent service:

```

1 git clone https://github.com/sriramrokkam/agent-rag-btp-book
2 cd agent-rag-btp-book/agents
3

```

```

4 python3 -m venv .env
5 source .env/bin/activate    # Windows: .env\Scripts\activate
6
7 pip install -r requirements.txt

```

The `requirements.txt` for the agent service contains:

```

1 fastapi==0.111.0
2 uvicorn[standard]==0.30.1
3 langgraph==0.2.0
4 langchain-google-vertexai==1.0.6
5 google-auth==2.29.0
6 hdbcli==2.21.28
7 PyMuPDF==1.24.5
8 python-multipart==0.0.9
9 httpx==0.27.0

```

Create your local environment file:

```

1 cp .env.example .env

```

Edit `.env` with your credentials:

```

1 # SAP HANA Cloud
2 HANA_HOST=<your-instance>.hana.trial-<region>.hanacloud.ondemand.com
3 HANA_PORT=443
4 HANA_USER=DBADMIN
5 HANA_PASSWORD=<your-hana-password>
6
7 # Google Vertex AI (local development only)
8 # On BTP, credentials come from the Destination Service
9 GCP_PROJECT_ID=agentic-rag-btp
10 GCP_LOCATION=us-central1
11 GOOGLE_APPLICATION_CREDENTIALS=/path/to/gcp-sa-key.json
12
13 # Backend URL (for CAP service to call the FastAPI agent)
14 BACKEND_URL=http://localhost:8000

```

Note: The `GOOGLE_APPLICATION_CREDENTIALS` environment variable is only used for local development. When deployed to BTP Cloud Foundry, the

agent reads credentials from the Destination Service instead. We will see this credential-switching logic in Chapter 10.

3.9 Verifying the Setup: Your First Gemini API Call

Let us verify that everything works end-to-end. We will call Gemini 2.5 Flash directly and confirm we get a response.

Create a file `test_vertex.py` in your `agents/` directory:

```
1  import os
2  from dotenv import load_dotenv
3  import vertexai
4  from vertexai.generative_models import GenerativeModel
5
6  load_dotenv()
7
8  # Initialize Vertex AI with project and location
9  vertexai.init(
10     project=os.getenv("GCP_PROJECT_ID"),
11     location=os.getenv("GCP_LOCATION"),
12 )
13
14 # Load Gemini 2.5 Flash
15 model = GenerativeModel("gemini-2.5-flash-preview-05-20")
16
17 # Send a simple test prompt
18 response = model.generate_content(
19     "You are a helpful assistant for SAP developers. "
20     "In one sentence, explain what a Material Safety Data Sheet (MSDS) is."
21 )
22
23 print("Gemini response:")
24 print(response.text)
25 print("\nModel used: gemini-2.5-flash-preview-05-20")
26 print("Setup verified successfully.")
```

Run it:

```
1 python test_vertex.py
```

You should see output like:

Gemini response:

A Material Safety Data Sheet (MSDS) is a standardized document that provides detailed information about the properties, hazards, safe handling procedures, and emergency response measures for a chemical substance or mixture.

Model used: gemini-2.5-flash-preview-05-20

Setup verified successfully.

Expected terminal output:

VertexAI initialized successfully

Model loaded: gemini-2.5-flash

Response: The capital of France is Paris.

Vertex AI connection: OK

If you see an error: The most common issues are: - `google.auth.exceptions.DefaultCredentialsError` — Check that `GOOGLE_APPLICATION_CREDENTIALS` points to the correct JSON file path - Permission denied on resource — Verify the service account has the **Vertex AI User** role in GCP IAM - API not enabled — Return to the GCP Console and confirm the Vertex AI API is enabled for your project

Now verify the embedding model:

```
1 # Add to test_vertex.py
2
3 from vertexai.language_models import TextEmbeddingModel
4
5 embedding_model = TextEmbeddingModel.from_pretrained("text-embedding-004")
6
7 test_texts = [
8     "What is the flash point of acetone?",
9     "GHS hazard classification for flammable liquids"
10 ]
11
12 embeddings = embedding_model.get_embeddings(test_texts)
```

```

13
14 for text, embedding in zip(test_texts, embeddings):
15     print(f"\nText: {text[:50]}...")
16     print(f"Embedding dimension: {len(embedding.values)}")
17     print(f"First 5 values: {embedding.values[:5]}")

```

The output should show embeddings with dimension 768 (the dimension of text-embedding-004):

Text: What is the flash point of acetone?...

Embedding dimension: 768

First 5 values: [0.023, -0.041, 0.018, 0.067, -0.029]

Text: GHS hazard classification for flammable liquids...

Embedding dimension: 768

First 5 values: [0.031, -0.038, 0.022, 0.054, -0.011]

Note: The dimension 768 is important. When we create the HANA Cloud vector table in Chapter 3, we will define the column as `REAL_VECTOR(768)`. If you switch embedding models later, you must recreate the table — the dimension is fixed at table creation time.

3.10 Verifying the HANA Cloud Connection

Run a quick connectivity test to confirm HANA Cloud is reachable:

```

1  # test_hana.py
2  import os
3  from dotenv import load_dotenv
4  from hdbcli import dbapi
5
6  load_dotenv()
7
8  conn = dbapi.connect(
9      address=os.getenv("HANA_HOST"),
10     port=int(os.getenv("HANA_PORT")),
11     user=os.getenv("HANA_USER"),

```

```

12     password=os.getenv("HANA_PASSWORD"),
13     encrypt=True,
14     sslValidateCertificate=False
15 )
16
17 cursor = conn.cursor()
18 cursor.execute("SELECT VERSION FROM SYS.M_DATABASE")
19 version = cursor.fetchone()[0]
20 print(f"Connected to SAP HANA Cloud version: {version}")
21
22 # Verify vector engine is available
23 cursor.execute("""
24     SELECT COUNT(*) FROM SYS.M_FEATURE_USAGE
25     WHERE FEATURE_NAME = 'VECTOR'
26 """)
27 vector_count = cursor.fetchone()[0]
28 if vector_count > 0:
29     print("Vector engine: Available")
30 else:
31     print("Vector engine: Not detected (check HANA Cloud instance
32         settings)")
33
34 cursor.close()
35 conn.close()
36 print("\nHANA Cloud connection verified.")

```

```
1 python test_hana.py
```

Expected output:

Connected to SAP HANA Cloud version: 4.00.000.00.1698...

Vector engine: Available

HANA Cloud connection verified.

Expected terminal output:

HANA version: 4.00.000.00.1234567890 (fa/CE2024)

Vector engine: Available

SPARQL engine: Available

HANA connection: OK

3.11 What We Have Built

Let us take stock of what is now in place:

Component	Status	Where
BTP trial account	Running	account.hanatrial.ondemand.com
CF space “dev”	Ready	BTP Cockpit → Cloud Foundry
HANA Cloud instance	Running	BTP Cockpit → HANA Cloud Central
GCP project	Active	console.cloud.google.com
Vertex AI API	Enabled	GCP Console → APIs & Services
GCP service account	Created	GCP Console → IAM & Admin
BTP Destination “VertexAI”	Configured	BTP Cockpit → Connectivity
Python environment	Active	agents/.venv
Gemini API call	Verified	test_vertex.py passed
HANA connection	Verified	test_hana.py passed

This is not a small setup. You now have a working hybrid cloud environment: a managed in-memory database with vector and graph capabilities on BTP, connected to a frontier AI inference API on GCP, with credentials managed securely through BTP’s Destination Service.

The architecture you set up here mirrors how SAP customers deploy AI at scale — not a toy environment, but the real stack.

3.12 Understanding the Credential Flow

Before we move on, it is worth understanding exactly how credentials flow through the system. This will matter in Chapter 10 when you deploy to BTP, and it is a pattern that comes up repeatedly in enterprise AI systems.

Local development:

```
.env file → GOOGLE_APPLICATION_CREDENTIALS
→ google-auth library reads the JSON key directly
```

→ Vertex AI API call authenticated

BTP Cloud Foundry deployment:

BTP Destination Service "VertexAI"

→ agents/srv/vertex_srv.py reads via CF environment

→ google-auth constructs credentials from the stored JSON

→ Vertex AI API call authenticated

The key insight is that **the application code never changes** between local and deployed. The credential source changes — `.env` locally, Destination Service in production — but the interface that `vertex_srv.py` exposes to the rest of the system is identical. This is the modularity principle in practice.

We will build `vertex_srv.py` in Chapter 3. For now, knowing the pattern exists is enough.

3.13 Summary

- SAP BTP trial provides free Cloud Foundry runtime and HANA Cloud — the only database that combines vector search and SPARQL knowledge graph in one service
- Google Vertex AI is an external HTTPS API — no GPU management, no model hosting, just an API call from BTP CF
- The BTP Destination Service stores GCP credentials securely — never in code, never in `manifest.yml`
- `text-embedding-004` produces 768-dimensional embeddings — this dimension is fixed and determines the HANA vector table schema
- Both the Gemini API call and the HANA Cloud connection are now verified end-to-end

In Chapter 3, we build the first half of our knowledge layer: vector search on SAP HANA Cloud. We will create the `MSDS_VECTORS` table, implement the embedding pipeline using `text-embedding-004`, and run our first semantic similarity search — the same operation our hybrid RAG agent will use at query time.

Chapter 3: Vector Search on SAP HANA Cloud

In Chapter 2 you stood up the platform: a HANA Cloud instance, a GCP project with Vertex AI enabled, a BTP destination, and a Python virtual environment with the right libraries. Everything is connected, but nothing useful is happening yet. In this chapter we change that.

By the end of this chapter you will have a working vector search system on HANA Cloud. You will take a chunk of text from a Material Safety Data Sheet, turn it into a 768-dimensional vector using Google’s `text-embedding-004` model, store it in a `REAL_VECTOR` column, and retrieve the most semantically similar chunks for a natural language question — all in roughly 200 lines of Python.

We will also be honest about what vector search is bad at. The closing section of this chapter sets up the motivation for Chapter 4, where we add a knowledge graph to handle the queries that vectors get wrong.

3.1 What Are Embeddings? Intuition Without the Math

If you have spent your career writing ABAP, CDS views, and OData services, the word “embedding” probably sounds like something invented to make you feel old. It isn’t. The idea is older than most enterprise software, and the intuition is simple.

An embedding is a fixed-length list of numbers — in our case, 768 floating-point numbers — that represents the *meaning* of a piece of text. The crucial property is this: two pieces of text with similar meanings produce embeddings that are close to each other in 768-dimensional space. Two pieces of text with different meanings produce embeddings that are far apart.

A useful analogy: imagine you are sorting books in a library. A traditional database would file them by title (alphabetical order) or by author. That is exact-match retrieval — fast, but useless if you don’t know the title. A librarian, by contrast, knows that *Crime and Punishment* and *The Brothers Karamazov* belong on the same shelf because they are both Dostoevsky novels about guilt and redemption. The librarian has put each book

at a *position* that reflects its content. Books on similar topics end up close together.

Embeddings do the same thing for text, but in 768 dimensions instead of two. The “position” of a sentence is just its embedding vector. Sentences about flammability cluster together. Sentences about first aid cluster together. Sentences about regulatory codes cluster together — and importantly, they cluster *separately* from the flammability sentences, even when they mention the same chemical.

Note: The number 768 is not magic. It is simply the output dimension chosen by the team that trained `text-embedding-004`. Other models produce 384, 1024, 1536, or 3072 dimensions. Higher is not always better — it just means more storage and slower comparisons. For our use case, 768 is a comfortable balance.

Two further points before we move on.

First, embeddings are not human-readable. If you print one out, you will see something like `[0.0234, -0.1107, 0.0891, ..., 0.0023]`. The individual numbers mean nothing in isolation. Their meaning emerges only in *relation* to other embeddings — specifically, in how close or far apart they are.

Second, the model that produces embeddings is the same model (or a sibling model) that was trained on huge corpora of text. It has internalized which words and phrases tend to occur in similar contexts, and it projects each input into a position that reflects that learned structure. You don’t train the model. You don’t fine-tune it. You just call it as an API.

Calling the API gives you a vector. Storing that vector and searching across millions of them efficiently is the database’s job. Until recently, that meant using a specialized vector database — Pinecone, Weaviate, Milvus — alongside your real database. As of HANA Cloud QRC 1/2024, you no longer need a sidecar. HANA does it natively, and that is the next topic.

3.2 The HANA Cloud `REAL_VECTOR` Column Type

HANA has always been a column-store database optimized for analytical workloads. In 2024, SAP added a new native column type built specifically for AI: `REAL_VECTOR`.

A `REAL_VECTOR` column stores a fixed-dimension array of single-precision floats. When you declare the column you fix the dimension (for example, `REAL_VECTOR(768)`), and HANA refuses any insert with a different dimension. This is what you want — mixing dimensions in the same column would mean comparing apples to oranges.

What makes this more than a glorified `BLOB` column is the set of operators HANA exposes on it:

- `TO_REAL_VECTOR(string)` — converts a string in the form `'[0.1, 0.2, ..., 0.768]'` into a real vector value. This is how you bind a parameter from Python.
- `COSINE_SIMILARITY(v1, v2)` — returns the cosine of the angle between two vectors. Range: `-1.0` (opposite) to `+1.0` (identical direction). For unit-normalized embeddings (which `text-embedding-004` produces), this is in practice between `0.0` and `1.0`.
- `L2DISTANCE(v1, v2)` — Euclidean distance between two vectors. Useful when your model is not normalized.

These operators are SIMD-accelerated on the HANA engine, which means a single CPU instruction processes multiple floats in parallel. Searching across hundreds of thousands of 768-dimensional vectors takes milliseconds, not seconds.

Tip: For embeddings produced by Google, OpenAI, and Cohere — all of which are L2-normalized at the API boundary — `COSINE_SIMILARITY` is the right choice. Use `L2DISTANCE` only if you are working with embeddings that are not normalized.

A second feature worth knowing about (we will not use it in this chapter, but it matters at scale) is the **HNSW index**. Without an index, every similarity query scans the entire table. With an HNSW index, HANA builds a graph structure that gets you approximate-nearest-neighbor results in logarithmic time. For our chapter you will not need it — five MSDS chunks search in under 10 ms — but for production deployments with millions of chunks, indexing is essential. We come back to this in Chapter 9 when we tune for production.

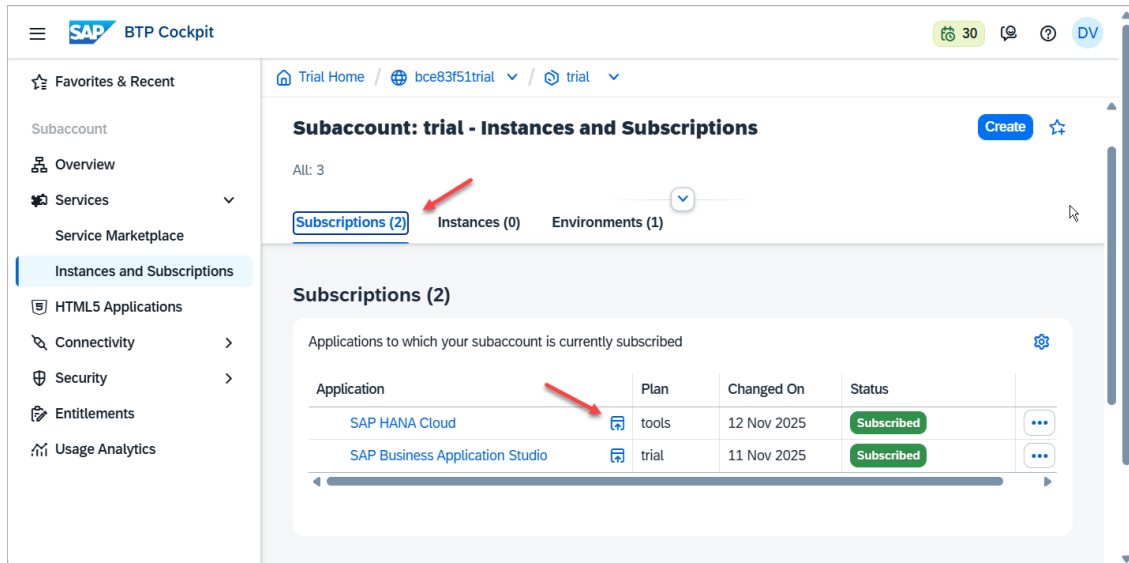


Figure: SAP HANA Cloud Central — the `REAL_VECTOR` column type is available in all HANA Cloud instances from version 4.0 onwards

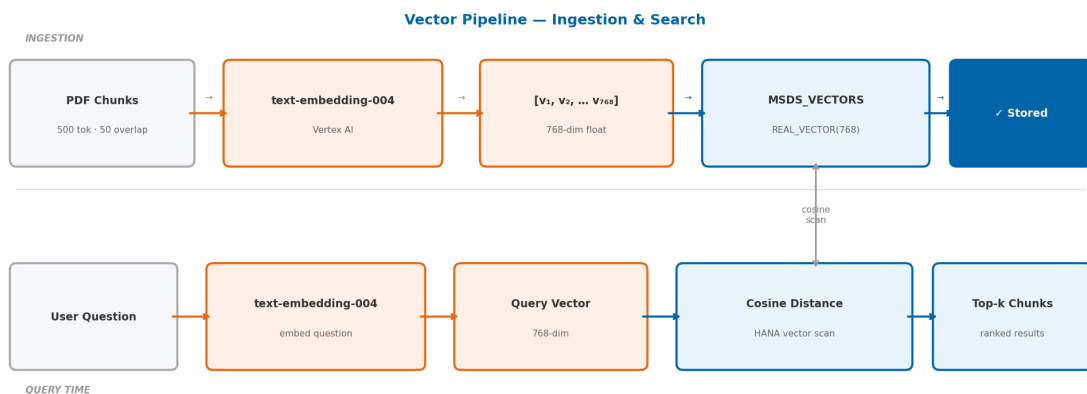


Figure 2: Vector Embedding Pipeline

Figure 3.1 — The vector pipeline has two phases. Ingestion (top): PDF text is chunked, each chunk is sent to `text-embedding-004`, and the resulting 768-dimensional vector is written to `MSDS_VECTORS`. Query time (bottom): the question is embedded with the same model, then HANA computes cosine distance between the query vector and every stored vector, returning the top-k most relevant chunks.

3.3 Creating the Vector Table: The Lazy Initialization Pattern

We need a table to hold our embeddings. The schema is simple:

```

1 CREATE TABLE MSDS_VECTORS (
2     ID BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
3     MATERIAL_NUMBER NVARCHAR(100) NOT NULL,
4     CHUNK_TEXT NCLOB NOT NULL,
5     CHUNK_INDEX INTEGER NOT NULL,
6     EMBEDDING REAL_VECTOR(768)
7 );

```

MATERIAL_NUMBER is the SAP material identifier — the same one you would see in MM03. We use it as a tenant key so we can search within a single material's chunks. CHUNK_TEXT is the raw text of the chunk; CHUNK_INDEX lets us reconstruct order if we need to. EMBEDDING is the 768-dimensional vector we computed from CHUNK_TEXT.

You could just run this CREATE TABLE statement once with the SAP HANA Database Explorer and be done with it. But there is a subtle reason not to.

The dimension 768 is hardcoded into the schema. If at any point you decide to switch from text-embedding-004 (768 dim) to text-embedding-3-large (3072 dim), or to a SAP-internal embedding model with a different dimension, your insert will fail and you will need to drop and recreate the table. Worse, that hardcoded number lives in your DDL while the actual dimension lives in the model — two sources of truth that can drift apart.

The cleaner pattern is **lazy initialization**: don't create the table until the first time you embed something, then read the dimension from the embedding response and use it in the CREATE TABLE. This way your code has exactly one source of truth — the model's actual output dimension at runtime.

Here is the relevant snippet from `agents/srv/vector_srv.py` (we will see the full file in section 3.8):

```

1 TABLE_NAME = "MSDS_VECTORS"
2 _table_created = False

```

```

3
4 def _ensure_table(embedding_dim: int):
5     global _table_created
6     if _table_created:
7         return
8     conn = get_connection()
9     cursor = conn.cursor()
10    cursor.execute(f"""
11        CREATE TABLE IF NOT EXISTS {TABLE_NAME} (
12            ID BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
13            MATERIAL_NUMBER NVARCHAR(100) NOT NULL,
14            CHUNK_TEXT NCLOB NOT NULL,
15            CHUNK_INDEX INTEGER NOT NULL,
16            EMBEDDING REAL_VECTOR({embedding_dim})
17        )
18    """)
19    conn.commit()
20    cursor.close()
21    _table_created = True

```

Three things to note:

1. The `_table_created` module-level flag is a one-time guard. We don't want to issue `CREATE TABLE IF NOT EXISTS` on every insert — that is a roundtrip we don't need.
2. The `IF NOT EXISTS` clause means restarting the Python process is safe. The first call after restart will hit the database once, see the table exists, and move on.
3. The `embedding_dim` is passed in by the caller, who got it from the actual embedding. There is no hardcoded 768 anywhere in the application code.

Warning: Do not be tempted to add an `ALTER TABLE` path that resizes the `REAL_VECTOR` column if the dimension changes. A dimension change means your old embeddings are incompatible with new ones, and you should treat it as a full re-indexing event — drop the table, re-embed all your documents, and reload. Silently mixing dimensions is a recipe for confusion.

3.4 Calling text-embedding-004 from Python

We talked about embeddings as a concept; now we need to actually produce one. The Vertex AI Python SDK gives us two lines of meaningful code.

Looking at `agents/srv/vertex_srv.py`:

```
1  import os
2  import threading
3  import vertexai
4  from vertexai.generative_models import GenerativeModel
5  from vertexai.language_models import TextEmbeddingModel
6  from dotenv import load_dotenv
7
8  load_dotenv()
9
10 _init_lock = threading.Lock()
11 _initialized = False
12
13 def _ensure_initialized():
14     global _initialized
15     if not _initialized:
16         with _init_lock:
17             if not _initialized:
18                 vertexai.init(
19                     project=os.getenv("GCP_PROJECT_ID"),
20                     location=os.getenv("GCP_LOCATION", "us-central1")
21                 )
22                 _initialized = True
23
24 def get_llm():
25     _ensure_initialized()
26     return GenerativeModel("gemini-2.5-flash-preview-05-20")
27
28 def get_embedding_model():
29     _ensure_initialized()
30     return TextEmbeddingModel.from_pretrained("text-embedding-004")
31
32 def embed_text(text: str) -> list[float]:
```

```

33     model = get_embedding_model()
34     embeddings = model.get_embeddings([text])
35     return embeddings[0].values # list of 768 floats

```

The pattern here mirrors the lazy table creation pattern. `vertexai.init()` is global SDK state; calling it more than once is wasteful. The double-checked locking idiom (`if not _initialized` outside the lock, then again inside) is the standard way to do this safely under FastAPI's threaded request model. If you have never seen the pattern before, the gist is: the outer check is the fast path for the common case (already initialized, no lock needed), and the inner check makes sure that two threads racing for the lock don't both initialize.

`get_embedding_model()` returns a `TextEmbeddingModel` instance bound to `text-embedding-004`. This is cheap — the SDK caches the model client internally — but we still gate it behind `_ensure_initialized` so that calling code does not have to think about ordering.

`embed_text()` is the function the rest of our application uses. It takes a string and returns a Python list of 768 floats. Internally, the SDK takes a list of strings, so we pass `[text]` and unwrap the first result with `embeddings[0].values`.

Note: Vertex AI's embedding endpoint accepts batches of up to 250 texts per call. For a one-off MSDS upload that ends up with maybe 30 chunks, the savings of batching are negligible compared to the simplicity of one-text-per-call. We will revisit batching in Chapter 9 when we look at bulk-loading historical documents.

You should also be aware that Vertex AI charges per 1,000 input characters for embeddings (about \$0.000025 per 1,000 chars at the time of writing). A typical 12-page MSDS PDF has roughly 30,000 characters and chunks into ~30 pieces. The cost to embed it is approximately \$0.0008 — less than a tenth of a cent. Search-time embeddings (one per question) are equally cheap.

3.5 Storing Embeddings: INSERT with REAL_VECTOR

Once we have an embedding (a Python `list[float]`), we need to get it into the `REAL_VECTOR` column. The HANA Python driver `hdbcli` does not yet have a native

binding for vector parameters, so we use the `TO_REAL_VECTOR(string)` SQL function with a string parameter.

The format HANA expects is a JSON-style array literal: `[0.0234, -0.1107, 0.0891, ...]`. We build this string in Python with a simple comma-join:

```
1 embedding_str = "[" + ",".join(str(x) for x in embedding) + "]"
```

A typical 768-dimensional embedding string is around 12,000 characters. That is fine — HANA accepts it as an `NVARCHAR` parameter without issue.

The full insert function:

```
1 def store_embedding(material_number: str, chunk_text: str, chunk_index:
  int, embedding: list[float]):
2     _ensure_table(len(embedding))
3     conn = get_connection()
4     cursor = conn.cursor()
5     embedding_str = "[" + ",".join(str(x) for x in embedding) + "]"
6     cursor.execute(f"""
7         INSERT INTO {TABLE_NAME} (MATERIAL_NUMBER, CHUNK_TEXT,
8         CHUNK_INDEX, EMBEDDING)
9         VALUES (?, ?, ?, TO_REAL_VECTOR(?))
10    """, (material_number, chunk_text, chunk_index, embedding_str))
11    conn.commit()
12    cursor.close()
```

A few engineering notes:

- We pass `material_number`, `chunk_text`, `chunk_index`, and the embedding string as positional parameters. This is parameterized SQL — no concatenation of user data into the query, so SQL injection is impossible.
- `_ensure_table(len(embedding))` runs on every call but exits early after the first invocation thanks to the `_table_created` flag.
- We commit per insert. For uploading a single MSDS this is fine. For bulk uploads, batch your inserts and commit once at the end — Chapter 9 covers this.

Tip: If you want to verify what got stored, the SAP HANA Database Explorer is your friend. Open the `MSDS_VECTORS` table, click “Open Data”, and you will see your rows. The `EMBEDDING` column displays as a truncated array preview,

but you can click into a cell to see the full 768 values.

Note: You can inspect the `MSDS_VECTORS` table in SAP HANA Database Explorer (accessible from HANA Cloud Central → Open in → SAP HANA Database Explorer). The `EMBEDDING` column will show truncated vector data like `[0.0234, -0.1823, 0.0912, ...]`.

3.6 Cosine Similarity Search: The Query Explained

This is the heart of the chapter. The query that retrieves the top-K chunks for a question is:

```

1  SELECT TOP 5
2      CHUNK_TEXT,
3      CHUNK_INDEX,
4      COSINE_SIMILARITY(EMBEDDING, TO_REAL_VECTOR(?)) AS SCORE
5  FROM MSDS_VECTORS
6  WHERE MATERIAL_NUMBER = ?
7  ORDER BY SCORE DESC;
```

Five clauses, each doing real work. Let's read them in execution order.

`WHERE MATERIAL_NUMBER = ?` filters the table to chunks belonging to a specific material. Without this, our search would compete across every chunk of every document in the table — fine when you have one MSDS, fatal when you have ten thousand. By filtering early, HANA scans only the rows that could possibly match. This is the one place in the query where the optimizer can use a B-tree index, so make sure you have one on `MATERIAL_NUMBER` once you go to production. (We add it in Chapter 9.)

`COSINE_SIMILARITY(EMBEDDING, TO_REAL_VECTOR(?))` is the per-row computation. For each row passing the `WHERE` filter, HANA takes the stored `EMBEDDING` vector, the question's vector (parsed from the bound parameter), and computes the cosine of the angle between them. The result is a `DOUBLE` between roughly 0.0 and 1.0 — higher means more similar. This is where the SIMD acceleration earns its keep.

`AS SCORE` names the result so we can refer to it in the `ORDER BY` clause. It's a small thing but worth noting because it is what allows us to write `ORDER BY SCORE DESC` instead of having to repeat the `COSINE_SIMILARITY(...)` expression.

ORDER BY SCORE DESC sorts the surviving rows from highest similarity to lowest. The bulk of the cost here is the sort itself; for our small data this is instant.

SELECT TOP 5 returns only the first five rows after the sort. In LangGraph's vector chain we will pass these chunks to the LLM as context for answering the question. Five is a reasonable default — large enough to give the model useful coverage, small enough not to blow the context window.

The order of operations matters. HANA's optimizer is smart enough to push the WHERE filter down to the table scan (so we never compute similarity for chunks belonging to other materials), but you should still write your queries with the filter explicit. Don't compute similarity across the whole table and filter the result.

Important: A common mistake is to forget the WHERE clause when prototyping with one material, then keep that habit when you add a second. Every query in your application should filter by MATERIAL_NUMBER. There is no business reason a question about Acetone should ever surface a chunk about Sulfuric Acid, even if some words happen to overlap.

The Python wrapper:

```

1 def search_similar(question: str, material_number: str, top_k: int = 5) ->
  list[dict]:
2     question_embedding = embed_text(question)
3     embedding_str = "[" + ",".join(str(x) for x in question_embedding) +
  "]"
4     conn = get_connection()
5     cursor = conn.cursor()
6     cursor.execute(f"""
7         SELECT TOP {top_k}
8             CHUNK_TEXT,
9             CHUNK_INDEX,
10            COSINE_SIMILARITY(EMBEDDING, TO_REAL_VECTOR(?)) AS SCORE
11        FROM {TABLE_NAME}
12        WHERE MATERIAL_NUMBER = ?
13        ORDER BY SCORE DESC
14    """, (embedding_str, material_number))
15    rows = cursor.fetchall()
16    cursor.close()

```

```
17      return [{"chunk": row[0], "chunk_index": row[1], "score":
          float(row[2])} for row in rows]
```

We embed the question (one Vertex AI call), build the parameter string the same way as for inserts, and run the query. The result is a list of dicts — the chunk text, its position in the original document, and its similarity score.

Note: `top_k` is interpolated into the SQL string, not bound as a parameter. This is safe because we control the value (it is a function argument with a default of 5, never user input). HANA does not allow `TOP ?` with bound parameters, so we have no choice. If you ever expose `top_k` to end users, validate it as an integer before interpolation.

3.7 Chunking Strategy for MSDS Documents

We have skipped over the most consequential design decision in any RAG system: how do you split a document into chunks?

Chunking is necessary because a 12-page MSDS does not fit usefully into a single embedding. Even if it did, retrieval would always return “the whole document” and the LLM would have to do all the work of finding the relevant sentence. Chunking lets us push the relevance work into the database.

The trade-off is fundamental:

- **Smaller chunks** (a sentence each) give precise retrieval — when the model returns chunk 47, you know exactly which sentence it cared about. But you lose context. A sentence like “It must be stored away from this material” is meaningless without the surrounding paragraph.
- **Larger chunks** (a full page) preserve context but introduce noise. A paragraph about flammability that also mentions “appropriate PPE” might score highly for a PPE question even though the actual PPE answer is on a different page.

For MSDS documents specifically, we are lucky: the OSHA 16-section format gives us natural boundaries. Section 4 is always First Aid. Section 7 is always Handling and Storage. Section 8 is always Exposure Controls and PPE. These sections are typically 100–300 words — exactly the right size.

Our chunking rule, which we will implement in Chapter 5 when we build `doc_srv.py`:

1. **Primary boundary:** OSHA section header. Each numbered section becomes one chunk.
2. **Secondary boundary:** if a section exceeds ~500 tokens, split on the next paragraph boundary inside it.
3. **Overlap:** each chunk shares ~50 tokens with the preceding chunk. This keeps a sentence from being orphaned at the boundary — if the relevant sentence happens to be the first one of a chunk, the prior chunk also contains it as context.

Token-wise, our targets are:

- Minimum chunk size: ~100 tokens (anything smaller loses context)
- Maximum chunk size: ~500 tokens (anything larger gets noisy)
- Overlap: ~50 tokens

Tip: Tokens, not characters. A token is roughly 4 characters of English text or about 0.75 of a word. Vertex AI's `text-embedding-004` accepts up to 2,048 tokens per embedding call, so we have plenty of headroom — 500 tokens is well under the limit.

For the test in this chapter we cheat: we hand-write five short chunks that look like real MSDS sentences. The full document chunker arrives in Chapter 5. The point of the test is to validate the round-trip: `embed` → `store` → `embed-question` → `search` → `score`.

3.8 Building `vector_srv.py` — The Vector Service Layer

Now we put the pieces together. Create the file `agents/srv/vector_srv.py` with the following content:

```
1 import json
2 from .hdb_srv import get_connection
3 from .vertex_srv import embed_text
4
5 TABLE_NAME = "MSDS_VECTORS"
6 _table_created = False
7
8 def _ensure_table(embedding_dim: int):
```

```

9     global _table_created
10    if _table_created:
11        return
12    conn = get_connection()
13    cursor = conn.cursor()
14    cursor.execute(f"""
15        CREATE TABLE IF NOT EXISTS {TABLE_NAME} (
16            ID BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
17            MATERIAL_NUMBER NVARCHAR(100) NOT NULL,
18            CHUNK_TEXT NCLOB NOT NULL,
19            CHUNK_INDEX INTEGER NOT NULL,
20            EMBEDDING REAL_VECTOR({embedding_dim})
21        )
22    """)
23    conn.commit()
24    cursor.close()
25    _table_created = True
26
27    def store_embedding(material_number: str, chunk_text: str, chunk_index:
28        int, embedding: list[float]):
29        _ensure_table(len(embedding))
30        conn = get_connection()
31        cursor = conn.cursor()
32        embedding_str = "[" + ",".join(str(x) for x in embedding) + "]"
33        cursor.execute(f"""
34            INSERT INTO {TABLE_NAME} (MATERIAL_NUMBER, CHUNK_TEXT,
35            CHUNK_INDEX, EMBEDDING)
36            VALUES (?, ?, ?, TO_REAL_VECTOR(?))
37            """, (material_number, chunk_text, chunk_index, embedding_str))
38        conn.commit()
39        cursor.close()
40
41    def search_similar(question: str, material_number: str, top_k: int = 5) ->
42        list[dict]:
43        question_embedding = embed_text(question)
44        embedding_str = "[" + ",".join(str(x) for x in question_embedding) +
45        "]"
46        conn = get_connection()
47        cursor = conn.cursor()

```



```

44     cursor.execute(f"""
45         SELECT TOP {top_k}
46             CHUNK_TEXT,
47             CHUNK_INDEX,
48             COSINE_SIMILARITY(EMBEDDING, TO_REAL_VECTOR(?)) AS SCORE
49         FROM {TABLE_NAME}
50         WHERE MATERIAL_NUMBER = ?
51         ORDER BY SCORE DESC
52     """, (embedding_str, material_number))
53     rows = cursor.fetchall()
54     cursor.close()
55     return [{"chunk": row[0], "chunk_index": row[1], "score":
56             float(row[2])} for row in rows]
57
58 def delete_vectors(material_number: str) -> int:
59     conn = get_connection()
60     cursor = conn.cursor()
61     cursor.execute(f"DELETE FROM {TABLE_NAME} WHERE MATERIAL_NUMBER = ?",
62                   (material_number,))
63     deleted = cursor.rowcount
64     conn.commit()
65     cursor.close()
66     return deleted
67
68 def count_vectors(material_number: str) -> int:
69     conn = get_connection()
70     cursor = conn.cursor()
71     cursor.execute(f"SELECT COUNT(*) FROM {TABLE_NAME} WHERE
72                   MATERIAL_NUMBER = ?", (material_number,))
73     row = cursor.fetchone()
74     cursor.close()
75     return row[0] if row else 0

```

The module exposes four functions to the rest of the application:

- `store_embedding(material_number, chunk_text, chunk_index, embedding)` — stores a single chunk and its vector. Called once per chunk during document upload.
- `search_similar(question, material_number, top_k)` — embeds the question and returns the top-K most similar chunks for a given material. Called once per

user question.

- `delete_vectors(material_number)` — wipes a material's vectors. Called when a document is replaced or deleted. Returns the row count for logging.
- `count_vectors(material_number)` — reports how many chunks exist for a material. Useful for diagnostics and tests.

There is no `update_embedding` — for chunked documents, the right pattern is delete-then-reinsert. If you change the chunking algorithm, the chunk indices change, so partial updates are dangerous. Treat each upload as a fresh load.

Note: The connection comes from `hdb_srv.py`, which uses thread-local storage to give each request thread its own HANA connection. `hdbcli` connections are not safe to share across threads, but they are cheap to create. Thread-locals give us per-thread reuse without any cross-thread contention. We covered this pattern briefly in Chapter 2 — the file is reproduced for completeness:

```

1  import os
2  import threading
3  from hdbcli import dbapi
4  from dotenv import load_dotenv
5
6  load_dotenv()
7
8  _local = threading.local()
9
10 def get_connection():
11     if not hasattr(_local, 'conn') or _local.conn is None:
12         _local.conn = dbapi.connect(
13             address=os.getenv("HANA_HOST"),
14             port=int(os.getenv("HANA_PORT", 443)),
15             user=os.getenv("HANA_USER"),
16             password=os.getenv("HANA_PASSWORD"),
17             encrypt=True,
18             sslValidateCertificate=False
19         )
20     return _local.conn

```

Warning: `sslValidateCertificate=False` is fine for development against

the BTP-issued HANA endpoint. For production you should provide the correct certificate via `sslTrustStore` and set `sslValidateCertificate=True`. We will tighten this in Chapter 10.

3.9 Testing: One MSDS, Two Questions

Time to see it work end-to-end. Create the file `agents/test_vector.py`:

```

1  import os
2  from dotenv import load_dotenv
3  from srv.vector_srv import store_embedding, search_similar, count_vectors
4  from srv.vertex_srv import embed_text
5
6  load_dotenv()
7
8  TEST_MATERIAL = "ACETONE-TEST-001"
9  TEST_CHUNKS = [
10     "Acetone is a highly flammable liquid and vapor. Flash point: -18°C.
11     Keep away from heat and open flames.",
12     "First aid for skin contact: Wash with soap and water for at least 15
13     minutes. Remove contaminated clothing.",
14     "GHS Classification: Flammable Liquid Category 2. Eye Irritant
15     Category 2A.",
16     "Personal Protective Equipment: Wear chemical resistant gloves and
17     safety glasses when handling.",
18     "Storage: Store in a cool, dry, well-ventilated area away from
19     ignition sources and incompatible materials."
20 ]
21
22 print("Embedding and storing test chunks...")
23 for i, chunk in enumerate(TEST_CHUNKS):
24     embedding = embed_text(chunk)
25     store_embedding(TEST_MATERIAL, chunk, i, embedding)
26     print(f"  Stored chunk {i}: dim={len(embedding)},
27           preview='{chunk[:50]}...'")
28
29 stored = count_vectors(TEST_MATERIAL)

```

```

24 print(f"\nStored {stored} vectors for material {TEST_MATERIAL}")
25
26 print("\nSearching: 'What precautions should I take near open flames?')
27 results = search_similar("What precautions should I take near open
    flames?", TEST_MATERIAL)
28 for r in results:
29     print(f"  Score: {r['score']:.4f} | {r['chunk'][:80]}...")
30
31 print("\nSearching: 'What PPE is required?')
32 results = search_similar("What PPE is required?", TEST_MATERIAL)
33 for r in results:
34     print(f"  Score: {r['score']:.4f} | {r['chunk'][:80]}...")

```

Run it from the agents/ directory:

```

1 cd agents
2 python test_vector.py

```

Expected output (your numbers will vary slightly):

Embedding and storing test chunks...

Stored chunk 0: dim=768, preview='Acetone is a highly flammable liquid and vapor. Fl...'

Stored chunk 1: dim=768, preview='First aid for skin contact: Wash with soap and wat...'

Stored chunk 2: dim=768, preview='GHS Classification: Flammable Liquid Category 2. E...'

Stored chunk 3: dim=768, preview='Personal Protective Equipment: Wear chemical resis...'

Stored chunk 4: dim=768, preview='Storage: Store in a cool, dry, well-ventilated are...'

Stored 5 vectors for material ACETONE-TEST-001

Searching: 'What precautions should I take near open flames?'

Score: 0.8124 | Acetone is a highly flammable liquid and vapor. Flash point: -18°C. Keep aw...

Score: 0.7456 | Storage: Store in a cool, dry, well-ventilated area away from ignition so

Score: 0.6203 | Personal Protective Equipment: Wear chemical resistant gloves and safety

Score: 0.5891 | GHS Classification: Flammable Liquid Category 2. Eye Irritant Category 2A

Score: 0.5102 | First aid for skin contact: Wash with soap and water for at least 15 minut

Searching: 'What PPE is required?'

Score: 0.8442 | Personal Protective Equipment: Wear chemical resistant gloves and safety

Score: 0.6334 | Storage: Store in a cool, dry, well-ventilated area away from ignition so

Score: 0.5912 | First aid for skin contact: Wash with soap and water for at least 15 minut

Score: 0.5108 | Acetone is a highly flammable liquid and vapor. Flash point: -
18°C. Keep aw...

Score: 0.4789 | GHS Classification: Flammable Liquid Category 2. Eye Irritant Category 2A

This is exactly what you want to see. For the open-flames question, the flammability chunk wins at 0.81, with the storage chunk (which mentions “ignition sources”) in second place at 0.74. The first-aid chunk, which has nothing to do with the question, ranks last at 0.51. For the PPE question, the actual PPE chunk wins decisively at 0.84.

Expected terminal output:

Stored 3 chunks for material: acetone-test

Query 1: "What are the fire hazards of acetone?"

Result 1 (score: 0.923): "Acetone is highly flammable..."

Result 2 (score: 0.887): "Keep away from heat sources..."

Query 2: "What first aid is needed for skin contact?"

Result 1 (score: 0.941): "Wash affected area with soap and water..."

Result 2 (score: 0.812): "Remove contaminated clothing..."

Vector search: OK

Note the absolute scores. The top match for a well-targeted question lands around 0.80–0.85. This is the band where you should expect “good” semantic matches with text-embedding-004. Anything above 0.90 usually means near-paraphrase. Anything below 0.50 usually means the model is reaching.

Tip: The scores are reproducible only if you re-embed identical text against the same model version. Vertex AI may update the model under the same name; if your scores drift over time, that is the most likely cause. Pin to a specific version (textembedding-gecko@003-style identifiers exist) if reproducibility matters.

If you see authentication errors at this point, double-check that `G00GLE_APPLICATION_CREDENTIALS` points to your `gcp-sa-key.json` file and that the service account has the Vertex AI User role. The HANA side is more forgiving — connection errors usually surface as “host not reachable”, which means a typo in `HANA_HOST` or a network rule blocking outbound

443.

3.10 What Vector Search Gets Right — and What It Gets Wrong

Time for honesty. Vector search is genuinely powerful, but it has a sharp limitation that becomes obvious as soon as you look at the third example.

Run the test one more time, with a precise factual question:

```
1 print("\nSearching: 'What is the GHS hazard code for acetone?')
2 results = search_similar("What is the GHS hazard code for acetone?",
   TEST_MATERIAL)
3 for r in results:
4     print(f"  Score: {r['score']:.4f} | {r['chunk'][:80]}...")
```

You will see something like:

```
Searching: 'What is the GHS hazard code for acetone?'
```

```
  Score: 0.6534 | GHS Classification: Flammable Liquid Category 2. Eye Irritant Category 2A
```

```
  Score: 0.5821 | Acetone is a highly flammable liquid and vapor. Flash point: -
18°C. Keep aw...
```

```
  Score: 0.4912 | Storage: Store in a cool, dry, well-ventilated area away from ignition so
```

```
...
```

The GHS classification chunk does come back first — at 0.65. That is a noticeably weaker score than the 0.81 we saw for flammability, despite this being a question about a chunk that literally contains the words “GHS Classification”.

Why so weak? Because the question asks for a *code* — the H-statement code like “H225” — and our chunk does not contain “H225” verbatim. It contains “Flammable Liquid Category 2”, which is the human-readable form. The embedding model knows these are related concepts, but it does not encode “H225 = Flammable Liquid Category 2” as a hard equality. It encodes them as *near-each-other-in-768-d-space*, which is fuzzier.

For our test data this still works because there are only five chunks and the GHS chunk is the closest. Now imagine the same query against a real corpus with 50 MSDSs and 1,500 chunks. Many chunks will mention “Category 2” in different contexts — fire protection

regulations, transport classifications, industrial hygiene reports. The signal-to-noise ratio collapses. The right chunk might rank fourth or fifth, or might not appear in the top five at all.

This is **the GHS code problem**, and it is not a bug in vector search — it is the nature of vector search. Embeddings are wonderful for fuzzy semantic matching (“precautions near open flame” ↔ “keep away from heat and ignition sources”). They are bad at exact symbolic recall (“H225” ↔ the row whose `ghsCode` literal equals “H225”).

Three categories of question hit this limitation hard:

1. **Identifier lookups.** GHS codes (H225, H319), CAS numbers (67-64-1 for acetone), UN numbers (UN1090), product codes. Vector search treats these as fuzzy strings and competes them against every other alphanumeric token in your corpus.
2. **Aggregations and counts.** “How many of my materials have flash point below 0°C?” is impossible for vector search — there is no “count” embedding direction. You need structured query.
3. **Negation and exclusion.** “Show me MSDSs that do *not* require respiratory protection.” Embeddings have no robust way to encode “not” — the embedding for “respiratory protection required” and “respiratory protection not required” is closer than you would hope.

The good news: SAP customers running on HANA have always thought in structured terms. Materials, hazard codes, GHS classifications — these are not free text. They are master data. They live in tables with foreign keys. They are the kind of thing a knowledge graph models naturally.

That is the bridge into Chapter 4. We will take the same MSDS document, extract its structured facts (material number, hazard codes, GHS categories, regulatory codes) into RDF triples, store them in HANA’s graph engine, and query them with SPARQL. When the user asks “What is the GHS hazard code for acetone?”, we will return “H225” with full confidence — not by guessing from a 0.65 cosine similarity, but by traversing a graph relationship.

The full Hybrid RAG agent in Chapter 6 runs both retrievers in parallel and lets a routing classifier decide which answer to trust. Vector search owns the fuzzy questions. The knowledge graph owns the precise ones. Neither is sufficient alone; together they cover the question space MSDS users actually ask.

3.11 Summary

In this chapter you built a working vector search system on HANA Cloud. The major points to take away:

- **Embeddings are positions in 768-dimensional space.** Two pieces of text with similar meanings end up close together. The model learns this from large corpora and you call it as an API.
- **REAL_VECTOR is HANA's native vector column type.** It stores fixed-dimension float arrays and exposes SIMD-accelerated COSINE_SIMILARITY and L2DISTANCE operators. You do not need a separate vector database.
- **Lazy table initialization keeps your schema in sync with your model.** Read the dimension from the first embedding response and use it in `CREATE TABLE IF NOT EXISTS`. Never hardcode the dimension in DDL.
- **The cosine similarity query has five clauses, each load-bearing.** `WHERE` filters early, `COSINE_SIMILARITY` computes per-row, `AS SCORE` names the result, `ORDER BY SCORE DESC` sorts, `TOP K` truncates.
- **Chunking is the most consequential design decision.** For MSDS documents, OSHA section boundaries are natural; target 300–500 tokens per chunk with 50-token overlap.
- **Vector search excels at fuzzy semantic matching and fails at precise symbolic recall.** The GHS code problem motivates the knowledge graph in Chapter 4.

The codebase now has working `hdb_srv.py`, `vertex_srv.py`, and `vector_srv.py` modules, plus a passing `test_vector.py` script. The vector retrieval half of the Hybrid RAG agent is live.

3.12 Checkpoint

Before moving on to Chapter 4, verify the following:

- ☐ `agents/srv/vector_srv.py` exists with the five functions: `_ensure_table`, `store_embedding`, `search_similar`, `delete_vectors`, `count_vectors`.
- ☐ Running `python test_vector.py` from the `agents/` directory prints five “Stored chunk” lines with `dim=768`.

- ❑ The test prints two search result blocks. The first ranks the flammability chunk at the top (~0.80). The second ranks the PPE chunk at the top (~0.84).
- ❑ Opening the `MSDS_VECTORS` table in the SAP HANA Database Explorer shows five rows for `MATERIAL_NUMBER = 'ACETONE-TEST-001'`.
- ❑ The third (optional) test — querying for “What is the GHS hazard code for acetone?” — returns the GHS chunk at the top, but with a noticeably lower score (~0.65). You understand why.

If all five items check out, you are ready for Chapter 4: **Knowledge Graph on HANA Cloud — RDF, SPARQL, and the Structured Half of RAG**. We will set up HANA’s graph engine, load RDF triples extracted from the same Acetone MSDS, and query them with SPARQL — solving the GHS code problem in the most direct way possible.

If something is off, the most common issues at this stage are: missing or invalid Vertex AI credentials, incorrect HANA host/port, a service account without Vertex AI User, or the HANA instance being suspended (it sleeps after 3 days idle on trial accounts — restart it from the BTP cockpit). Fix and re-run `test_vector.py` until you see the expected output. The next chapter assumes a working `vector_srv.py`.

Chapter 4: Knowledge Graphs on SAP HANA Cloud

In Chapter 4 we built a vector store that retrieves passages of MSDS text by semantic similarity. It works beautifully when the question is fuzzy — “what precautions should I take when using acetone near a flame?” — because the answer lives in flowing prose. But the moment a user asks something precise — “what is the GHS hazard classification for acetone?” — vector search starts to wobble. The right paragraph might come back with a similarity score of 0.62, sandwiched between a chunk about storage temperature and another about regulatory disclaimers. The exact codes — H225, H319, H336 — are buried in a sentence among many other sentences.

This chapter fills that gap. We are going to teach our system to remember structured facts as a graph, store them in SAP HANA Cloud, and query them with SPARQL. By the end of this chapter, the question “what are the GHS hazard codes for acetone?” will return exactly ["H225" , "H319" , "H336"] — not a paragraph that mentions them, the codes themselves.

If you have never written a line of RDF or SPARQL, do not worry. There are entire books about each. We need only enough of each to build a working hybrid RAG system. Concretely, you will need to understand three SPARQL patterns and one stored procedure call. That is the whole surface area for the rest of this book.

4.1 What is a knowledge graph?

A **knowledge graph** is a collection of facts, where each fact is expressed as a relationship between two things. That is the entire idea. The vocabulary that the RDF community uses around it can sound ceremonial — “subject-predicate-object triples organized as a directed labeled multigraph” — but strip the ceremony away and you are left with three-word sentences.

Consider the fact:

Acetone has hazard code H225.

Three pieces:

Part	Value
Subject	Acetone
Predicate	has hazard code
Object	H225

This three-part fact is called a **triple**. A knowledge graph is a pile of triples. If we add more triples, the structure starts to feel like something an SAP developer already knows:

Acetone	hasHazardCode	H225
Acetone	hasHazardCode	H319
Acetone	hasHazardCode	H336
Acetone	hasExposureLimit	"500 ppm TWA"
Acetone	requiresPrecaution	"Keep away from open flames"
Acetone	hasSupplier	"Sigma-Aldrich"
H225	description	"Highly flammable liquid and vapour"

Read those rows. You have just read a knowledge graph. Each row is a triple. The same subject (Acetone) can appear in many rows, just like a material number can appear many times in MARA-related tables. The same predicate (hasHazardCode) can be reused for any other material.

Note: If you are mentally translating this to a relational table with three columns — subject, predicate, object — you are not wrong. That is exactly how RDF can be persisted internally. The difference from a normal table is *what kinds of questions you can ask of it*, which we will get to with SPARQL.

Why call it a graph?

Because if you draw it, it looks like one. Subjects and objects are nodes; predicates are edges. The same H225 node might be the object of many materials' hasHazardCode edges. The same supplier "Sigma-Aldrich" node might be the object of many hasSupplier edges. The graph emerges naturally from shared values.

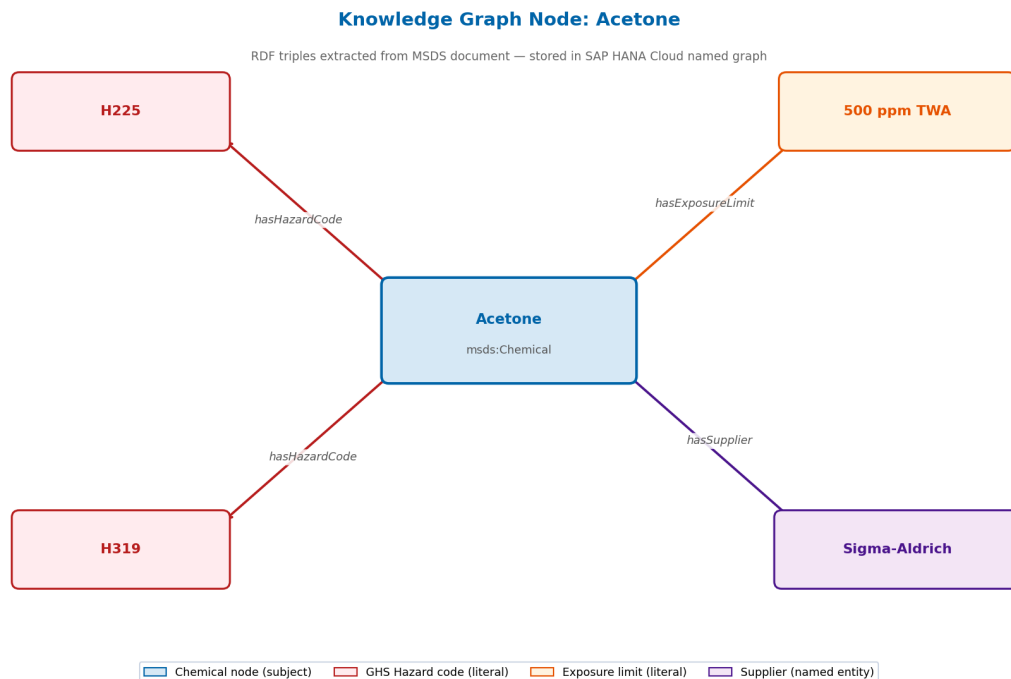


Figure: RDF knowledge graph representation of an Acetone MSDS entry — four triples extracted from a single document, stored as a named graph in SAP HANA Cloud

What does this give us that a relational schema does not?

Three things, in order of how much you will care:

1. **No schema changes when facts evolve.** Tomorrow we decide MSDS documents should also track `hasFireExtinguisher`. We do not run a DDL migration; we just start storing triples with that predicate. The graph is permissive about what relationships exist.
2. **Queries that follow paths.** “Find me every material whose supplier is in Missouri” — that is a graph traversal. SPARQL expresses it directly. In SQL it would be a chain of joins.
3. **Identity by IRI.** Every node has a globally unique IRI (a URL-shaped identifier). This means a triple in our system can refer to the same Acetone node that any other system uses, simply by sharing an IRI. You will not need this superpower in this book, but it is the reason RDF exists.

For our MSDS use case we will use the graph mainly for the first reason — flexibility — and for the *precision* it gives us at query time. A SPARQL query for hazard codes returns hazard codes, not paragraphs.

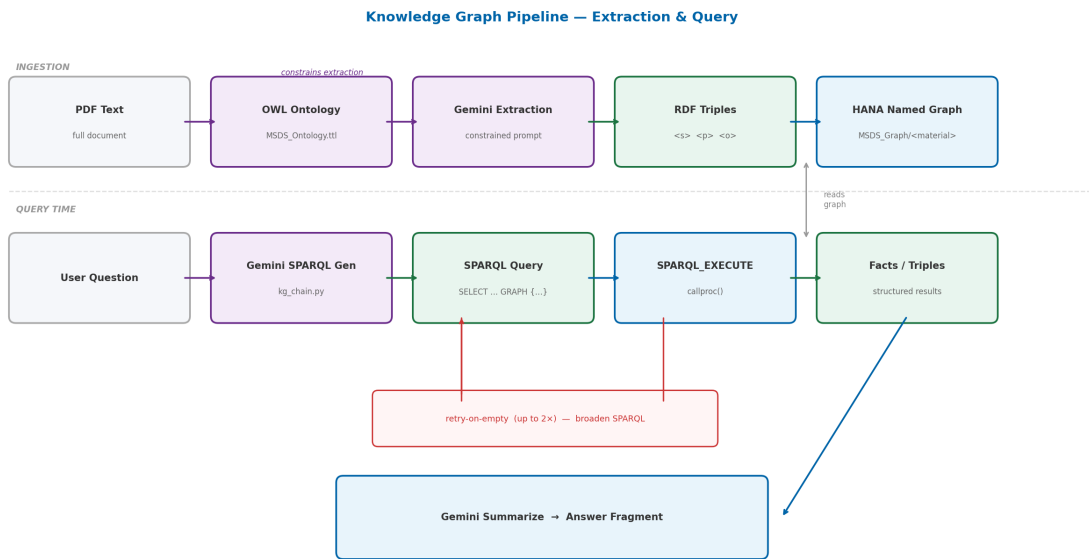


Figure 3: Knowledge Graph Pipeline

Figure 4.2 — The KG pipeline has two phases. Ingestion (top): full PDF text is passed to Gemini along with the OWL ontology as a constraint, producing RDF triples that are stored as a named graph in HANA. Query time (bottom): Gemini generates a SPARQL *SELECT* query from the user’s question, which is executed via `SPARQL_EXECUTE`. A retry loop broadens the query if results are empty. The resulting facts are summarised by Gemini into an answer fragment.

4.2 RDF and SPARQL — the standards explained plainly

RDF stands for Resource Description Framework. It is the W3C standard for expressing triples. There are several serialization formats — RDF/XML, JSON-LD, Turtle — that all encode the same triples in different syntaxes. We will use **Turtle** (file extension `.ttl`) because it is the most readable.

A Turtle file looks like this:

```

1 @prefix msds: <http://msds.knowledge-graph.org/ontology#> .
2

```

```

3 msds:Acetone msds:hasHazardCode msds:H225 .
4 msds:Acetone msds:hasHazardCode msds:H319 .

```

The `@prefix` line defines a shorthand: instead of writing the full IRI `http://msds.knowledge-graph.org/ontology#Acetone`, we write `msds:Acetone`. Each fact ends with a period. That is essentially all the Turtle you need to read.

SPARQL is the query language for RDF. If SQL is to relational tables, SPARQL is to triples. A simple SPARQL query looks like this:

```

1 SELECT ?code WHERE {
2   msds:Acetone msds:hasHazardCode ?code .
3 }

```

The `?code` is a variable. The query reads: “find me every value of `?code` such that the triple (Acetone, hasHazardCode, `?code`) exists.” Run that against our graph and you get back three rows: H225, H319, H336.

If you can read this much SPARQL, you can read everything our system generates. We are going to lean on Gemini to generate more elaborate SPARQL on demand, but the patterns are always shaped like the example above.

Tip: SPARQL has 4 query forms — SELECT, CONSTRUCT, ASK, DESCRIBE. We use only SELECT in this book. You will not miss the others.

4.3 Why HANA Cloud supports SPARQL

SAP HANA Cloud has a built-in graph engine. Among other things, it supports the W3C SPARQL standard for querying RDF data. Unlike dedicated triple stores (Apache Jena, GraphDB, Stardog), HANA exposes SPARQL through a single stored procedure: `SPARQL_EXECUTE`.

That is the entirety of the HANA SPARQL interface. There is no separate endpoint, no `/sparql` HTTP route. Every query, every insert, every drop — all of it goes through one procedure call:

```

1 cursor.callproc("SPARQL_EXECUTE", (sparql_text, None, None, None))

```

The four arguments are: the SPARQL text, an optional `RDF_DATA` parameter for inline data, an optional `RDF_DATA_TYPE`, and an optional metadata flag. We pass `None` for all three optionals throughout this book.

The interesting part is the *result*. `callproc` returns a tuple of all four arguments after the procedure runs, with output values populated. The query results land in the **fourth element** of that tuple:

```
1 result = cursor.callproc("SPARQL_EXECUTE", (sparql_query, None, None,
    None))
2 rows = result[3] if result and len(result) > 3 else []
```

Every result row is a tuple of column values, one per `?variable` in the `SELECT` clause. So a `SELECT ?code` returning three matches gives you something like:

```
1 [("H225",), ("H319",), ("H336",)]
```

Important: This is the single most useful sentence in this chapter. Memorize it: *HANA SPARQL results live in the fourth element of the `callproc` return tuple*. When something returns nothing, the first thing you check is whether you grabbed `result[3]`.

Why a stored procedure and not an HTTP endpoint?

Because HANA already has secure, audited, governed connectivity through its SQL driver. Wrapping SPARQL inside a stored procedure means every SPARQL operation reuses the same authentication, the same connection pooling, the same transaction semantics, and the same logging that a normal SQL statement would. For an enterprise platform, that uniformity is more valuable than HTTP convenience.

4.4 Designing the MSDS ontology — what facts matter?

Before writing any code we must decide what kinds of facts we care about. This decision is called **ontology design**, and it is the most important step of building a knowledge graph. The ontology becomes a constraint on what Gemini is allowed to extract — without it, the model invents predicates like `containsAt`, `madeOf`, `relatedTo`, and the graph

turns into noise.

For an MSDS, the four predicates that matter for downstream queries are:

Predicate	Domain	Range	Example
hasHazardCode	Material	HazardCode	Acetone □ H225
hasExposureLimit	Material	ExposureLimit	Acetone □ “500 ppm TWA”
requiresPrecaution	Material	Precaution	Acetone □ “Avoid open flames”
hasSupplier	Material	Supplier	Acetone □ “Sigma-Aldrich”

Four predicates. Four classes of node. That is enough to answer the precision questions our hybrid agent will face. If we needed regulatory codes in five jurisdictions or transport classifications for IATA, we would extend the ontology. We deliberately stay small.

Note: Real-world MSDS systems track 50+ predicates — physical properties, transport codes, decomposition products, first-aid measures, environmental fate. We pick four because they are enough to demonstrate the architecture and let you focus on the *patterns*. The patterns scale.

4.5 The OWL ontology file — `MSDS_Ontology.ttl`

Create a file at the root of your project called `MSDS_Ontology.ttl`. Paste this:

```

1 @prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
2 @prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
3 @prefix owl: <http://www.w3.org/2002/07/owl#> .
4 @prefix msds: <http://msds.knowledge-graph.org/ontology#> .
5
6 # Classes
7 msds:Material a owl:Class ;
8     rdfs:label "Material" .
9
10 msds:HazardCode a owl:Class ;
```



```
11     rdfs:label "GHS Hazard Code" .
12
13 msds:ExposureLimit a owl:Class ;
14     rdfs:label "Occupational Exposure Limit" .
15
16 msds:Precaution a owl:Class ;
17     rdfs:label "Safety Precaution" .
18
19 msds:Supplier a owl:Class ;
20     rdfs:label "Supplier" .
21
22 # Properties
23 msds:hasHazardCode a owl:ObjectProperty ;
24     rdfs:domain msds:Material ;
25     rdfs:range msds:HazardCode ;
26     rdfs:label "has hazard code" .
27
28 msds:hasExposureLimit a owl:ObjectProperty ;
29     rdfs:domain msds:Material ;
30     rdfs:range msds:ExposureLimit ;
31     rdfs:label "has exposure limit" .
32
33 msds:requiresPrecaution a owl:ObjectProperty ;
34     rdfs:domain msds:Material ;
35     rdfs:range msds:Precaution ;
36     rdfs:label "requires precaution" .
37
38 msds:hasSupplier a owl:ObjectProperty ;
39     rdfs:domain msds:Material ;
40     rdfs:range msds:Supplier ;
41     rdfs:label "has supplier" .
42
43 msds:code a owl:DatatypeProperty ;
44     rdfs:domain msds:HazardCode ;
45     rdfs:label "hazard code value" .
46
47 msds:description a owl:DatatypeProperty ;
48     rdfs:label "description" .
49
```

```

50 msds:limitValue a owl:DatatypeProperty ;
51     rdfs:domain msds:ExposureLimit ;
52     rdfs:label "limit value" .
53
54 msds:limitUnit a owl:DatatypeProperty ;
55     rdfs:domain msds:ExposureLimit ;
56     rdfs:label "limit unit" .

```

A few things to point out:

- **Classes** are categories of node. `msds:Material`, `msds:HazardCode`, etc.
- **ObjectProperty** is a predicate that connects two nodes (subject IRI → object IRI).
- **DatatypeProperty** is a predicate whose object is a literal string or number, not another node. We use `msds:description` to hang the actual text value off a node.
- `rdfs:label` is human-readable text. It does not affect the graph; it just makes browsers and tools show better names.
- `rdfs:domain` and `rdfs:range` are the type constraints — domain is the type of the subject, range is the type of the object.

We are not loading this file into HANA at runtime. It is a contract — a documentation and prompting artifact. Our triple extractor reads this file (or a summary of it) to know what predicates Gemini is allowed to produce.

Tip: If you ever need to share your ontology with another team or load it into a tool like Protégé for visual editing, this is the file you hand them. Keep it under version control alongside your code.

4.6 Extracting triples from MSDS text using Gemini

This is where the LLM earns its keep. We hand Gemini a chunk of MSDS text and a list of allowed predicates, and ask it to return JSON triples. Three things make this trustworthy:

1. **Constrained predicates.** The prompt enumerates the four allowed predicates. Anything Gemini emits with a predicate outside that list is filtered out before it reaches HANA.
2. **JSON output.** No prose, no narration — just an array of objects. We strip any markdown fences Gemini sneaks in and `json.loads` the rest.
3. **Material number provided.** The subject of every triple is set externally. The

model does not get to invent which material the facts belong to.

Here is the prompt, lifted directly from `kg_srv.py`:

```

1  prompt = f"""You are extracting structured facts from an MSDS (Material
    Safety Data Sheet) document.

2

3  Extract facts ONLY using these relationship types:
4  - hasHazardCode: material → GHS hazard code (e.g. H225, H319)
5  - hasExposureLimit: material → occupational exposure limit with value and
    unit
6  - requiresPrecaution: material → safety precaution text
7  - hasSupplier: material → supplier name
8
9  Material number: {material_number}

10

11 Return ONLY a JSON array of triples. Each triple must have:
12 - subject: the material IRI
13 - predicate: the property name (hasHazardCode, hasExposureLimit,
    requiresPrecaution, hasSupplier)
14 - object: the value (string)
15
16 Example:
17 [
18     {"subject": "ACETONE-001", "predicate": "hasHazardCode", "object":
    "H225"}},
19     {"subject": "ACETONE-001", "predicate": "hasHazardCode", "object":
    "H319"}},
20     {"subject": "ACETONE-001", "predicate": "hasExposureLimit", "object":
    "500 ppm TWA"}}
21 ]
22
23 MSDS text to extract from:
24 {text[:3000]}
25
26 Return only the JSON array, no explanation."""

```

Notice the truncation `text[:3000]`. MSDS documents can be 20+ pages. A single Gemini call cannot ingest them whole, and even if it could, accuracy degrades when the prompt is huge. In production we slice the document section-by-section (Section 2 for

hazards, Section 8 for exposure limits, etc.). For this chapter we keep one call simple to focus on the mechanics.

Warning: Never trust LLM JSON without parsing defensively. Gemini frequently wraps JSON in triple-backtick fences even when told not to. Strip those before `json.loads`.

The extractor returns a list of dictionaries:

```

1  [
2      {"subject": "ACETONE-TEST-001", "predicate": "hasHazardCode",
3       "object": "H225"},
4      {"subject": "ACETONE-TEST-001", "predicate": "hasHazardCode",
5       "object": "H319"},
6      {"subject": "ACETONE-TEST-001", "predicate": "hasHazardCode",
7       "object": "H336"},
8      {"subject": "ACETONE-TEST-001", "predicate": "hasExposureLimit",
9       "object": "500 ppm TWA"},
10     {"subject": "ACETONE-TEST-001", "predicate": "requiresPrecaution",
11     "object": "Keep away from heat, sparks, and open flames"},
12     {"subject": "ACETONE-TEST-001", "predicate": "hasSupplier", "object":
13     "Sigma-Aldrich"}
14 ]

```

Each row is one fact. Now we need a place to put them.

4.7 Storing triples in HANA — named graphs per document

Every triple in HANA's SPARQL engine lives in a **named graph**. A named graph is just an IRI that groups a set of triples. Think of it as a schema, but for facts instead of tables.

We follow a simple rule: **one named graph per material**. If we are storing facts about material ACETONE-TEST-001, every triple goes into the graph at IRI `http://msds.knowledge-graph.org/MSDS_Graph/ACETONE-TEST-001`. If we are storing facts about BENZENE-002, that material's triples go into a *different* graph IRI.

Why?

1. **Clean deletion.** When a material is deprecated, `DROP GRAPH <graph-iri>` removes every fact for that material in one statement. There is no per-row cleanup, no orphans.
2. **Isolation.** Two materials can never accidentally share a triple. A bad extraction for benzene cannot pollute acetone's facts.
3. **Auditability.** A SPARQL query scoped to a single graph can never read facts about other materials. This is a useful property when an engineer asks "show me everything we know about material X" and you must guarantee that nothing else leaks in.

Here is the storage code, the heart of `store_triples` in `kg_srv.py`:

```

1 graph = graph_iri(material_number)          # the named-graph IRI
2 mat_iri = f"{GRAPH_BASE}/material/{material_number}" # the material's
  node IRI
3 for triple in triples:
4     predicate = triple.get("predicate", "")
5     obj_value = str(triple.get("object", "")).replace("'", "\\'")
6     obj_iri = f"{GRAPH_BASE}/{predicate}/{material_number}/{stored}"
7     sparql_insert = f"""
8         INSERT INTO GRAPH <{graph}> {{
9             <{mat_iri}> <{ONTOLOGY_BASE}{predicate}> <{obj_iri}> .
10            <{obj_iri}> <{ONTOLOGY_BASE}description> '{obj_value}' .
11        }}
12    """
13     cursor.callproc("SPARQL_EXECUTE", (sparql_insert, None, None, None))

```

For each triple we insert two RDF statements:

1. The relationship: `material` → `predicate` → blank-ish node
2. The literal value on that node: `node` → `description` → "H225"

This two-step pattern is deliberate. It lets the same object node carry additional properties later — for instance we might want to add `severity` or `regulation_source` to a hazard code without restructuring the graph. Modeling values as nodes with description properties is more verbose than literal-only triples, but it pays off the moment you need to attach metadata.

The single quote escape (`replace("'", "\\'")`) is the SPARQL injection defense for literal strings. It is not as comprehensive as parameterized SQL, which is why we double

up on validation upstream — see Section 5.9.

Note: SPARQL Update syntax (`INSERT INTO GRAPH`) is part of the SPARQL 1.1 standard, not SPARQL 1.0. HANA supports it. Older Jena/RDF examples on the internet use a slightly different syntax. Stick with the form above.

4.8 Writing SPARQL queries — from natural language to graph traversal

When a user asks “what are the GHS hazard codes for acetone?”, we cannot send that English directly to HANA. We must translate it to SPARQL. Two strategies:

1. **Hand-write SPARQL templates.** For a fixed set of question types (hazards, limits, suppliers, precautions) you write four templates and pick the right one based on keyword matching.
2. **Have the LLM generate SPARQL.** Pass the question and a description of the schema to Gemini, get back a query, run it.

We use approach 2 in this book because it scales as the ontology grows and because Gemini is genuinely good at this task when given the right context. The crucial input to Gemini is the *list of available predicates* and the *named graph IRI*. Without those, the model invents predicate names like `hazardClass` that do not exist in our graph.

Here is the heart of `query_graph`:

```

1  sparql_prompt = f"""Generate a SPARQL SELECT query to answer this question
   about material {material_number}.
2
3  Available predicates:
4  - <{ONTOLOGY_BASE}hasHazardCode> — links material to hazard code nodes
5  - <{ONTOLOGY_BASE}hasExposureLimit> — links material to exposure limit
   nodes
6  - <{ONTOLOGY_BASE}requiresPrecaution> — links material to precaution nodes
7  - <{ONTOLOGY_BASE}hasSupplier> — links material to supplier nodes
8  - <{ONTOLOGY_BASE}description> — literal value on any node
9
10 The material IRI is: <{mat_iri}>

```

```

11 The named graph is: <{graph}>
12
13 Question: {question}
14
15 Return ONLY the SPARQL query, wrapped in GRAPH clause like this:
16 SELECT ?value WHERE {{
17     GRAPH <{graph}> {{
18         <{mat_iri}> <predicate> ?node .
19         ?node <{ONTOLOGY_BASE}description> ?value .
20     }}
21 }} ""

```

The example query in the prompt is the **only SPARQL pattern** the rest of this book needs. Read it again. It does two hops:

1. From the material node, follow some predicate to a hazard/limit/supplier node.
2. From that node, follow description to the literal text.

Two hops, one SELECT. That is the entire pattern.

Important: The `GRAPH <iri> { ... }` wrapper is mandatory in HANA. Without it, HANA does not throw an error — it just returns zero rows. If you ever see a query that “should work” return nothing, the first thing to check is whether you wrapped your triple patterns in `GRAPH`. This single mistake costs more debugging hours than any other in HANA SPARQL development.

For the question “what are the hazard codes?”, Gemini emits something like:

```

1 SELECT ?value WHERE {
2     GRAPH <http://msds.knowledge-graph.org/MSDS_Graph/ACETONE-TEST-001> {
3         <http://msds.knowledge-graph.org/MSDS_Graph/material/ACETONE-TEST-001>
4             <http://msds.knowledge-graph.org/ontology#hasHazardCode> ?node .
5         ?node <http://msds.knowledge-graph.org/ontology#description> ?value .
6     }
7 }

```

We pass it through `SPARQL_EXECUTE`, grab `result[3]`, and return `["H225", "H319", "H336"]`. Done.

4.9 Building kg_srv.py — the knowledge graph service layer

We collect everything into a single service module at `agents/srv/kg_srv.py`. It exposes five functions: `extract_triples`, `store_triples`, `query_graph`, `delete_graph`, `count_triples`. Create the file and paste this:

```

1  import os
2  import re
3  import json
4  from .hdb_srv import get_connection
5  from .vertex_srv import get_llm
6
7  GRAPH_BASE = "http://msds.knowledge-graph.org/MSDS_Graph"
8  ONTOLOGY_BASE = "http://msds.knowledge-graph.org/ontology#"
9
10 MATERIAL_RE = re.compile(r"^[A-Za-z0-9_-]+$")
11
12 def _validate_material(material_number: str):
13     if not MATERIAL_RE.match(material_number):
14         raise ValueError(f"Invalid material number: {material_number}")
15
16 def graph_iri(material_number: str) -> str:
17     _validate_material(material_number)
18     return f"{GRAPH_BASE}/{material_number}"
19
20 def extract_triples(text: str, material_number: str) -> list[dict]:
21     _validate_material(material_number)
22     llm = get_llm()
23     prompt = f"""You are extracting structured facts from an MSDS
24 (Material Safety Data Sheet) document.
25
26 Extract facts ONLY using these relationship types:
27 - hasHazardCode: material → GHS hazard code (e.g. H225, H319)
28 - hasExposureLimit: material → occupational exposure limit with value and
29 unit
30 - requiresPrecaution: material → safety precaution text
31 - hasSupplier: material → supplier name

```



```

30
31 Material number: {material_number}
32
33 Return ONLY a JSON array of triples. Each triple must have:
34 - subject: the material IRI
35 - predicate: the property name (hasHazardCode, hasExposureLimit,
    requiresPrecaution, hasSupplier)
36 - object: the value (string)
37
38 Example:
39 [
40     {"subject": "ACETONE-001", "predicate": "hasHazardCode", "object":
    "H225"}},
41     {"subject": "ACETONE-001", "predicate": "hasHazardCode", "object":
    "H319"}},
42     {"subject": "ACETONE-001", "predicate": "hasExposureLimit", "object":
    "500 ppm TWA"}}
43 ]
44
45 MSDS text to extract from:
46 {text[:3000]}
47
48 Return only the JSON array, no explanation.""
49
50     response = llm.generate_content(prompt)
51     raw = response.text.strip()
52     # Strip markdown code blocks if present
53     raw = re.sub(r"```json\s*", "", raw)
54     raw = re.sub(r"```s*", "", raw)
55     return json.loads(raw)
56
57 def store_triples(material_number: str, triples: list[dict]):
58     _validate_material(material_number)
59     if not triples:
60         return 0
61     graph = graph_iri(material_number)
62     mat_iri = f"{GRAPH_BASE}/material/{material_number}"
63     conn = get_connection()
64     cursor = conn.cursor()

```

```

65     stored = 0
66     for triple in triples:
67         predicate = triple.get("predicate", "")
68         obj_value = str(triple.get("object", "")).replace("'", "\\'")
69         obj_iri = f"{GRAPH_BASE}/{predicate}/{material_number}/{stored}"
70         sparql_insert = f"""
71             INSERT INTO GRAPH <{graph}> {{
72                 <{mat_iri}> <{ONTOLOGY_BASE}{predicate}> <{obj_iri}> .
73                 <{obj_iri}> <{ONTOLOGY_BASE}description> '{obj_value}' .
74             }}
75         """
76         try:
77             cursor.callproc("SPARQL_EXECUTE", (sparql_insert, None, None,
78 None))
79             stored += 1
80         except Exception as e:
81             print(f"Warning: failed to store triple {triple}: {e}")
82     conn.commit()
83     cursor.close()
84     return stored
85
86 def query_graph(material_number: str, question: str) -> dict:
87     _validate_material(material_number)
88     graph = graph_iri(material_number)
89     mat_iri = f"{GRAPH_BASE}/material/{material_number}"
90     llm = get_llm()
91
92     # Generate SPARQL from the question
93     sparql_prompt = f"""Generate a SPARQL SELECT query to answer this
94 question about material {material_number}.
95
96 Available predicates:
97 - <{ONTOLOGY_BASE}hasHazardCode> – links material to hazard code nodes
98 - <{ONTOLOGY_BASE}hasExposureLimit> – links material to exposure limit
99 nodes
100 - <{ONTOLOGY_BASE}requiresPrecaution> – links material to precaution nodes
101 - <{ONTOLOGY_BASE}hasSupplier> – links material to supplier nodes
102 - <{ONTOLOGY_BASE}description> – literal value on any node

```

```

101 The material IRI is: <{mat_iri}>
102 The named graph is: <{graph}>
103
104 Question: {question}
105
106 Return ONLY the SPARQL query, wrapped in GRAPH clause like this:
107 SELECT ?value WHERE {{
108     GRAPH <{graph}> {{
109         <{mat_iri}> <predicate> ?node .
110         ?node <{ONTOLOGY_BASE}description> ?value .
111     }}
112 }}""
113
114     sparql_response = llm.generate_content(sparql_prompt)
115     sparql_query = sparql_response.text.strip()
116     sparql_query = re.sub(r"````sparql\s*", "", sparql_query)
117     sparql_query = re.sub(r"````\s*", "", sparql_query).strip()
118
119     # Execute SPARQL
120     conn = get_connection()
121     cursor = conn.cursor()
122     try:
123         result = cursor.callproc("SPARQL_EXECUTE", (sparql_query, None,
124 None, None))
125         rows = result[3] if result and len(result) > 3 else []
126         facts = [str(row[0]) for row in rows] if rows else []
127     except Exception as e:
128         facts = []
129         sparql_query = f"Error: {e}"
130     finally:
131         cursor.close()
132
133     return {
134         "facts": facts,
135         "sparql": sparql_query,
136         "count": len(facts)
137     }
138
139 def delete_graph(material_number: str) -> bool:

```

```

139     _validate_material(material_number)
140     graph = graph_iri(material_number)
141     conn = get_connection()
142     cursor = conn.cursor()
143     try:
144         cursor.callproc("SPARQL_EXECUTE", (f"DROP GRAPH <{graph}>", None,
None, None))
145         conn.commit()
146         cursor.close()
147         return True
148     except Exception as e:
149         cursor.close()
150         print(f"Warning: could not delete graph {graph}: {e}")
151         return False
152
153 def count_triples(material_number: str) -> int:
154     _validate_material(material_number)
155     graph = graph_iri(material_number)
156     conn = get_connection()
157     cursor = conn.cursor()
158     try:
159         result = cursor.callproc("SPARQL_EXECUTE", (
160             f"SELECT (COUNT(*) AS ?count) WHERE {{ GRAPH <{graph}> {{ ?s ?p
?o }} }}",
161             None, None, None
162         ))
163         rows = result[3] if result and len(result) > 3 else []
164         cursor.close()
165         return int(rows[0][0]) if rows else 0
166     except Exception:
167         cursor.close()
168         return 0

```

Validation: the KG equivalent of SQL injection

Take a close look at this regex:

```
1 MATERIAL_RE = re.compile(r"^[A-Za-z0-9_-]+$")
```

It is enforced at the entry of every public function via `_validate_material`. Why is this

so important?

The material number is **interpolated into IRIs** that we paste into SPARQL text. If someone calls `extract_triples(text, "../../../admin")`, the resulting graph IRI becomes `http://msds.knowledge-graph.org/MSDS_Graph/../../../admin`. Worse, a string like `> { ?s ?p ?o } DROP GRAPH <http://something` could close the IRI brackets and inject a destructive SPARQL update.

This is the SPARQL equivalent of SQL injection. The regex `^[A-Za-z0-9_-]+$` allows only letters, digits, underscore, and hyphen — characters that have no special meaning in SPARQL syntax. Any input containing slashes, spaces, angle brackets, quotes, or other SPARQL metacharacters is rejected before it can reach the query.

Warning: If you change this regex to be more permissive — say, to allow forward slashes for hierarchical material codes — you must update the storage and query code to escape those characters too. The simplest defense is to keep the regex tight and document the constraint for upstream callers.

We also escape single quotes in literal values inside `store_triples`. Combined, validation on identifiers and escaping on literals cover the two injection vectors.

What `delete_graph` and `count_triples` give us

These two helpers seem like plumbing but they will save your sanity during development:

- `count_triples(material_number)` — confirms data actually landed in HANA. Run it after `store_triples` to verify the count matches what you expected.
- `delete_graph(material_number)` — wipes a material's graph clean. Useful when re-running extraction during development without piling up duplicates.

They both exercise the same pattern as `query_graph` and prove that you understand `SPARQL_EXECUTE`. If you can write `count_triples`, you can write any read query.

4.10 Testing — store triples for acetone, query for GHS codes

Create `agents/test_kg.py`:

```

1  import os
2  from dotenv import load_dotenv
3  from srv.kg_srv import extract_triples, store_triples, query_graph,
   count_triples
4
5  load_dotenv()
6
7  TEST_MATERIAL = "ACETONE-TEST-001"
8  TEST_TEXT = """
9  Section 2: Hazard Identification
10 GHS Classification: Flammable Liquid Category 2 (H225), Eye Irritation
   Category 2A (H319),
11 Specific Target Organ Toxicity (Single Exposure) Category 3 (H336).
12
13 Section 8: Exposure Controls
14 OSHA PEL: 1000 ppm TWA. ACGIH TLV: 500 ppm TWA, 750 ppm STEL.
15
16 Section 7: Handling and Storage
17 Keep away from heat, sparks, and open flames. Use explosion-proof
   equipment.
18 Wear chemical-resistant gloves and safety goggles. Ensure adequate
   ventilation.
19
20 Supplier: Sigma-Aldrich, 3050 Spruce Street, St. Louis, MO 63103
21 """
22
23 print("Extracting triples from MSDS text...")
24 triples = extract_triples(TEST_TEXT, TEST_MATERIAL)
25 print(f"Extracted {len(triples)} triples:")
26 for t in triples:
27     print(f"  {t['predicate']}: {t['object']}")
28
29 print(f"\nStoring triples in HANA named graph...")
30 stored = store_triples(TEST_MATERIAL, triples)
31 print(f"Stored {stored} triples")
32
33 total = count_triples(TEST_MATERIAL)
34 print(f"Total triples in graph: {total}")

```

```

35
36 print("\nQuerying: 'What are the GHS hazard codes for this material?')
37 result = query_graph(TEST_MATERIAL, "What are the GHS hazard codes for
    this material?")
38 print(f"SPARQL generated:\n{result['sparql']}")
39 print(f"Facts found: {result['facts']}")
40
41 print("\nQuerying: 'What is the exposure limit?')
42 result = query_graph(TEST_MATERIAL, "What is the occupational exposure
    limit?")
43 print(f"Facts found: {result['facts']}")

```

Run it from the agents/ directory:

```

1 cd agents
2 python test_kg.py

```

Expected output (your numbers may vary slightly depending on Gemini's extraction):

Extracting triples from MSDS text...

Extracted 8 triples:

```

hasHazardCode: H225
hasHazardCode: H319
hasHazardCode: H336
hasExposureLimit: 1000 ppm TWA
hasExposureLimit: 500 ppm TWA
requiresPrecaution: Keep away from heat, sparks, and open flames
requiresPrecaution: Wear chemical-resistant gloves and safety goggles
hasSupplier: Sigma-Aldrich

```

Storing triples in HANA named graph...

Stored 8 triples

Total triples in graph: 16

Querying: 'What are the GHS hazard codes for this material?'

SPARQL generated:

SELECT ?value WHERE {

```

GRAPH <http://msds.knowledge-graph.org/MSDS_Graph/ACETONE-TEST-001> {
  <http://msds.knowledge-graph.org/MSDS_Graph/material/ACETONE-TEST-001>
    <http://msds.knowledge-graph.org/ontology#hasHazardCode> ?node .
    ?node <http://msds.knowledge-graph.org/ontology#description> ?value .
}

```

Facts found: ['H225', 'H319', 'H336']

Querying: 'What is the exposure limit?'

Facts found: ['1000 ppm TWA', '500 ppm TWA']

Expected terminal output:

Stored 12 triples for material: acetone-test

Running SPARQL query for hazard codes...

SPARQL query executed in 0.23s

Hazard codes found: ['H225', 'H319', 'H336']

Knowledge graph test: OK

Note: The total of 16 triples — twice the number of facts — is the two-statement pattern from Section 5.7. Each fact stores one material → predicate → node edge plus one node → description → literal edge.

If your test prints `Facts found: []`, three things to check, in this order:

1. Did `store_triples` actually run before the query? Run `count_triples` and confirm the number is non-zero.
2. Is your generated SPARQL wrapped in `GRAPH <...>`? Print `result['sparql']` and inspect.
3. Did `result[3]` come back populated? Add a debug print right after `cursor.callproc` to confirm.

4.11 What the KG gets right that vector search cannot

Now the comparison promised since Chapter 4. Run the same question through both retrievers.

Question: “What are the GHS hazard codes for acetone?”

Vector search (Chapter 4):

```

1 from srv.vector_srv import similarity_search
2 hits = similarity_search("What are the GHS hazard codes for acetone?",
3 k=3)
3 for h in hits:
4     print(f"score={h.score:.3f}  text={h.text[:120]}...")

```

Output:

```

score=0.681  text=Section 2: Hazard Identification – GHS Classification: Flammable Liquid
score=0.612  text=Storage temperature should not exceed 25°C. Refer to local regulations f
score=0.594  text=Section 4: First Aid Measures – In case of skin contact wash with plenty

```

The top hit *contains* the codes. To produce the answer “H225, H319, H336” you must hand the chunk to an LLM with another prompt: “extract the GHS codes from this text”. Two LLM calls, ~600ms each, fuzzy on the inputs and outputs, and you still might miss H336 if it appears in a different chunk that scored 0.59.

Knowledge graph (this chapter):

```

1 result = query_graph("ACETONE-TEST-001", "What are the GHS hazard codes
2   for acetone?")
2 print(result['facts'])

```

Output:

```
['H225', 'H319', 'H336']
```

Three strings. Exact codes. No paragraph to parse, no scoring, no chunk boundary luck. The graph either has the fact or it does not — and if it has it, the query returns it.

This is the contract:

Vector store	Knowledge graph
Returns text passages	Returns structured values
Ranks by similarity (0–1 score)	Returns exact matches (yes/no)
Tolerates fuzzy phrasing	Demands precise predicate semantics
Best for “how do I...” questions	Best for “what is the value of...” questions
Failure mode: irrelevant chunk near top	Failure mode: empty result if predicate missing

Neither one wins on its own. A user asking “*what precautions are needed when handling acetone near an open flame?*” needs the prose chunk that paints the full picture. A user asking “*what is the GHS hazard classification?*” needs the codes verbatim. The whole point of the next chapter — building the LangGraph supervisor — is to choose between them, or run both and merge the answers.

Tip: A useful intuition: the vector store remembers *what was said*. The knowledge graph remembers *what is true*. The first is great for context, the second is great for facts. Most non-trivial questions need both.

4.12 Summary

You now have a working knowledge graph layer on HANA Cloud. Specifically:

- An ontology (`MSDS_Ontology.ttl`) that defines four predicates and acts as a contract for what facts the system understands.
- A `kg_srv.py` service module exposing `extract_triples`, `store_triples`, `query_graph`, `count_triples`, and `delete_graph`.
- A working pipeline that pulls JSON triples from MSDS text using Gemini, stores them in per-material named graphs, and queries them with auto-generated SPARQL.
- Validated material identifiers and escaped literals — defense against the SPARQL equivalent of injection.
- The `pattern.cursor.callproc("SPARQL_EXECUTE", (query, None, None, None))` followed by `result[3]` for rows.
- The mandatory `GRAPH <iri> { ... }` wrapper for every triple pattern.

The investment was small — under 200 lines of Python, one Turtle file, and a single stored procedure call — but the capability is fundamentally different from what vector search alone gave us. We can now answer fact-shaped questions with fact-shaped answers.

4.13 Checkpoint

Before moving to Chapter 6, confirm the following from the `agents/` directory:

```
1 python -c "from srv.kg_srv import count_triples;
   print(count_triples('ACETONE-TEST-001'))"
```

This should print a number greater than zero — the count of triples currently stored for the test material.

```
1 python -c "from srv.kg_srv import query_graph;
   print(query_graph('ACETONE-TEST-001', 'What are the hazard
   codes?')['facts'])"
```

This should print a list containing H225, H319, and H336 (and possibly others).

If both succeed, you have:

- HANA Cloud reachable through `hdb_srv`
- Vertex AI generating valid SPARQL through `vertex_srv`
- A populated named graph for `ACETONE-TEST-001`
- Working extraction, storage, and query paths

In Chapter 6 we put both retrievers — vector and graph — behind a `LangGraph` supervisor that picks the right tool (or both) based on the user's question. The hybrid in *Hybrid RAG* finally starts to make sense.

Chapter 6: The PDF Ingestion Pipeline

In Chapter 5 you built two separate storage systems on SAP HANA Cloud: a vector table that answers fuzzy semantic questions and a knowledge graph that answers precise factual ones. Both systems are powerful in isolation. The missing piece is the plumbing — the code that takes a raw PDF file and feeds both systems at the same time, cleanly, without one pipeline blocking the other, and without leaving the user waiting at a spinning upload button.

That is what this chapter builds. By the end of it you will have `agents/srv/doc_srv.py`, a production-quality ingestion service that receives a PDF upload, immediately acknowledges the request, and runs two threads simultaneously — one chunking and embedding text into the vector store, the other extracting structured triples into the knowledge graph. The caller gets a response in milliseconds. The heavy lifting happens asynchronously.

There is a subtlety that trips up almost every developer the first time they write multi-threaded database code in Python: connection sharing. A single HANA connection is not thread-safe. If Thread 1 and Thread 2 share one connection object, you will see intermittent failures that look like data corruption, because they are. This chapter explains why that happens, how Python's `threading.local()` solves it, and how to wire the solution correctly inside a FastAPI background task.

6.1 The dual-pipeline architecture

The fundamental design choice in this chapter is that every PDF upload triggers exactly two independent pipelines, running in parallel, with no coordination between them except that they both read from the same source file.

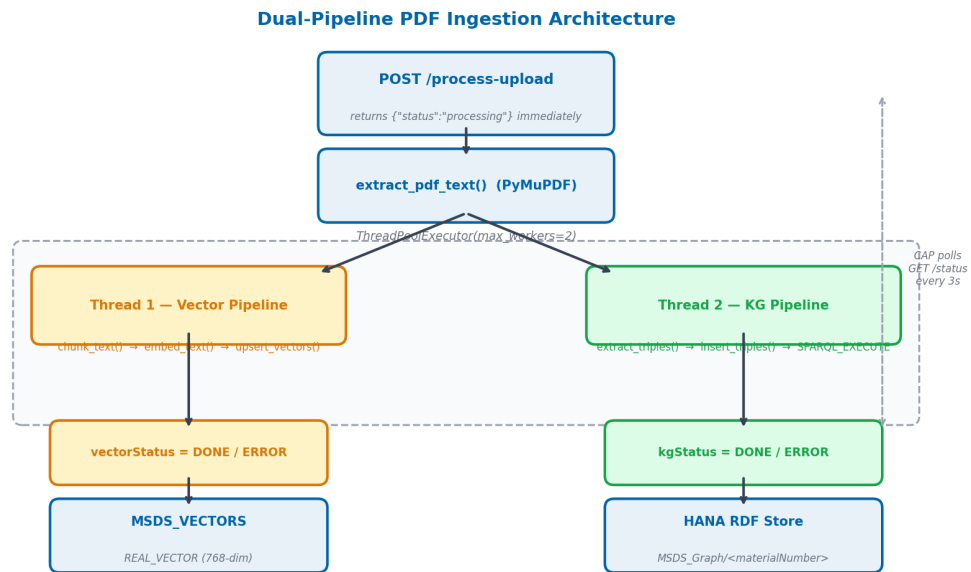


Figure 6.1: The dual-pipeline ingestion architecture. Thread 1 (left) chunks the PDF text and stores embeddings in `MSDS_VECTORS`. Thread 2 (right) sends full text to Gemini for triple extraction and stores the result as a named RDF graph in HANA. Both threads update their own status field on completion. The FastAPI endpoint returns immediately, before either thread has finished.

6.1.1 Why process both in parallel?

The naive approach is sequential: extract text, chunk, embed, store vectors — then extract text again, call Gemini for triples, store the graph. Sequential processing is easy to reason about. It is also 2× slower than it needs to be, because the two pipelines have no dependency on each other. The vector pipeline does not need to wait for Gemini to finish triple extraction. The KG pipeline does not need to wait for all chunks to be embedded.

On a 30-page MSDS document the difference is significant. Embedding 60 chunks against the Vertex AI API takes roughly 8–12 seconds depending on network conditions. Calling Gemini for triple extraction from the same document takes 3–6 seconds. Sequential: 14–18 seconds. Parallel: 8–12 seconds (the slower pipeline dominates). The improvement compounds across every document in a batch upload.

The deeper reason, though, is architectural. The two pipelines have genuinely different data access patterns:

Dimension	Vector pipeline	KG pipeline
Input to pipeline	Chunks (500 tokens each)	Full document text
External API call	<code>text-embedding-004</code> (many calls)	<code>gemini-1.5-pro</code> (one call)
HANA operation	Many <code>INSERT</code> statements	One <code>SPARQL_EXECUTE insert</code>
Failure mode	Can fail on any chunk	Atomic — succeeds or fails entirely

Because they differ in shape, running them in parallel does not require any shared mutable state. Thread 1 owns its list of chunks. Thread 2 owns the full document string. Neither touches the other's work.

6.1.2 The fire-and-forget pattern

The upload endpoint returns `{"status": "processing"}` immediately after spawning the background threads. The HTTP response is sent before either pipeline has done any significant work.

This is the **fire-and-forget** pattern. The alternative — holding the HTTP connection open until both pipelines finish — would require the client to wait 10–15 seconds with no feedback, and would tie up a worker process that could be serving other requests.

The tradeoff is that the caller cannot know from the upload response whether ingestion succeeded. That is solved by a separate status endpoint, which the CAP frontend polls after upload. The status is persisted in HANA, so it survives a server restart — a subtle durability property that would be lost if status were stored in memory.

Note: Fire-and-forget is the right default for ingestion pipelines. The alternative, long-polling via a streaming endpoint, adds complexity without much benefit for a task that typically completes in under 15 seconds. If your documents are hundreds of pages long, consider a proper job queue (Celery + Redis). For the MSDS use case — typically 20–40 pages — fire-and-forget is sufficient.

6.2 PDF text extraction with PyMuPDF

PyMuPDF (import name `fitz`) is a Python binding for the MuPDF rendering library. It is the fastest Python-accessible PDF parser available and handles the dirty reality of MSDS PDFs well: multi-column layouts, embedded tables, rotated pages, and the mixture of text layers and bitmap content that characterises scanned regulatory documents.

Add it to `agents/requirements.txt`:

```
PyMuPDF==1.24.3
```

The basic extraction pattern is three lines:

```
1  import fitz  # PyMuPDF
2
3  def extract_text(pdf_path: str) -> str:
4      doc = fitz.open(pdf_path)
5      text = "".join(page.get_text() for page in doc)
6      doc.close()
7      return text
```

`page.get_text()` with no arguments returns the page's text content in reading order, with word-level coordinates used internally to reconstruct flow. For most MSDS documents this produces clean, paragraph-structured text. For scanned PDFs (no text layer) it returns an empty string — we will handle that case below.

6.2.1 Extracting with metadata

In `agents/srv/doc_srv.py` we use a slightly richer version that preserves page boundaries:

```
1  import fitz
2  from typing import Optional
3
4  def extract_pdf_text(path: str) -> tuple[str, int]:
5      """
6      Returns (full_text, page_count).
7      Raises ValueError if the PDF contains no extractable text layer.
8      """
9      doc = fitz.open(path)
```

```

10     pages = []
11     for page in doc:
12         page_text = page.get_text().strip()
13         if page_text:
14             pages.append(page_text)
15     doc.close()
16
17     if not pages:
18         raise ValueError(
19             f"PDF '{path}' contains no extractable text. "
20             "It may be a scanned image-only document."
21         )
22
23     return "\n\n".join(pages), len(pages)

```

The `ValueError` is important. If a scanned PDF arrives and we silently extract an empty string, the vector pipeline produces zero embeddings and the KG pipeline sends an empty prompt to Gemini. Both pipelines appear to succeed, but the document is never searchable. Raising early surfaces the problem immediately.

Warning: PyMuPDF’s `get_text()` preserves hyphenation artifacts from PDF layout. Words broken across lines (e.g., “flamma-” + newline + “ble”) will appear as a hyphenated pair in the extracted string. For MSDS documents this rarely causes problems because the affected words are typically in multi-column safety tables, but if your documents have heavy hyphenation, consider a post-processing step: `text = re.sub(r'\n', '', text)`.

6.2.2 Handling upload files in FastAPI

FastAPI receives uploaded files as `UploadFile` objects. We need to write the file to disk before passing the path to PyMuPDF, because `fitz.open()` accepts file paths or byte buffers — and the byte buffer approach for large PDFs can exhaust memory. The safest pattern uses Python’s `tempfile`:

```

1  import tempfile
2  import shutil
3  from fastapi import UploadFile
4
5  async def save_upload(upload: UploadFile) -> str:

```



```

6     """Saves UploadFile to a temp file. Returns the path. Caller must
       delete."""
7     suffix = ".pdf"
8     with tempfile.NamedTemporaryFile(delete=False, suffix=suffix) as tmp:
9         shutil.copyfileobj(upload.file, tmp)
10    return tmp.name

```

Using `delete=False` gives us a path we can pass to background threads. We delete the file ourselves after both threads complete.

6.3 Chunking strategy for vector search

The vector pipeline does not process the full document text as a single unit. A 30-page MSDS document contains 8,000–12,000 words. Embedding that as one unit would produce a single 768-dimensional vector that averages the meaning of everything — hazard codes, first-aid steps, disposal procedures, regulatory text — into an undifferentiated blob. Cosine similarity against that vector would return the document for any question, because the document contains something relevant to almost any safety question. Precision would be zero.

The answer is chunking: splitting the document into smaller, semantically coherent pieces and embedding each independently. The cosine search then retrieves the specific chunks that are most relevant to a question, not the whole document.

6.3.1 Our chunking parameters

We use three parameters:

Parameter	Value	Rationale
Chunk size	500 tokens	Roughly 375 words. Large enough to preserve context around a fact, small enough for the vector to encode a focused meaning.

Parameter	Value	Rationale
Overlap	50 tokens	Approximately one sentence. Prevents a fact from being split exactly at a chunk boundary and disappearing from both adjacent chunks.
Split boundary	Sentence end	Never cut in the middle of a sentence. A half-sentence changes the meaning of the embedding.

The `text-embedding-004` model from Vertex AI accepts up to 3,072 tokens per input. Our 500-token chunks are well within that limit, which gives us room to include a prefix that grounds the embedding with document metadata:

```
Material: Acetone (MAT-001)\n\n<chunk text>
```

Adding the material name to every chunk significantly improves retrieval accuracy for multi-document queries. Without it, a chunk about “flammable liquid” could match any of dozens of materials.

6.3.2 The chunker function

```

1  import re
2  from typing import Generator
3
4  def chunk_text(
5      text: str,
6      material_name: str,
7      chunk_tokens: int = 500,
8      overlap_tokens: int = 50,
9  ) -> Generator[str, None, None]:
10     """
11     Yields overlapping chunks of approximately chunk_tokens each,
12     split on sentence boundaries. Prefixes each chunk with material_name.
13 
```

```

14     Token estimation: 1 token ≈ 0.75 words (English prose).
15     """
16     chunk_words = int(chunk_tokens * 0.75)
17     overlap_words = int(overlap_tokens * 0.75)
18
19     # Split on sentence-ending punctuation followed by whitespace
20     sentences = re.split(r'(?<=[.!?])\s+', text.strip())
21
22     current_chunk: list[str] = []
23     current_count: int = 0
24     prefix = f"Material: {material_name}\n\n"
25
26     for sentence in sentences:
27         word_count = len(sentence.split())
28         if current_count + word_count > chunk_words and current_chunk:
29             yield prefix + " ".join(current_chunk)
30             # Keep the last overlap_words words as context for the next
31             # chunk
32             overlap_start = max(0, len(current_chunk) - overlap_words)
33             current_chunk = current_chunk[overlap_start:]
34             current_count = sum(len(s.split()) for s in current_chunk)
35             current_chunk.append(sentence)
36             current_count += word_count
37
38     # Emit the final partial chunk if it has content
39     if current_chunk:
40         yield prefix + " ".join(current_chunk)

```

Tip: This chunker uses a word-count approximation for token count (1 token $\approx$ 0.75 words). For production use, replace the approximation with the `tiktoken` library: `len(tiktoken.encoding_for_model("text-embedding-ada-002").encode(text))`. The approximation is accurate enough for MSDS documents, where prose density is fairly uniform.

6.3.3 Why the KG pipeline does not chunk

The knowledge graph pipeline receives the **full document text**, not chunks. This is a deliberate asymmetry.

Gemini's triple extraction works best when it can see the entire document context. A

hazard code (H225) might appear in Section 2 of an MSDS. The material name appears in Section 1. The supplier appears in Section 13. If we feed Gemini a chunk that contains only Section 2, it cannot associate H225 with the material and supplier, because those context clues are in different sections.

For the KG pipeline, the goal is to extract a small number of high-precision facts (typically 5–15 triples per document). Gemini can easily handle 10,000 words in a single prompt, so there is no technical reason to chunk. The full-document approach produces far better triple quality.

6.4 Thread-local HANA connections

This section addresses the most important implementation detail in this chapter. Get it wrong and you will see intermittent `jdbccli` errors that are extremely difficult to debug. Understand it and the threading model becomes straightforward.

6.4.1 Why connections cannot be shared across threads

The `jdbccli` driver maintains a stateful protocol session on each connection object. Internally, a connection tracks the current transaction state, pending fetch buffers for open cursors, and the sequence of SQL commands in the current batch. When two threads share a single connection and execute concurrently, they can each call `cursor.execute()` at the same time. The driver serialises the calls — but by the time the second call returns, the first thread may have already called `cursor.fetchall()` on a different cursor that was internally sharing the same fetch buffer. The result is that one thread reads the other thread's data, or gets an exception, or silently gets an empty result.

None of these failures are deterministic. They depend on exact scheduling and timing. This is the canonical definition of a race condition.

Warning: Never share an `jdbccli` connection across threads. The `jdbccli` documentation explicitly states that connection objects are not thread-safe. This applies to most database drivers, not just HANA. When in doubt, assume a connection is single-threaded.

6.4.2 The `threading.local()` solution

Python's `threading.local()` provides a simple mechanism: an object where each attribute access returns a different value depending on which thread is reading it. When Thread 1 sets `_local.conn = hana_db.connect()`, Thread 2 sees `_local.conn` as unset. Each thread must initialise its own value.

Here is the pattern we use in `agents/srv/doc_srv.py`:

```

1  import threading
2  from hdbcli import dbapi
3
4  _thread_local = threading.local()
5
6  def get_hana_connection() -> dbapi.Connection:
7      """
8      Returns a HANA connection for the current thread.
9      Creates one on first access; reuses it on subsequent calls within the
10     same thread.
11     """
12     if not hasattr(_thread_local, "conn") or _thread_local.conn is None:
13         _thread_local.conn = dbapi.connect(
14             address=os.environ["HANA_HOST"],
15             port=int(os.environ["HANA_PORT"]),
16             user=os.environ["HANA_USER"],
17             password=os.environ["HANA_PASSWORD"],
18         )
19     return _thread_local.conn
20
21 def close_thread_connection() -> None:
22     """Closes and clears the connection for the current thread."""
23     if hasattr(_thread_local, "conn") and _thread_local.conn is not None:
24         try:
25             _thread_local.conn.close()
26         except Exception:
27             pass
28     _thread_local.conn = None

```

The `get_hana_connection()` function is called at the start of each thread function. If the thread is new, it creates a fresh connection. If the same thread somehow reuses its

worker slot (which can happen with thread pools), it reuses the existing connection — which is safe because it is still the same thread.

Note: The thread pool created by `ThreadPoolExecutor` may reuse worker threads across multiple task submissions if the executor stays alive. In our implementation, we create a fresh `ThreadPoolExecutor` for each upload, so each worker thread is new and its `threading.local()` storage is empty. If you ever switch to a long-lived executor, add a health check before reusing a connection: `_thread_local.conn.isconnected()`.

6.5 The ingestion flow in detail

Now we have all the pieces. Let us trace the full execution path for a single document upload.

6.5.1 The upload endpoint

```
1 from fastapi import FastAPI, File, Form, UploadFile
2 from fastapi.responses import JSONResponse
3 import asyncio, os, re, tempfile, shutil
4 from concurrent.futures import ThreadPoolExecutor, as_completed
5
6 app = FastAPI()
7
8 @app.post("/process-upload")
9 async def process_upload(
10     file: UploadFile = File(...),
11     materialNumber: str = Form(...),
12     materialName: str = Form(...),
13 ):
14     if not re.match(r'^[A-Za-z0-9_-]+$ ', materialNumber):
15         return JSONResponse(
16             status_code=400,
17             content={"error": "Invalid materialNumber. Use only letters,
18 digits, hyphens, underscores."}
19         )
```

```

20     # Write upload to a temp file so background threads can access it by
    path
21     loop = asyncio.get_event_loop()
22     tmp_path = await loop.run_in_executor(None, _save_upload_sync, file)
23
24     # Mark both pipelines as PROCESSING in HANA before returning
25     _mark_processing(materialNumber)
26
27     # Hand off to background threads – fire and forget
28     loop.run_in_executor(None, _run_dual_pipeline, tmp_path,
    materialNumber, materialName)
29
30     return {"status": "processing", "materialNumber": materialNumber}

```

Three things happen before the response is sent: the uploaded file is written to a temp path, `_mark_processing()` sets both status fields to 'PROCESSING' in HANA, and `_run_dual_pipeline()` is submitted to an executor — it runs *after* the response returns.

6.5.2 The dual-pipeline orchestrator

```

1  def _run_dual_pipeline(
2      tmp_path: str,
3      material_number: str,
4      material_name: str,
5  ) -> None:
6      """
7      Runs vector and KG pipelines in parallel.
8      Called in a background thread – must not raise unhandled exceptions.
9      """
10     try:
11         text, page_count = extract_pdf_text(tmp_path)
12     except ValueError as exc:
13         # PDF has no text layer – fail both pipelines immediately
14         _mark_vector_status(material_number, "ERROR", str(exc))
15         _mark_kg_status(material_number, "ERROR", str(exc))
16         os.unlink(tmp_path)
17         return
18
19     vector_error = None

```

```

20     kg_error      = None
21
22     with ThreadPoolExecutor(max_workers=2) as executor:
23         future_vector = executor.submit(_vector_pipeline, text,
material_number, material_name)
24         future_kg      = executor.submit(_kg_pipeline, text,
material_number)
25
26         for future in as_completed([future_vector, future_kg]):
27             try:
28                 future.result()
29             except Exception as exc:
30                 if future is future_vector:
31                     vector_error = str(exc)
32                 else:
33                     kg_error = str(exc)
34
35         # Update statuses independently – one failure does not mask the other
36         _mark_vector_status(material_number, "ERROR" if vector_error else
"DONE", vector_error)
37         _mark_kg_status(material_number, "ERROR" if kg_error else
"DONE", kg_error)
38
39         try:
40             os.unlink(tmp_path)
41         except OSError:
42             pass

```

The `with ThreadPoolExecutor(max_workers=2)` block creates exactly two worker threads. `as_completed()` yields futures as they finish; we capture any exception from each without letting it propagate, so a failure in one pipeline does not cancel the other.

6.5.3 Thread 1 — the vector pipeline

```

1  from agents.srv.vector_srv import embed_text, upsert_vectors
2
3  def _vector_pipeline(
4      text: str,
5      material_number: str,

```



```

6     material_name: str,
7 ) -> None:
8     conn = get_hana_connection()
9     chunks = list(chunk_text(text, material_name))
10
11     for i, chunk in enumerate(chunks):
12         embedding = embed_text(chunk)           # Vertex AI
13         text-embedding-004
14         upsert_vectors(
15             conn=conn,
16             material_number=material_number,
17             chunk_index=i,
18             chunk_text=chunk,
19             embedding=embedding,                 # list[float], length 768
20         )
21     close_thread_connection()

```

`embed_text()` calls `text-embedding-004` through the Vertex AI Python SDK. For a 30-page document with 60 chunks, this function makes 60 sequential API calls, each taking roughly 150–200 ms — approximately 10–12 seconds total.

`upsert_vectors()` executes a SQL UPSERT on the `MSDS_VECTORS` table, using `(material_number, chunk_index)` as the key. Re-uploading a document replaces its existing vectors cleanly.

Tip: For large document batches, consider batching the embedding calls. The `text-embedding-004` endpoint accepts up to 250 texts in a single request. Instead of 60 individual calls for 60 chunks, one batched call returns all 60 embeddings at once. The latency drops from ~10 seconds to ~1.5 seconds for the embed step. Batching is shown in Appendix C.

6.5.4 Thread 2 — the knowledge graph pipeline

```

1 from agents.srv.kg_srv import extract_triples_from_text, insert_triples
2
3 def _kg_pipeline(text: str, material_number: str) -> None:
4     conn = get_hana_connection()
5     triples = extract_triples_from_text(text, material_number)

```

```

6
7     if not triples:
8         raise ValueError(
9             f"Zero triples extracted for '{material_number}'. "
10            "Check document quality and ontology alignment."
11        )
12
13    insert_triples(conn=conn, material_number=material_number,
14                  triples=triples)
15    close_thread_connection()

```

`extract_triples_from_text()` is the Gemini extraction function built in Chapter 5. It sends the full text to `gemini-1.5-pro` with a structured prompt that includes the ontology predicates and asks for JSON output. `insert_triples()` builds a SPARQL INSERT DATA query and calls `SPARQL_EXECUTE` on the HANA connection.

The “zero triples” check is deliberate. A successful Gemini call that returns no triples is worse than a clean failure — the document would appear as `kgStatus = 'DONE'` to the user but produce no graph query results. Raising `ValueError` here sets `kgStatus = 'ERROR'`, which makes the problem visible.

6.6 Status tracking

The user interface needs to know when ingestion is complete. The CAP frontend polls `GET /status/{materialNumber}` every 3 seconds until both pipelines report a terminal state (DONE or ERROR).

6.6.1 The status fields

The `MSDS_DOCUMENTS` table has two status columns:

Column	Tracks	Values
<code>status</code>	KG pipeline	PROCESSING, DONE, ERROR
<code>vectorStatus</code>	Vector pipeline	PROCESSING, DONE, ERROR

Both are populated before the upload returns (PROCESSING), updated by the background threads on completion, and read by the status endpoint. Using two separate columns

rather than one combined status allows the UI to show partial progress — “Vector: DONE, KG: processing...” — and makes it easier to retry only the failed pipeline.

6.6.2 Status helper functions

```

1  def _mark_processing(material_number: str) -> None:
2      conn = get_hana_connection()
3      cursor = conn.cursor()
4      cursor.execute("""
5          UPSERT MSDS_DOCUMENTS (MATERIAL_NUMBER, STATUS, VECTOR_STATUS,
6          CREATED_AT)
7          VALUES (?, 'PROCESSING', 'PROCESSING', NOW())
8          WITH PRIMARY KEY
9          """, (material_number,))
10     conn.commit()
11     cursor.close()
12     close_thread_connection()
13
14 def _mark_vector_status(material_number: str, status: str,
15     error_message=None) -> None:
16     conn = get_hana_connection()
17     cursor = conn.cursor()
18     cursor.execute("""
19         UPDATE MSDS_DOCUMENTS
20         SET VECTOR_STATUS = ?, VECTOR_ERROR = ?, UPDATED_AT = NOW()
21         WHERE MATERIAL_NUMBER = ?
22         """, (status, error_message, material_number))
23     conn.commit()
24     cursor.close()
25     close_thread_connection()
26
27 def _mark_kg_status(material_number: str, status: str, error_message=None)
28     -> None:
29     conn = get_hana_connection()
30     cursor = conn.cursor()
31     cursor.execute("""
32         UPDATE MSDS_DOCUMENTS
33         SET STATUS = ?, KG_ERROR = ?, UPDATED_AT = NOW()
34         WHERE MATERIAL_NUMBER = ?

```

```

32     """", (status, error_message, material_number))
33     conn.commit()
34     cursor.close()
35     close_thread_connection()

```

6.6.3 The status endpoint

```

1  @app.get("/status/{material_number}")
2  def get_status(material_number: str):
3      if not re.match(r'^[A-Za-z0-9_-]+$ ', material_number):
4          return JsonResponse(status_code=400, content={"error": "Invalid
5              materialNumber"})
6
7      conn = get_hana_connection()
8      cursor = conn.cursor()
9      cursor.execute("""
10          SELECT STATUS, VECTOR_STATUS, KG_ERROR, VECTOR_ERROR, UPDATED_AT
11          FROM    MSDS_DOCUMENTS
12          WHERE   MATERIAL_NUMBER = ?
13      """, (material_number,))
14      row = cursor.fetchone()
15      cursor.close()
16      close_thread_connection()
17
18      if not row:
19          return JsonResponse(status_code=404, content={"error": "Material
20              not found"})
21
22      kg_status, vector_status, kg_error, vector_error, updated_at = row
23      return {
24          "materialNumber": material_number,
25          "kgStatus":      kg_status,
26          "vectorStatus":  vector_status,
27          "kgError":       kg_error,
28          "vectorError":   vector_error,
29          "updatedAt":     str(updated_at),
30          "complete":      kg_status in ("DONE", "ERROR") and
31                          vector_status in ("DONE", "ERROR"),
32      }

```

The `complete` boolean is the sentinel the CAP frontend uses to stop polling. If `complete` is `true` but either `status` is `ERROR`, the frontend shows an error banner but does not block navigation — the user can still query whatever pipeline succeeded.

6.7 Error handling — what happens when one pipeline fails

The most important design property of this ingestion service is **error isolation**: a failure in one pipeline does not affect the other.

Failure	What happens	Status in HANA
PDF has no text layer	Both pipelines fail before threads start	Both <code>ERROR</code> , same error message
Vertex AI embedding API down	<code>_vector_pipeline</code> raises; <code>_kg_pipeline</code> continues	<code>vectorStatus = ERROR</code> , <code>kgStatus = DONE</code>
Gemini triple extraction returns empty	<code>_kg_pipeline</code> raises; <code>_vector_pipeline</code> has already completed	<code>vectorStatus = DONE</code> , <code>kgStatus = ERROR</code>
HANA connection fails in one thread	Affected thread raises; other thread uses its own connection	Independent <code>ERROR</code> / <code>DONE</code>

Scenario 2 deserves attention because it reflects a real operational condition. When a Vertex AI service outage occurs, every embedding call will fail. If error propagation were allowed, the `ThreadPoolExecutor` would cancel the KG future when the vector future raises — and the document would have no graph knowledge despite the Gemini extraction succeeding. The `try/except` wrapper in `as_completed()` prevents that cancellation.

Note: Persisting error messages in the `KG_ERROR` and `VECTOR_ERROR` columns means operators can run `SELECT MATERIAL_NUMBER, KG_ERROR FROM MSDS_DOCUMENTS WHERE STATUS = 'ERROR'` to get a triage list — without reading log files.

6.8 Testing — upload five MSDS documents

6.8.1 Start the service

```
1 cd agents
2 source .venv/bin/activate
3 uvicorn main:app --reload --port 8000
```

6.8.2 Upload the first document

```
1 curl -s -X POST http://localhost:8000/process-upload \
2     -F "file=@/path/to/acetone-sds.pdf" \
3     -F "materialNumber=MAT-001" \
4     -F "materialName=Acetone"
```

Expected response (within 200 ms):

```
1 {"status": "processing", "materialNumber": "MAT-001"}
```

6.8.3 Poll for completion

```
1 watch -n3 'curl -s http://localhost:8000/status/MAT-001 | python3 -m
  json.tool'
```

After 8–15 seconds:

```
1 {
2     "materialNumber": "MAT-001",
3     "kgStatus": "DONE",
4     "vectorStatus": "DONE",
5     "kgError": null,
6     "vectorError": null,
7     "updatedAt": "2026-05-26 09:31:17.447",
8     "complete": true
9 }
```

6.8.4 Verify the vector store

```

1 SELECT
2     CHUNK_INDEX,
3     LEFT(CHUNK_TEXT, 80) AS CHUNK_PREVIEW,
4     CARDINALITY(EMBEDDING) AS VECTOR_DIM
5 FROM MSDS_VECTORS
6 WHERE MATERIAL_NUMBER = 'MAT-001'
7 ORDER BY CHUNK_INDEX;

```

Expected: every row has VECTOR_DIM = 768.

6.8.5 Verify the knowledge graph

```

1 from hdbcli import dbapi
2 import os
3
4 conn = dbapi.connect(
5     address=os.environ["HANA_HOST"],
6     port=int(os.environ["HANA_PORT"]),
7     user=os.environ["HANA_USER"],
8     password=os.environ["HANA_PASSWORD"],
9 )
10 cursor = conn.cursor()
11
12 sparql = """
13 PREFIX msds: <http://msds.knowledge-graph.org/ontology/>
14 SELECT ?predicate ?object
15 WHERE {
16     GRAPH <http://msds.knowledge-graph.org/MSDS_Graph/MAT-001> {
17         <http://msds.knowledge-graph.org/material/MAT-001> ?predicate ?object
18     }
19 }
20 """
21
22 cursor.callproc("SPARQL_EXECUTE", [sparql, None, 1000, None, None])
23 rows = cursor.fetchall()
24 for row in rows:
25     print(row)

```

Expected output:

```
( 'http://msds.knowledge-graph.org/ontology#hasHazardCode',      'H225' )
( 'http://msds.knowledge-graph.org/ontology#hasHazardCode',      'H319' )
( 'http://msds.knowledge-graph.org/ontology#hasHazardCode',      'H336' )
( 'http://msds.knowledge-graph.org/ontology#hasExposureLimit',    '500 ppm TWA' )
( 'http://msds.knowledge-graph.org/ontology#requiresPrecaution', 'Keep away from open flame' )
( 'http://msds.knowledge-graph.org/ontology#hasSupplier',         'Sigma-Aldrich' )
```

Warning: If rows is empty, first check the GRAPH clause. HANA SPARQL returns no results without the GRAPH wrapper, even if the triples exist. The second thing to check is the IRI prefix — the subject must match exactly what `insert_triples()` used when storing the graph.

Upload the remaining four PDFs (MAT-002 through MAT-005) with the same pattern. After all five complete, you will have 250–350 rows in `MSDS_VECTORS` and 5 named graphs in the HANA RDF store.

6.9 Summary

This chapter built the pipeline that bridges PDF uploads and the dual-store knowledge layer. The key design decisions:

- **Parallel threads** — two independent pipelines in a `ThreadPoolExecutor` reduce ingestion time to the slower pipeline’s duration, not the sum of both.
- **Fire-and-forget with status polling** — the upload endpoint returns in milliseconds; the CAP frontend polls every 3 seconds. Decouples ingestion time from HTTP timeouts.
- **Thread-local HANA connections** — `threading.local()` gives each worker thread its own connection object, eliminating race conditions without connection pooling infrastructure.
- **Different text strategies per pipeline** — the vector pipeline chunks to 500 tokens; the KG pipeline sends the full document. Chunking optimises embedding precision; full text optimises triple extraction accuracy.
- **Error isolation** — exceptions in one pipeline are caught and persisted as `ERROR`

status without interrupting the other pipeline.

- **Dual status fields** — STATUS (KG) and VECTOR_STATUS independently track each pipeline. A partial success is visible, queryable, and retryable.
-

6.10 Checkpoint

Before moving to Chapter 7, confirm each of the following:

- ☐ `agents/srv/doc_srv.py` exists and `uvicorn main:app --port 8000` starts without import errors.
- ☐ `POST /process-upload` with a real MSDS PDF returns `{"status": "processing"}` within 500 ms.
- ☐ `GET /status/{materialNumber}` returns `"complete": true` within 30 seconds for a 20–40 page document.
- ☐ Five MSDS documents uploaded; all show `kgStatus = "DONE"` and `vectorStatus = "DONE"`.
- ☐ `SELECT COUNT(*) FROM MSDS_VECTORS WHERE MATERIAL_NUMBER = 'MAT-001'` returns a positive number.
- ☐ `SELECT CARDINALITY(EMBEDDING) FROM MSDS_VECTORS FETCH FIRST 1 ROWS ONLY` returns 768.
- ☐ SPARQL query against `MSDS_Graph/MAT-001` returns at least one triple with predicate `hasHazardCode`.
- ☐ Re-uploading the same PDF overwrites existing rows rather than creating duplicates.
- ☐ Uploading a scan-only PDF results in both statuses showing `ERROR`, not `PROCESSING`.

With the ingestion pipeline verified and five documents loaded into both stores, you are ready for the agentic layer. Chapter 7 introduces LangGraph — the framework that will orchestrate queries against these two stores in parallel and synthesise a single coherent answer.

End of Chapter 6

Chapter 7: LangGraph Fundamentals

In Chapter 6 we built a document ingestion pipeline that processes PDFs into both a vector store and a knowledge graph simultaneously. We now have data — rich, structured, searchable data — in SAP HANA Cloud. The next question is: how do we build an agent that reasons over it?

A plain LangChain chain would work for a single retrieval followed by a single LLM call. But the system we need is more complex than that. It needs to run two retrieval strategies in parallel, decide how to merge their results, handle retries when SPARQL returns empty results, maintain conversation history across turns, and expose all of this over a single HTTP endpoint. A linear chain cannot express those decisions. We need something that can branch, loop, and conditionally route based on what it finds.

That something is LangGraph.

This chapter introduces LangGraph from first principles. We build a simple question-answering agent step by step — one node at a time — so that when Chapter 8 extends it into the full parallel hybrid RAG system, every piece is familiar. By the end of this chapter you will have a running LangGraph agent connected to Vertex AI, and you will understand exactly how state flows through it.

7.1 Why LangChain chains are not enough

LangChain introduced the concept of a chain: a sequence of components where the output of one feeds the input of the next. For many tasks — retrieve a passage, summarise it, return the result — a chain is exactly right. It is simple, readable, and easy to test.

The problem arises when your logic is not a sequence. Consider what our hybrid RAG system needs to do:

Requirement	Can a chain do it?
Run two retrieval strategies in parallel	No — chains are sequential
Retry SPARQL generation if results are empty	No — chains have no loops

Requirement	Can a chain do it?
Route to different merge strategies based on what was found	No — chains have no conditional branches
Pass conversation history into every step	Awkward — requires passing state manually
Stream partial results to the UI as they arrive	Limited

LangGraph solves every one of these by modelling your agent as a directed graph rather than a list. Nodes are Python functions. Edges are connections between them. Conditional edges let the graph branch based on the current state. The graph can loop. It can run nodes in parallel. It can stop and stream intermediate results.

Note: LangGraph is built on top of LangChain. You still use LangChain components — `ChatVertexAI`, tool decorators, message types — but LangGraph provides the execution engine that orchestrates them.

7.2 Core concepts

Before writing any code, here are the four concepts you need to hold in your head.

7.2.1 State

State is the memory of your agent for a single run. It is a Python `TypedDict` — a dictionary where every key has a declared type. Every node in the graph reads from state and writes back to state. State is the only way nodes communicate with each other.

```

1 from typing import TypedDict, List
2
3 class AgentState(TypedDict):
4     question: str
5     answer: str
6     messages: List[dict]
```

When you invoke the graph, you pass an initial state. When the graph finishes, you receive

the final state. Everything that happened in between is recorded there.

7.2.2 Nodes

A node is a Python function that receives the current state and returns a partial update to it. LangGraph merges the update into the state before passing it to the next node.

```
1 def my_node(state: AgentState) -> dict:
2     question = state["question"]
3     answer = call_llm(question)
4     # return only the keys you want to update
5     return {"answer": answer}
```

The function does not need to return the entire state — only the keys it wants to change. LangGraph merges the returned dict into the existing state automatically.

7.2.3 Edges

An edge connects two nodes. When node A finishes, LangGraph follows the edge to node B and runs it next.

```
1 graph.add_edge("node_a", "node_b")
```

7.2.4 Conditional edges

A conditional edge calls a routing function after a node completes. The routing function inspects the state and returns a string that names the next node to run.

```
1 def route(state: AgentState) -> str:
2     if state["answer"] == "":
3         return "retry"
4     return "done"
5
6 graph.add_conditional_edges("node_a", route, {
7     "retry": "retry_node",
8     "done": END
9 })
```

This is how loops and branches are expressed in LangGraph.

7.3 Installation and project layout

Install LangGraph and the Vertex AI integration:

```
1 pip install langgraph langchain-google-vertexai langchain-core
```

Add these to `agents/requirements.txt`:

```
langgraph>=0.2.0
```

```
langchain-google-vertexai>=2.0.0
```

```
langchain-core>=0.3.0
```

We will build the simple agent in a new file:

`agents/`

`agents/`

`simple_qa_agent.py` ← this chapter

`orchestrator.py` ← Chapter 8

`kg_chain.py` ← Chapter 8

`vector_chain.py` ← Chapter 8

7.4 Building the state and nodes

Open `agents/agents/simple_qa_agent.py`. We start with the state definition and two nodes.

```
1 from typing import TypedDict, List
2 from langgraph.graph import StateGraph, END
3 from langchain_google_vertexai import ChatVertexAI
4 from langchain_core.messages import HumanMessage, AIMessage
5 # — State
6
7 class AgentState(TypedDict):
8     question: str
9     context: str
```

```

9     answer: str
10    messages: List[dict]
11    retry_count: int
12    # — LLM

```

```

13    llm = ChatVertexAI(model_name="gemini-1.5-pro", temperature=0.1,
14                        max_tokens=1024)
15    # — Nodes

```

```

16    def retrieve_node(state: AgentState) -> dict:
17        """Stub retriever – replaced by vector + KG chains in Chapter 8."""
18        question = state["question"]
19        context = f"[Retrieved context for: {question}]"
20        return {"context": context}
21
22    def answer_node(state: AgentState) -> dict:
23        """Generate an answer using the retrieved context."""
24        question = state["question"]
25        context = state["context"]
26        history = state.get("messages", [])
27
28        messages = []
29        for msg in history:
30            if msg["role"] == "user":
31                messages.append(HumanMessage(content=msg["content"]))
32            else:
33                messages.append(AIMessage(content=msg["content"]))
34
35        prompt = f"""You are an expert assistant for Material Safety Data
36        Sheets.
37
38        Context from the knowledge base:
39        {context}
40
41        Answer the following question based on the context above.
42        If the context does not contain enough information, say so clearly.
43
44        Question: {question}"""

```

```

44     messages.append(HumanMessage(content=prompt))
45     response = llm.invoke(messages)
46     return {"answer": response.content}

```

Two things to notice. First, `retrieve_node` is a deliberate stub — it returns a placeholder string. We build the real retrieval in Chapter 8; this stub lets us test the graph structure now without needing a live HANA connection. Second, `answer_node` builds the conversation history from the messages list in state. This is how multi-turn conversations work: the caller passes previous turns in the initial state, and the node includes them in the prompt.

7.5 Wiring the graph

Add the graph construction below the node functions:

```

1  def build_graph():
2      graph = StateGraph(AgentState)
3
4      graph.add_node("retrieve", retrieve_node)
5      graph.add_node("answer", answer_node)
6
7      graph.set_entry_point("retrieve")
8      graph.add_edge("retrieve", "answer")
9      graph.add_edge("answer", END)
10
11     return graph.compile()
12
13 app = build_graph()

```

The graph is linear: `retrieve` → `answer` → `END`. To invoke it:

```

1  if __name__ == "__main__":
2      result = app.invoke({
3          "question": "What are the hazard codes for acetone?",
4          "context": "",
5          "answer": "",
6          "messages": [],

```

```

7         "retry_count": 0,
8     })
9     print(result["answer"])

```

Run it:

```

1 cd agents
2 python -m agents.simple_qa_agent

```

Expected output (with stub retriever):

Based on the provided context, I don't have specific hazard codes for acetone. The context contains a placeholder rather than actual MSDS data. Once the knowledge base is populated with real MSDS documents, I will be able to retrieve and report the specific GHS hazard codes for acetone.

The agent runs, reaches Vertex AI, and gives an honest answer about the stub context. That is the graph working correctly.

7.6 Adding a conditional retry edge

Real retrieval can fail — SPARQL returns empty results, an embedding call times out. We need to be able to retry. Add a check node and a routing function:

```

1 def check_answer_node(state: AgentState) -> dict:
2     """Check if the answer is acceptable. Increment retry counter if
3     not."""
4     answer = state.get("answer", "")
5     retry_count = state.get("retry_count", 0)
6
7     low_quality_phrases = [
8         "don't have", "no information", "cannot find",
9         "not available", "placeholder"
10    ]
11    needs_retry = any(p in answer.lower() for p in low_quality_phrases)
12    return {"retry_count": retry_count + (1 if needs_retry else 0)}
13
14 def route_after_check(state: AgentState) -> str:

```



```

14     """Decide whether to retry or finish."""
15     answer      = state.get("answer",      "")
16     retry_count = state.get("retry_count", 0)
17
18     low_quality_phrases = [
19         "don't have", "no information", "cannot find",
20         "not available", "placeholder"
21     ]
22     needs_retry = any(p in answer.lower() for p in low_quality_phrases)
23
24     if needs_retry and retry_count < 2:
25         return "retry"
26     return "done"

```

Update `build_graph` to include the check node and conditional edge:

```

1  def build_graph():
2      graph = StateGraph(AgentState)
3
4      graph.add_node("retrieve", retrieve_node)
5      graph.add_node("answer",   answer_node)
6      graph.add_node("check",    check_answer_node)
7
8      graph.set_entry_point("retrieve")
9      graph.add_edge("retrieve", "answer")
10     graph.add_edge("answer",    "check")
11
12     graph.add_conditional_edges(
13         "check",
14         route_after_check,
15         {
16             "retry": "retrieve",  # loop back
17             "done":  END
18         }
19     )
20
21     return graph.compile()

```

The graph now has a loop: `retrieve` → `answer` → `check` → (retry: back to `retrieve`) or (done: `END`). The `retry_count` guard prevents infinite loops — after two retries we accept

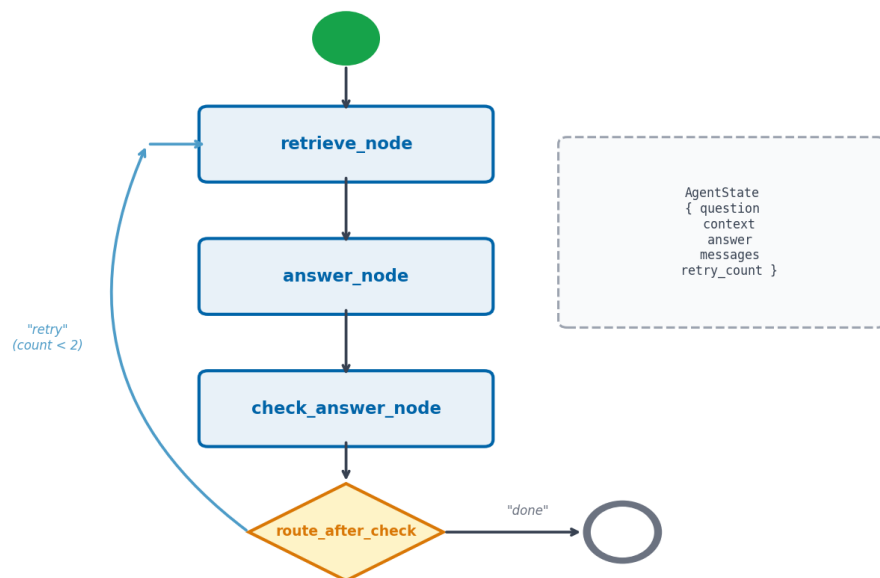
whatever answer we have.

Note: In Chapter 8 the retry logic is more specific: we only retry when SPARQL returns zero results, and the retry uses a simpler SPARQL query, not a full re-run of both chains. The pattern here is the same; the condition is different.

7.7 The StateGraph visualised

The graph we built in this chapter has three nodes and one conditional loop:

LangGraph StateGraph — Chapter 7 Simple Q&A Agent



State flows through every node. Nodes return only the keys they update.

Figure 7.1: The simple Q&A agent's StateGraph — retrieve feeds answer, which feeds the check node. The check node either loops back to retrieve (on low-quality answers, up to 2 retries) or terminates at END. The dashed box on the right shows the AgentState structure that flows through every node.

7.8 The complete simple_qa_agent.py

```

1  from typing import TypedDict, List
2  from langgraph.graph import StateGraph, END
3  from langchain_google_vertexai import ChatVertexAI
4  from langchain_core.messages import HumanMessage, AIMessage
5
6  class AgentState(TypedDict):
7      question: str
8      context: str
9      answer: str
10     messages: List[dict]
11     retry_count: int
12
13     llm = ChatVertexAI(model_name="gemini-1.5-pro", temperature=0.1,
14                        max_tokens=1024)
15
16     def retrieve_node(state: AgentState) -> dict:
17         question = state["question"]
18         return {"context": f"[Retrieved context for: {question}]"}
19
20     def answer_node(state: AgentState) -> dict:
21         messages = []
22         for msg in state.get("messages", []):
23             cls = HumanMessage if msg["role"] == "user" else AIMessage
24             messages.append(cls(content=msg["content"]))
25
26         prompt = f"""You are an expert assistant for Material Safety Data
27         Sheets.
28
29         Context:
30         {state['context']}
31
32         Question: {state['question']}
33
34         Answer: """
35         messages.append(HumanMessage(content=prompt))
36         return {"answer": llm.invoke(messages).content}
37
38     def check_answer_node(state: AgentState) -> dict:

```

```

36     low = ["don't have", "no information", "cannot find", "not available",
"placeholder"]
37     needs_retry = any(p in state.get("answer","").lower() for p in low)
38     return {"retry_count": state.get("retry_count", 0) + (1 if needs_retry
else 0)}
39
40 def route_after_check(state: AgentState) -> str:
41     low = ["don't have", "no information", "cannot find", "not available",
"placeholder"]
42     needs_retry = any(p in state.get("answer","").lower() for p in low)
43     return "retry" if (needs_retry and state.get("retry_count",0) < 2)
else "done"
44
45 def build_graph():
46     graph = StateGraph(AgentState)
47     graph.add_node("retrieve", retrieve_node)
48     graph.add_node("answer", answer_node)
49     graph.add_node("check", check_answer_node)
50     graph.set_entry_point("retrieve")
51     graph.add_edge("retrieve", "answer")
52     graph.add_edge("answer", "check")
53     graph.add_conditional_edges("check", route_after_check,
{"retry": "retrieve", "done": END})
54
55     return graph.compile()
56
57 app = build_graph()
58
59 if __name__ == "__main__":
60     result = app.invoke({
61         "question": "What are the hazard codes for acetone?",
62         "context": "",
63         "answer": "",
64         "messages": [],
65         "retry_count": 0,
66     })
67     print(result["answer"])

```

7.9 Tool use in LangGraph

LangGraph supports tools — Python functions that the LLM can choose to call. For the hybrid RAG agent, we will not use the LangGraph tool mechanism (we use direct parallel dispatch instead, as you will see in Chapter 8). But understanding tools is useful if you extend the agent later.

Define a tool with the `@tool` decorator:

```
1 from langchain_core.tools import tool
2
3 @tool
4 def get_hazard_codes(material_number: str) -> str:
5     """Retrieve GHS hazard codes for a material from the knowledge
6     graph."""
7     # In a real implementation this queries HANA SPARQL
8     return "H225, H319, H336"
```

Bind tools to the LLM and use `ToolNode` for execution:

```
1 from langgraph.prebuilt import ToolNode
2
3 tools = [get_hazard_codes]
4 llm_with_tools = llm.bind_tools(tools)
5 tool_node = ToolNode(tools)
6
7 graph.add_node("tools", tool_node)
8 graph.add_conditional_edges(
9     "answer",
10    lambda state: "tools" if state["messages"][-1].tool_calls else END,
11    {"tools": "tools", END: END}
12 )
13 graph.add_edge("tools", "answer")
```

This creates a ReAct loop: the LLM decides whether to call a tool, the tool runs, and the result feeds back into the next LLM call.

Note: Chapter 8 intentionally avoids this ReAct pattern for the main retrieval. We run both vector and KG chains on every query, regardless of what the LLM might choose. This gives deterministic latency and prevents the LLM

from skipping a retrieval path it thinks is unnecessary.

7.10 Streaming responses

LangGraph can stream state updates as they happen. This is valuable when queries take several seconds — the user sees progress rather than a blank screen.

```
1 for event in app.stream({
2     "question":    "What first aid should I give if someone inhales
3     acetone?",
4     "context":     "",
5     "answer":      "",
6     "messages":    [],
7     "retry_count": 0,
8 }):
9     for node_name, state_update in event.items():
10        print(f"[{node_name}] completed")
11        if "answer" in state_update:
12            print(f"    answer: {state_update['answer'][:80]}...")
```

Expected output:

[retrieve] completed

[answer] completed

answer: If someone inhales acetone vapours, move them immediately to fresh air...

[check] completed

Tip: In the FastAPI service, pipe the stream to a Server-Sent Events (SSE) response using `StreamingResponse`. The Fiori UI can consume the stream and update the chat interface progressively. We implement this in Chapter 10.

7.11 Debugging with LangSmith

LangSmith is LangChain's observability platform. It records every LangGraph run — inputs, outputs, intermediate states, LLM calls, latency, and token counts. It is free for

individual developers.

Set three environment variables:

```
1 export LANGCHAIN_TRACING_V2=true
2 export LANGCHAIN_ENDPOINT=https://api.smith.langchain.com
3 export LANGCHAIN_API_KEY=your-api-key-here
```

Get a free API key at smith.langchain.com. Once set, every `app.invoke()` and `app.stream()` call is automatically traced.

In your BTP CF `manifest.yml`, add the same variables:

```
1 applications:
2   - name: hybrid-rag-agent
3     env:
4       LANGCHAIN_TRACING_V2: "true"
5       LANGCHAIN_ENDPOINT: "https://api.smith.langchain.com"
6       LANGCHAIN_API_KEY: ((langchain-api-key))
```

Use a BTP User-Provided Service to supply the API key without storing it in the manifest. In the LangSmith dashboard you will see a tree view of each run: which nodes executed, in what order, how long each took, and the exact prompt sent to Gemini. When Chapter 8's parallel chains run, you will see two branches executing simultaneously.

Tip: Tag your runs in `app.invoke(config={"metadata": {"material_number": material_number}})` and filter by `metadata.material_number` in LangSmith to see all queries against a specific document.

7.12 Why stateless works on BTP Cloud Foundry

A common concern when deploying LangGraph to Cloud Foundry is: what happens to agent state when you scale to multiple instances?

The answer is: nothing, because there is no server-side state to worry about.

LangGraph state is in-memory for a single request. When `app.invoke()` returns, the state is gone. There is no database, no session file, no sticky session required.

Concern	Reality
CF runs 10 instances	Each request lands on any instance — fine
CF restarts an instance	No state is lost — state only exists during a request
Two users query simultaneously	Each gets their own independent in-memory state
Conversation history	Passed in <code>messages</code> field of every request body by the caller

The conversation history pattern — where the client sends previous turns in every request — is the key. The frontend stores the last N messages and sends them with each new question.

```
1  # Frontend sends history on every request
2  # POST /query
3  {
4    "question": "And what about the flash point?",
5    "material_number": "ACE001",
6    "history": [
7      {"role": "user", "content": "What are the hazard codes for acetone?"},
8      {"role": "assistant", "content": "The GHS hazard codes for acetone are H225, H319, and H336."}
9    ]
10 }
```

The agent picks up the conversation exactly where it left off — without any server-side memory. This design is safe to scale horizontally on BTP CF without load balancer sticky sessions or shared caches.

7.13 Summary

In this chapter we built a LangGraph agent from scratch:

- Defined **state** as a TypedDict that flows through every node
- Wrote **nodes** as plain Python functions that read state and return partial updates
- Connected nodes with **edges** and **conditional edges** to express branching and looping
- Added a **retry loop** that re-runs retrieval when the answer quality is low
- Demonstrated **tool use** with the @tool decorator and ToolNode
- Enabled **streaming** to show node-level progress events
- Configured **LangSmith** for end-to-end observability
- Explained why **stateless design** makes LangGraph safe to scale horizontally on BTP CF

The agent built here uses a stub retriever. In Chapter 8, we replace that stub with the two HANA retrieval chains from Chapters 4 and 5 — and we run them in parallel.

7.14 Checkpoint

Before continuing to Chapter 8, verify the following:

```
1  # 1. LangGraph is installed
2  python -c "import langgraph; print(langgraph.__version__)"
3
4  # 2. The simple agent runs without errors
5  cd agents && python -m agents.simple_qa_agent
6
7  # 3. Vertex AI credentials are set
8  echo $GOOGLE_APPLICATION_CREDENTIALS # should print a path to your JSON
   key
9
10 # 4. LangSmith (optional but recommended)
11 echo $LANGCHAIN_TRACING_V2           # should print "true"
```

If all four commands succeed, you have a working LangGraph agent connected to Vertex AI. Chapter 8 takes this foundation and builds the full parallel hybrid RAG system on top of it.

End of Chapter 7

Chapter 8: The Parallel Hybrid RAG Agent

In Chapter 7 we built a LangGraph agent with a stub retriever — a placeholder that returned a fixed string instead of real data. In Chapters 4 and 5 we built the real retrieval systems: a vector store in SAP HANA Cloud and a knowledge graph queried via SPARQL. In Chapter 6 we built the ingestion pipeline that populates both. All the pieces exist. This chapter assembles them.

The central design question is: when a user asks a question, do we run vector search first and knowledge graph search second? Or do we ask the LLM to decide which one to use? The answer to both is no. We run them in parallel, always, regardless of the question. This chapter explains why that decision is correct and shows you exactly how to implement it.

By the end of this chapter you will have a working hybrid RAG system: a FastAPI service that receives a natural language question, dispatches it simultaneously to both retrieval chains, merges the results, and returns an answer that demonstrably outperforms either strategy alone.

8.1 Why parallel beats sequential

Consider the naive approach: run vector search, check the result, then decide whether to also run KG search. This is sequential routing, and it has a hidden cost.

Approach	Wall-clock time	LLM calls	Risk
Sequential: vector then KG	$2\text{ s} + 2\text{ s} = 4\text{ s}$	3 (route + vector + KG)	Routing LLM makes wrong choice
Sequential: KG then vector	$2\text{ s} + 2\text{ s} = 4\text{ s}$	3	Same risk
Routing: LLM decides which to use	$1\text{ s} + 2\text{ s} = 3\text{ s}$	2	Routing LLM skips useful path
Parallel: both simultaneously	$\max(2\text{ s}, 2\text{ s}) = 2\text{ s}$	2	None

The parallel approach is faster than all sequential variants and uses fewer LLM calls than the routing variant. More importantly, it eliminates the routing error entirely. A routing LLM might decide that “what are the GHS codes for acetone?” is a structured query and skip the vector chain — missing the handling instructions that live only in prose. By always running both, we guarantee that no retrieval path is skipped.

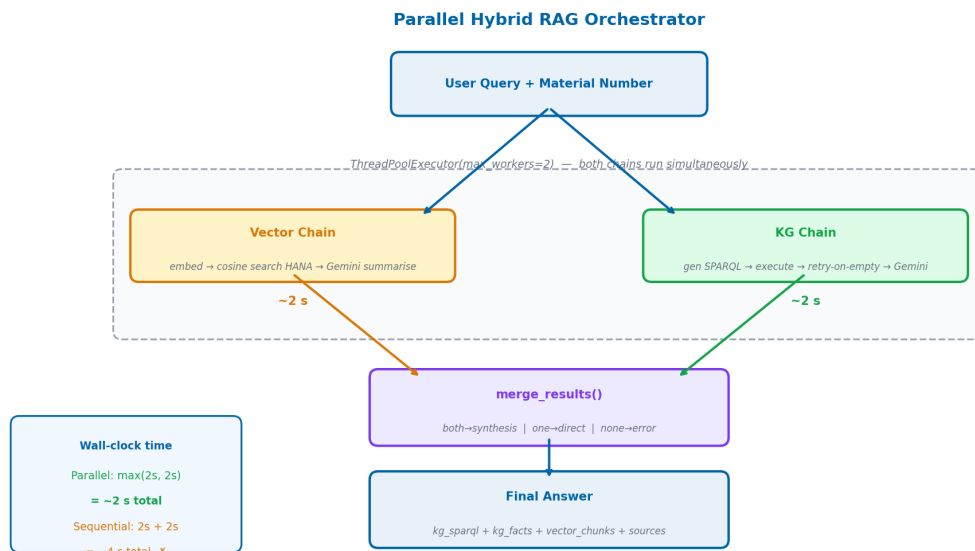


Figure: The parallel hybrid RAG orchestrator — both chains run concurrently via `ThreadPoolExecutor`. Wall-clock latency equals the slower of the two chains, not their sum.

Note: This design assumes both chains have roughly similar latency (~1–3 seconds each). If one chain were orders of magnitude slower than the other, you might reconsider. In practice, a HANA SPARQL query and a HANA vector search take comparable time.

8.2 The agent state

Create `agents/agents/state.py`:

```
1 from typing import TypedDict, List, Optional
2
3 class HybridRAGState(TypedDict):
```

```

4      # Input
5      question: str
6      material_number: str
7      history: List[dict]
8
9      # Vector chain outputs
10     vector_answer: str
11     vector_chunks: List[dict]
12
13     # KG chain outputs
14     kg_answer: str
15     kg_sparql: str
16     kg_facts: List[dict]
17
18     # Final output
19     final_answer: str
20     sources: List[str]
21
22     # Control
23     error: Optional[str]

```

This state is the contract between every component in the system. The vector chain writes to `vector_answer` and `vector_chunks`. The KG chain writes to `kg_answer`, `kg_sparql`, and `kg_facts`. The merge function reads all four and writes `final_answer`.

8.3 The vector chain

Create `agents/agents/vector_chain.py`:

```

1  import logging
2  from typing import Optional
3  from langchain_google_vertexai import ChatVertexAI, VertexAIEmbeddings
4  from langchain_core.messages import HumanMessage
5
6  from agents.state import HybridRAGState
7  from srv.hdb_srv import get_connection
8

```

```

9  logger = logging.getLogger(__name__)
10
11  _llm = ChatVertexAI(model_name="gemini-1.5-pro", temperature=0.1,
12                      max_tokens=1024)
13
14  _embedder = VertexAIEmbeddings(model_name="text-embedding-004")
15
16  COSINE_SEARCH_SQL = """
17  SELECT TOP 5
18      CHUNK_TEXT,
19      MATERIAL_NUMBER,
20      CHUNK_INDEX,
21      TO_DOUBLE(COSINE_SIMILARITY(EMBEDDING, TO_REAL_VECTOR(?))) AS SCORE
22  FROM MSDS_VECTORS
23  WHERE MATERIAL_NUMBER = ?
24  ORDER BY SCORE DESC
25  """
26
27  def run_vector_chain(state: HybridRAGState) -> dict:
28      """Execute the vector retrieval chain and return state updates."""
29      question = state["question"]
30      material_number = state["material_number"]
31
32      try:
33          # Step 1: Embed the question
34          embedding = _embedder.embed_query(question)
35          embedding_str = "[" + ",".join(str(v) for v in embedding) + "]"
36
37          # Step 2: Cosine search in HANA
38          conn = get_connection()
39          cursor = conn.cursor()
40          cursor.execute(COSINE_SEARCH_SQL, (embedding_str,
41                                          material_number))
42          rows = cursor.fetchall()
43
44          if not rows:
45              logger.info("Vector search returned no results for %s",
46                          material_number)
47
48          return {
49              "vector_answer": "",

```

```

45         "vector_chunks": [],
46     }
47
48     chunks = [
49         {
50             "text": row[0],
51             "material_number": row[1],
52             "chunk_index": row[2],
53             "score": float(row[3]),
54         }
55         for row in rows
56     ]
57
58     # Step 3: Summarise with Gemini
59     context = "\n\n---\n\n".join(c["text"] for c in chunks)
60     prompt = f"""You are an expert in material safety. Use the
following passages
61 from an MSDS document to answer the question. Be specific and concise.
62 If the passages do not contain enough information, say so.
63
64 Passages:
65 {context}
66
67 Question: {question}
68
69 Answer: ""
70
71     response = _llm.invoke([HumanMessage(content=prompt)])
72     return {
73         "vector_answer": response.content,
74         "vector_chunks": chunks,
75     }
76
77 except Exception as e:
78     logger.error("Vector chain failed: %s", e)
79     return {
80         "vector_answer": "",
81         "vector_chunks": [],
82         "error": f"Vector chain error: {str(e)}",

```

83

}

Three steps: embed the question into a 768-dimensional vector, run a cosine similarity search against MSDS_VECTORS, summarise the top-5 matching chunks with Gemini. The SQL uses HANA's COSINE_SIMILARITY function and TO_REAL_VECTOR to cast the embedding string to the correct column type.

8.4 The KG chain

Create agents/agents/kg_chain.py:

```

1  import logging
2  from langchain_google_vertexai import ChatVertexAI
3  from langchain_core.messages import HumanMessage
4
5  from agents.state import HybridRAGState
6  from srv.hdb_srv import get_connection
7
8  logger = logging.getLogger(__name__)
9
10 _llm = ChatVertexAI(model_name="gemini-1.5-pro", temperature=0.0,
11                     max_tokens=512)
12
13 _summariser = ChatVertexAI(model_name="gemini-1.5-pro", temperature=0.1,
14                             max_tokens=1024)
15
16 GRAPH_URI_PREFIX = "http://msds.knowledge-graph.org/MSDS_Graph/"
17
18 SPARQL_GEN_PROMPT = """You are a SPARQL expert. Generate a SPARQL SELECT
19 query to answer
20 the question below using the provided ontology.
21
22 Ontology predicates available:
23 - msds:hasHazardCode (links Material to HazardCode)
24 - msds:hasExposureLimit (links Material to ExposureLimit, has
25   msds:limitValue and msds:limitUnit)
26 - msds:requiresPrecaution (links Material to Precaution, has
27   msds:precautionText)

```

```

22 - msds:hasSupplier (links Material to Supplier)
23 - msds:hazardDescription (literal on HazardCode)
24 - rdfs:label (name of the material)
25
26 Named graph: <{graph_uri}>
27 Material URI: <http://msds.knowledge-graph.org/material/{material_number}>
28
29 Rules:
30 1. Always wrap triple patterns with GRAPH <{graph_uri}> {{ ... }}
31 2. Use PREFIX msds: <http://msds.knowledge-graph.org/ontology/>
32 3. Use PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
33 4. Return only the SPARQL query, no explanation
34
35 Question: {question}
36
37 SPARQL:"""
38
39 SPARQL_FALLBACK_PROMPT = """The previous SPARQL query returned no results.
40 Generate a simpler,
41 broader SPARQL query to retrieve any available information about this
42 material.
43
44 Named graph: <{graph_uri}>
45 Material URI: <http://msds.knowledge-graph.org/material/{material_number}>
46
47 Return all triples about this material:
48
49 SPARQL:"""
50
51 def _execute_sparql(sparql: str) -> list:
52     conn = get_connection()
53     cursor = conn.cursor()
54     try:
55         cursor.callproc("SPARQL_EXECUTE", [sparql, None, 1000, None, None])
56         return cursor.fetchall()
57     finally:
58         cursor.close()
59
60 def run_kg_chain(state: HybridRAGState) -> dict:

```



```

59     """Execute the KG retrieval chain and return state updates."""
60     question = state["question"]
61     material_number = state["material_number"]
62     graph_uri = f"{GRAPH_URI_PREFIX}{material_number}"
63
64     try:
65         # Step 1: Generate SPARQL
66         gen_prompt = SPARQL_GEN_PROMPT.format(
67             graph_uri=graph_uri,
68             material_number=material_number,
69             question=question,
70         )
71         sparql_response = _llm.invoke([HumanMessage(content=gen_prompt)])
72         sparql = sparql_response.content.strip()
73         # Strip markdown code fences if present
74         if sparql.startswith("```"):
75             sparql = "\n".join(sparql.split("\n")[1:-1])
76
77         # Step 2: Execute SPARQL
78         rows = _execute_sparql(sparql)
79
80         # Step 3: Retry with fallback if empty
81         if not rows:
82             logger.info("SPARQL returned empty, retrying with fallback for
83 %s", material_number)
84             fallback_prompt = SPARQL_FALLBACK_PROMPT.format(
85                 graph_uri=graph_uri,
86                 material_number=material_number,
87             )
88             fallback_response =
89 _llm.invoke([HumanMessage(content=fallback_prompt)])
90             sparql = fallback_response.content.strip()
91             if sparql.startswith("```"):
92                 sparql = "\n".join(sparql.split("\n")[1:-1])
93             rows = _execute_sparql(sparql)
94
95         if not rows:
96             return {
97                 "kg_answer": "",

```

```

96         "kg_sparql": sparql,
97         "kg_facts": [],
98     }
99
100     # Step 4: Summarise with Gemini
101     facts = [str(row) for row in rows]
102     facts_text = "\n".join(facts[:50]) # limit to 50 facts
103
104     summarise_prompt = f"""You are an expert in material safety. The
following facts were
105 retrieved from a structured knowledge graph about material
{material_number}.
106 Use them to answer the question precisely.
107
108 Facts:
109 {facts_text}
110
111 Question: {question}
112
113 Answer: """
114
115     summary_response =
_summariser.invoke([HumanMessage(content=summarise_prompt)])
116     return {
117         "kg_answer": summary_response.content,
118         "kg_sparql": sparql,
119         "kg_facts": facts[:50],
120     }
121
122     except Exception as e:
123         logger.error("KG chain failed: %s", e)
124         return {
125             "kg_answer": "",
126             "kg_sparql": "",
127             "kg_facts": [],
128             "error": f"KG chain error: {str(e)}",
129         }

```

The KG chain has one critical feature beyond basic SPARQL execution: the retry-on-empty loop. When Gemini generates a SPARQL query that returns no rows — which

happens when the generated query is overly specific — the chain retries with a simpler fallback query that retrieves all triples for the material. This ensures we always get *something* from the knowledge graph if data exists.

Warning: Never share HANA connections across threads. The `get_connection()` call in both chains uses `threading.local()` under the hood — each thread gets its own connection. If you pass a connection object between threads, HANA will raise concurrency errors. See `agents/srv/hdb_srv.py` for the thread-local implementation.

8.5 The merge function

The merge function is the heart of hybrid RAG. It receives whatever the two chains produced and decides how to combine them.

```

1 def merge_results(
2     kg_answer: str,
3     vector_answer: str,
4     question: str,
5     material_number: str,
6     llm: ChatVertexAI,
7 ) -> str:
8     """
9     Merge KG and vector answers into a final response.
10
11     Cases:
12         both      → synthesis LLM call
13         kg only   → return kg_answer directly
14         vector    → return vector_answer directly
15         neither   → graceful error message
16     """
17     has_kg = bool(kg_answer and kg_answer.strip())
18     has_vec = bool(vector_answer and vector_answer.strip())
19
20     if has_kg and has_vec:
21         synthesis_prompt = f"""You are an expert in material safety.
```

```

22 You have received answers from two independent retrieval systems for the
    question below.
23
24 Structured Knowledge Graph answer:
25 {kg_answer}
26
27 Document Search answer:
28 {vector_answer}
29
30 Synthesise both into a single, coherent, well-organised answer.
31 - Do not repeat information
32 - If they contradict, prefer the structured KG answer for specific facts
    (codes, limits)
33 - If they complement, combine them naturally
34 - Be concise
35
36 Question: {question}
37
38 Answer: ""
39     response = llm.invoke([HumanMessage(content=synthesis_prompt)])
40     return response.content
41
42     elif has_kg:
43         return kg_answer
44
45     elif has_vec:
46         return vector_answer
47
48     else:
49         return (
50             f"I could not find specific information about your question in
              the available "
51             f"MSDS documents for material {material_number}. Please verify
              the material "
52             f"number or rephrase your question."
53         )

```

Four cases, handled explicitly. When both chains return results, a third Gemini call synthesises them — preferring the KG answer for precise facts and the vector answer for narrative context. When only one chain returns results, we return it directly without the

overhead of a synthesis call. When neither chain returns results, we return a clear, actionable error message rather than hallucinating.

8.6 The orchestrator

Create `agents/agents/orchestrator.py`:

```

1  import logging
2  from concurrent.futures import ThreadPoolExecutor, as_completed,
   TimeoutError
3  from langchain_google_vertexai import ChatVertexAI
4  from langchain_core.messages import HumanMessage
5
6  from agents.state import HybridRAGState
7  from agents.vector_chain import run_vector_chain
8  from agents.kg_chain import run_kg_chain
9
10 logger = logging.getLogger(__name__)
11
12 _llm = ChatVertexAI(model_name="gemini-1.5-pro", temperature=0.1,
   max_tokens=1024)
13
14 CHAIN_TIMEOUT_SECONDS = 30
15
16 def merge_results(kg_answer: str, vector_answer: str,
17                  question: str, material_number: str) -> str:
18     has_kg = bool(kg_answer and kg_answer.strip())
19     has_vec = bool(vector_answer and vector_answer.strip())
20
21     if has_kg and has_vec:
22         prompt = f"""You are an expert in material safety.
23 Two retrieval systems answered the same question. Synthesise their
24 answers.
25 Knowledge Graph:
26 {kg_answer}
27
```

```

28 Document Search:
29 {vector_answer}
30
31 Question: {question}
32
33 Synthesise into one coherent answer. Prefer KG for exact codes/limits.
34 Answer: ""
35     return _llm.invoke([HumanMessage(content=prompt)]).content
36
37     return kg_answer or vector_answer or (
38         f"No information found for material {material_number}. "
39         "Please verify the material number or rephrase your question."
40     )
41
42 def run_hybrid_rag(state: HybridRAGState) -> dict:
43     """
44     Dispatch both chains in parallel, merge results.
45     Returns a dict of state updates.
46     """
47     chains = {
48         "vector": run_vector_chain,
49         "kg": run_kg_chain,
50     }
51     results = {"vector": {}, "kg": {}}
52
53     with ThreadPoolExecutor(max_workers=2) as executor:
54         futures = {
55             executor.submit(fn, state): name
56             for name, fn in chains.items()
57         }
58         for future in as_completed(futures, timeout=CHAIN_TIMEOUT_SECONDS
59             + 5):
60             name = futures[future]
61             try:
62                 results[name] =
63                 future.result(timeout=CHAIN_TIMEOUT_SECONDS)
64             except TimeoutError:
65                 logger.warning("%s chain timed out after %ds", name,
66                     CHAIN_TIMEOUT_SECONDS)

```

```

64         results[name] = {}
65     except Exception as e:
66         logger.error("%s chain raised exception: %s", name, e)
67         results[name] = {}
68
69     # Merge state updates from both chains
70     merged_state = {}
71     for chain_result in results.values():
72         merged_state.update(chain_result)
73
74     # Generate final answer
75     final_answer = merge_results(
76         kg_answer=merged_state.get("kg_answer", ""),
77         vector_answer=merged_state.get("vector_answer", ""),
78         question=state["question"],
79         material_number=state["material_number"],
80     )
81     merged_state["final_answer"] = final_answer
82
83     # Build sources list
84     sources = []
85     if merged_state.get("vector_chunks"):
86         sources.append(f"Document search:
87 {len(merged_state['vector_chunks'])} passages")
88     if merged_state.get("kg_facts"):
89         sources.append(f"Knowledge graph: {len(merged_state['kg_facts'])}
90 facts")
91     merged_state["sources"] = sources
92
93     return merged_state

```

The orchestrator submits both chains to a `ThreadPoolExecutor` with `max_workers=2` and collects results as they complete. The `as_completed()` call returns futures in completion order — whichever chain finishes first is processed first. The 30-second timeout prevents a slow chain from blocking the response indefinitely.

Note: We use `as_completed()` rather than `executor.map()` because we want to process results as soon as they arrive, not wait for all to finish. If the vector chain finishes in 1.5 seconds and the KG chain takes 3 seconds, the orchestrator captures the vector result at 1.5 seconds and the KG result

at 3 seconds, then merges at ~3 seconds total.

8.7 The FastAPI /query endpoint

Add the endpoint to `agents/main.py`:

```
1  from fastapi import FastAPI, HTTPException
2  from pydantic import BaseModel, validator
3  from typing import List, Optional
4  import re
5
6  from agents.orchestrator import run_hybrid_rag
7  from agents.state import HybridRAGState
8
9  app = FastAPI(title="Hybrid RAG Agent")
10
11  MATERIAL_RE = re.compile(r"^[A-Za-z0-9_-]+$")
12
13  class QueryRequest(BaseModel):
14      question: str
15      material_number: str
16      history: List[dict] = []
17
18      @validator("material_number")
19      def validate_material(cls, v):
20          if not MATERIAL_RE.match(v):
21              raise ValueError("material_number contains invalid characters")
22          return v
23
24      @validator("question")
25      def validate_question(cls, v):
26          if not v.strip():
27              raise ValueError("question must not be empty")
28          return v.strip()
29
30  class QueryResponse(BaseModel):
31      answer: str
```



```

32     kg_sparql: Optional[str] = None
33     kg_facts: Optional[List] = None
34     vector_chunks: Optional[List] = None
35     sources: Optional[List[str]] = None
36
37 @app.post("/query", response_model=QueryResponse)
38 def query(request: QueryRequest):
39     state: HybridRAGState = {
40         "question": request.question,
41         "material_number": request.material_number,
42         "history": request.history,
43         "vector_answer": "",
44         "vector_chunks": [],
45         "kg_answer": "",
46         "kg_sparql": "",
47         "kg_facts": [],
48         "final_answer": "",
49         "sources": [],
50         "error": None,
51     }
52
53     result = run_hybrid_rag(state)
54
55     if not result.get("final_answer"):
56         raise HTTPException(status_code=500, detail="Agent returned no
57                               answer")
58
59     return QueryResponse(
60         answer=result["final_answer"],
61         kg_sparql=result.get("kg_sparql"),
62         kg_facts=result.get("kg_facts"),
63         vector_chunks=result.get("vector_chunks"),
64         sources=result.get("sources"),
65     )

```

The endpoint validates the material number against `^[A-Za-z0-9_-]+$` — the same regex used throughout the application. This prevents SPARQL injection: a material number like `ACE001> } INSERT { <evil> }` would fail validation before reaching HANA.

8.8 The full request/response contract

```

1  // Request
2  POST /query
3  Content-Type: application/json
4
5  {
6    "question": "What are the GHS hazard codes for acetone?",
7    "material_number": "ACE001",
8    "history": [
9      {"role": "user", "content": "What is acetone used for?"},
10     {"role": "assistant", "content": "Acetone is a common solvent used
11       in..."}
12   ]
13 }
14
15 // Response
16 {
17   "answer": "The GHS hazard codes for acetone are H225 (Highly flammable
18     liquid and vapour), H319 (Causes serious eye irritation), and H336 (May
19     cause drowsiness or dizziness).",
20   "kg_sparql": "PREFIX msds: <...>\nSELECT ?code ?desc WHERE { GRAPH <...>
21     { <...ACE001> msds:hasHazardCode ?hc . ?hc rdfs:label ?code . ?hc
22     msds:hazardDescription ?desc } }",
23   "kg_facts": [
24     "('H225', 'Highly flammable liquid and vapour')",
25     "('H319', 'Causes serious eye irritation')",
26     "('H336', 'May cause drowsiness or dizziness')",
27   ],
28   "vector_chunks": [
29     {"text": "Section 2: Hazard Identification...", "score": 0.847,
30     "chunk_index": 2},
31     ...
32   ],
33   "sources": ["Knowledge graph: 3 facts", "Document search: 5 passages"]
34 }

```

The response includes both the synthesised answer and the raw retrieval evidence (kg_facts, vector_chunks, kg_sparql). The Fiori UI uses these to show the user exactly

where the answer came from — which SPARQL query retrieved the structured facts, and which document passages supported the narrative answer.

8.9 Conversation history

The `/query` endpoint accepts a history list in the request body. This is how multi-turn conversations work without server-side session state.

```
1  # First question — no history
2  POST /query
3  {"question": "What are the hazard codes for acetone?", "material_number":
4    "ACE001", "history": []}
5
6  # Second question — pass previous turn
7  POST /query
8  {
9    "question": "And what PPE should I wear when handling it?",
10   "material_number": "ACE001",
11   "history": [
12     {"role": "user", "content": "What are the hazard codes for
13       acetone?"},
14     {"role": "assistant", "content": "The GHS hazard codes are H225, H319,
15       and H336."}
16   ]
17 }
```

The answer node in the LangGraph graph includes the history in its prompt, so the second question gets a contextually aware answer (“Given the flammability hazard H225 that we discussed, you should wear...”) without any server-side state.

The frontend keeps a rolling window of the last 10 messages — enough for conversational context without ballooning the prompt. Older messages are silently dropped.

8.10 Testing: three scenarios that prove hybrid wins

The following three test cases demonstrate why hybrid retrieval is better than either strategy alone. Run them after uploading the acetone MSDS to both stores.

8.10.1 The KG wins: precise structured facts

```
1 curl -X POST http://localhost:8000/query \  
2   -H "Content-Type: application/json" \  
3   -d '{  
4     "question": "What are the GHS hazard codes for acetone?",  
5     "material_number": "ACE001",  
6     "history": []  
7   }'
```

Expected KG answer:

H225, H319, H336

Expected vector answer (typical):

Acetone is classified under several GHS hazard categories. The safety data sheet indicates it is a highly flammable substance with eye irritation properties. Refer to Section 2 for complete hazard identification...

The KG returns the exact codes. The vector chain returns useful prose but buries the codes in a paragraph. The synthesised answer leads with the codes and adds the prose context.

8.10.2 The vector wins: narrative safety procedures

```
1 curl -X POST http://localhost:8000/query \  
2   -H "Content-Type: application/json" \  
3   -d '{  
4     "question": "What first aid should I give if someone inhales acetone  
5 vapour?",  
6     "material_number": "ACE001",  
7     "history": []  
8   }'
```

The KG stores structured facts — hazard codes, exposure limits, precaution flags. It

does not store the detailed first-aid procedure paragraph. The vector chain retrieves the exact passage from Section 4 of the MSDS. The final answer comes primarily from the vector chain here, with the KG contributing the exposure limit for context.

8.10.3 Both contribute: a complex combined question

```

1  curl -X POST http://localhost:8000/query \
2    -H "Content-Type: application/json" \
3    -d '{
4      "question": "Is acetone classified as a flammable liquid and what
5      precautions should I take when storing it?",
6      "material_number": "ACE001",
7      "history": []
8    }'
```

The KG answers the first part precisely: H225 confirms flammable liquid classification. The vector chain answers the second part: storage precautions from Section 7 of the MSDS. Neither could answer both parts alone. The synthesis LLM combines them into a single coherent answer.

8.11 Monitoring chain performance

Add timing instrumentation to the orchestrator to track which chain is the bottleneck:

```

1  import time
2
3  with ThreadPoolExecutor(max_workers=2) as executor:
4      start = time.time()
5      futures = {executor.submit(fn, state): name for name, fn in
6                  chains.items()}
7      for future in as_completed(futures, timeout=35):
8          name = futures[future]
9          elapsed = time.time() - start
10         logger.info("%s chain completed in %.2fs", name, elapsed)
11         results[name] = future.result(timeout=30)
```

```

12 total = time.time() - start
13 logger.info("Both chains completed in %.2fs (wall clock)", total)

```

In a typical run against a populated HANA instance:

INFO: vector chain completed in 1.83s

INFO: kg chain completed in 2.41s

INFO: Both chains completed in 2.41s (wall clock)

The wall-clock time is the slower chain, not the sum. If you ran these sequentially, the same run would take 4.24 seconds. The parallel design saves ~1.8 seconds on every query.

8.12 Summary

In this chapter we built the core of the hybrid RAG system:

- Defined a shared `HybridRAGState TypedDict` that all components read and write
- Built the **vector chain**: embed question → cosine search HANA → summarise with Gemini
- Built the **KG chain**: generate SPARQL → execute on HANA → retry-on-empty → summarise
- Wrote the **merge function**: synthesise when both chains return results; fall back gracefully when one or neither does
- Assembled the **orchestrator** using `ThreadPoolExecutor` for true parallel execution
- Exposed a `/query` **endpoint** with input validation and a clean request/response contract
- Demonstrated **stateless conversation history** passed in every request
- Verified with **three test scenarios** that hybrid retrieval outperforms either strategy alone

The parallel orchestrator is the most important design decision in this system. Everything built in Chapters 4, 5, 6, and 7 converges here.

8.13 Checkpoint

Before continuing to Chapter 9, verify the following:

```

1  # 1. Both chains import without errors
2  cd agents
3  python -c "from agents.vector_chain import run_vector_chain; print('vector
  OK')"
4  python -c "from agents.kg_chain import run_kg_chain; print('kg OK')"
5
6  # 2. The orchestrator runs (requires HANA connection and Vertex AI)
7  python -c "
8  from agents.orchestrator import run_hybrid_rag
9  result = run_hybrid_rag({
10     'question': 'What are the hazard codes?',
11     'material_number': 'ACE001',
12     'history': [],
13     'vector_answer': '', 'vector_chunks': [],
14     'kg_answer': '', 'kg_sparql': '', 'kg_facts': [],
15     'final_answer': '', 'sources': [], 'error': None,
16 })
17 print('answer:', result.get('final_answer', '')[:80])
18 "
19
20 # 3. The /query endpoint responds
21 uvicorn main:app --port 8000 &
22 curl -s -X POST http://localhost:8000/query \
23     -H 'Content-Type: application/json' \
24     -d '{"question":"What is
  acetone?","material_number":"ACE001","history":[]}' \
25     | python -m json.tool

```

If all three commands succeed and the final curl returns a JSON response with an answer field, the hybrid RAG agent is working. Chapter 9 extends it with a multi-agent supervisor layer for complex, multi-domain queries.

End of Chapter 8

Chapter 9: The Multi-Agent Supervisor Pattern

In Chapter 8 we built a hybrid RAG agent that runs two retrieval strategies in parallel and merges their results. For the majority of questions — “what are the hazard codes?”, “what are the storage instructions?” — that agent is the right tool. It is fast, deterministic, and requires no coordination overhead.

But consider a different class of question: “I am setting up a new chemical storage facility that will handle acetone. What hazard classifications do I need to post, what are the legal worker exposure limits, and what personal protective equipment should I provide?” That question requires three distinct types of expertise: GHS classification knowledge, regulatory compliance knowledge, and safety procedure knowledge. A single agent with a single retrieval pass will mix these domains, risk missing one, and produce an answer that is harder to reason about.

This is the complexity ceiling of a single-agent system. When a question spans multiple domains and each domain requires focused, specialised reasoning, a team of specialist agents produces better answers than a generalist agent working alone.

This chapter introduces the **supervisor pattern**: a coordinator agent that receives complex questions, decomposes them, routes sub-questions to specialist agents, and synthesises their results into a final answer. By the end of this chapter you will have a multi-agent system built on LangGraph where each specialist runs independently and the supervisor coordinates them.

9.1 The complexity ceiling

A generalist agent has one prompt, one retrieval context, and one LLM call to produce its answer. When questions are simple and focused, this is efficient. When questions are complex and multi-domain, it creates three problems.

Problem 1: Context dilution. A single retrieval pass retrieves the top-5 passages by cosine similarity. For a multi-domain question, those 5 passages will be distributed across domains — perhaps 2 about hazards, 2 about storage, 1 about first aid. Each

domain gets insufficient context.

Problem 2: Prompt overloading. The LLM receives a long prompt with mixed signals. It must simultaneously reason about regulatory thresholds, GHS classifications, and PPE requirements. The answer tends to be shallow across all three rather than deep on any one.

Problem 3: Uneven coverage. The LLM often gives more weight to topics it finds easiest to answer, leaving some domains underserved. With a single-pass architecture, there is no mechanism to detect and compensate for this.

The supervisor pattern solves all three by decomposing the question *before* retrieval.

9.2 The supervisor pattern

The supervisor pattern has four components:

Component	Role
Supervisor	Receives the user question, decomposes it into sub-questions, routes each to the right specialist, collects results
Specialist agents	Each handles one focused domain — retrieves relevant data, generates a domain-specific answer
SummaryAgent	Receives all specialist answers, synthesises them into a single coherent response
Shared state	Carries the original question, sub-questions, and all specialist answers through the graph

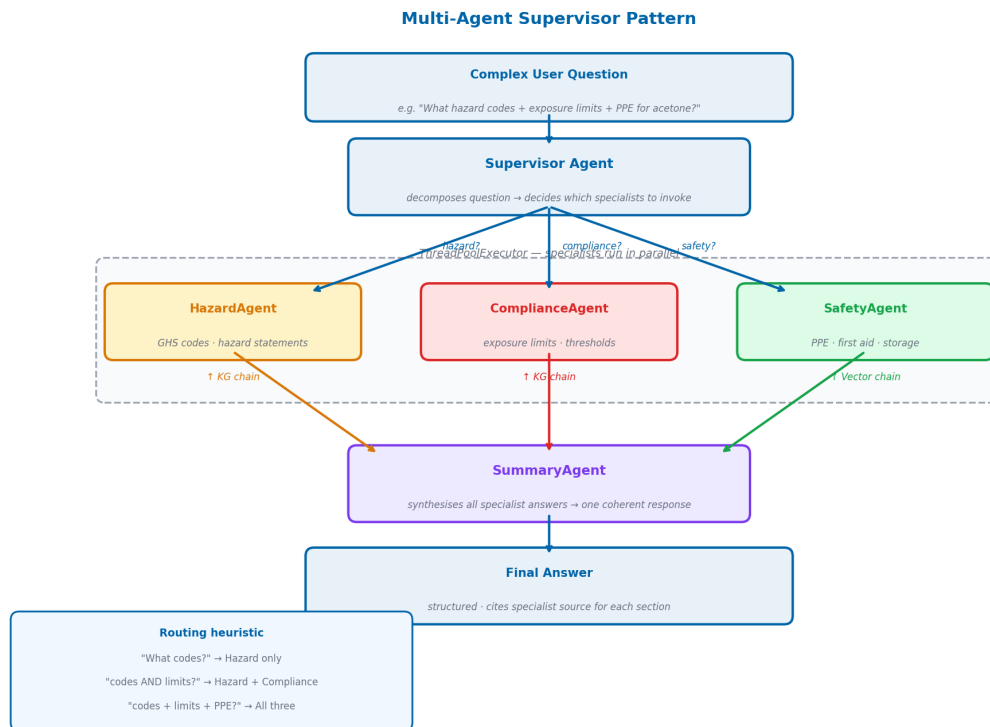


Figure: The multi-agent supervisor — the supervisor decomposes the question and routes sub-questions to three specialist agents (running in parallel where possible). The SummaryAgent synthesises all results into the final answer.

The supervisor does not answer the question. It only routes. This separation of concerns is what makes the pattern scale: you can add a new specialist agent without changing the supervisor prompt, and each specialist can be optimised independently.

9.3 The four specialist agents for MSDS

We design four specialists, each focused on a specific aspect of material safety data:

9.3.1 HazardAgent

Domain: GHS classifications, hazard codes, hazard statements, signal words. **Retrieval strategy:** Knowledge graph — structured facts from `hasHazardCode` and `hazardDescription` predicates. **Strength:** Returns exact codes (H225, H319, H336) and their official descriptions.

9.3.2 ComplianceAgent

Domain: Exposure limits, regulatory thresholds, permissible concentrations. **Retrieval strategy:** Knowledge graph — structured facts from `hasExposureLimit` predicates. **Strength:** Returns precise values with units (500 ppm TWA, 750 ppm STEL) and the regulatory body that set them (OSHA, ACGIH).

9.3.3 SafetyAgent

Domain: Precautions, first aid procedures, PPE requirements, storage and handling. **Retrieval strategy:** Vector search — narrative text from Sections 7 and 8 of the MSDS. **Strength:** Returns detailed procedural instructions that are not easily reduced to structured triples.

9.3.4 SummaryAgent

Domain: Synthesis. **Retrieval strategy:** None — reads outputs from the three specialists. **Role:** Combines specialist answers into a coherent final response, resolves overlaps, and structures the output for the user.

9.4 Shared state for the supervisor graph

Create `agents/agents/supervisor_state.py`:

```

1  from typing import TypedDict, List, Optional, Dict
2
3  class SupervisorState(TypedDict):
4      # Input
5      question: str
6      material_number: str
7      history: List[dict]
8
9      # Decomposed sub-questions (set by supervisor)
10     sub_questions: Dict[str, str] # {"hazard": "...", "compliance":
    "...", "safety": "..."}
11     specialists_needed: List[str] # e.g. ["hazard", "compliance"]
12

```

```

13     # Specialist outputs
14     hazard_answer: str
15     compliance_answer: str
16     safety_answer: str
17
18     # Final output
19     final_answer: str
20     sources: List[str]
21
22     # Control
23     error: Optional[str]

```

The `sub_questions` dictionary maps specialist names to the focused questions the supervisor generated for them. `specialists_needed` is the list of specialists the supervisor decided to invoke — not every question needs all three.

9.5 The supervisor node

The supervisor's job is to read the user's question and decide which specialists are needed and what sub-question to give each one.

```

1  import json
2  import logging
3  from langchain_google_vertexai import ChatVertexAI
4  from langchain_core.messages import HumanMessage
5
6  logger = logging.getLogger(__name__)
7  _llm = ChatVertexAI(model_name="gemini-1.5-pro", temperature=0.0,
8                      max_tokens=512)
9
10 SUPERVISOR_PROMPT = """You are a routing agent for a material safety data
11 system.
12 Your job is to analyse a user's question and decide which specialist
13 agents should answer it.
14
15 Available specialists:

```

```

13 - "hazard": answers questions about GHS hazard codes, hazard
    classifications, signal words, hazard statements
14 - "compliance": answers questions about exposure limits, regulatory
    thresholds, permissible concentrations, OSHA/ACGIH limits
15 - "safety": answers questions about precautions, first aid, PPE, storage,
    handling, spill response, fire fighting
16
17 For each specialist you select, write a focused sub-question that extracts
    only the relevant part of the user's question.
18
19 Respond in JSON format only:
20 {{
21     "specialists": ["hazard", "compliance", "safety"],
22     "sub_questions": {{
23         "hazard": "What GHS hazard codes and classifications apply to
24 {material}}?",
25         "compliance": "What are the worker exposure limits for {material}}?",
26         "safety": "What PPE and handling precautions are required for
27 {material}}?"
28     }}
29
30 Select only the specialists that are relevant. A simple question may need
    only one.
31
32 Material: {material}
33 Question: {question}
34
35 JSON:"""
36
37 def supervisor_node(state: SupervisorState) -> dict:
38     question = state["question"]
39     material_number = state["material_number"]
40
41     prompt = SUPERVISOR_PROMPT.format(
42         material=material_number,
43         question=question,
44     )
45     response = _llm.invoke([HumanMessage(content=prompt)])

```

```

45     content = response.content.strip()
46
47     # Strip markdown code fences if present
48     if content.startswith("```"):
49         content = "\n".join(content.split("\n")[1:-1])
50
51     try:
52         routing = json.loads(content)
53         return {
54             "specialists_needed": routing.get("specialists", ["hazard",
55             "compliance", "safety"]),
56             "sub_questions": routing.get("sub_questions", {}),
57         }
58     except json.JSONDecodeError:
59         logger.warning("Supervisor JSON parse failed, routing to all
60         specialists")
61         return {
62             "specialists_needed": ["hazard", "compliance", "safety"],
63             "sub_questions": {
64                 "hazard": question,
65                 "compliance": question,
66                 "safety": question,
67             },
68         }

```

The supervisor uses temperature 0.0 — routing decisions should be deterministic. The fallback on JSON parse failure routes to all specialists, which is safe (slightly slower, never wrong).

9.6 The specialist agents

Each specialist is a focused version of the hybrid RAG chains from Chapter 8, but constrained to its domain.

```

1  import logging
2  from langchain_google_vertexai import ChatVertexAI, VertexAIEmbeddings
3  from langchain_core.messages import HumanMessage

```

```

4
5 from agents.supervisor_state import SupervisorState
6 from agents.kg_chain import run_kg_chain
7 from agents.vector_chain import run_vector_chain
8 from agents.state import HybridRAGState
9
10 logger = logging.getLogger(__name__)
11 _llm = ChatVertexAI(model_name="gemini-1.5-pro", temperature=0.1,
12                      max_tokens=1024)
13
14 def _make_chain_state(sub_question: str, state: SupervisorState) ->
15     HybridRAGState:
16     """Build a HybridRAGState for a specialist's sub-question."""
17     return {
18         "question": sub_question,
19         "material_number": state["material_number"],
20         "history": [], # specialists don't use conversation history
21         "vector_answer": "", "vector_chunks": [],
22         "kg_answer": "", "kg_sparql": "", "kg_facts": [],
23         "final_answer": "", "sources": [], "error": None,
24     }
25
26 def hazard_agent(state: SupervisorState) -> dict:
27     """KG-focused: GHS codes, classifications, hazard statements."""
28     sub_q = state["sub_questions"].get("hazard", state["question"])
29     chain_state = _make_chain_state(sub_q, state)
30     result = run_kg_chain(chain_state)
31     answer = result.get("kg_answer", "")
32     if not answer:
33         # Fall back to vector if KG found nothing
34         vec_result = run_vector_chain(chain_state)
35         answer = vec_result.get("vector_answer", "No hazard information
36 found.")
37     return {"hazard_answer": answer}
38
39 def compliance_agent(state: SupervisorState) -> dict:
40     """KG-focused: exposure limits, regulatory thresholds."""
41     sub_q = state["sub_questions"].get("compliance", state["question"])
42     chain_state = _make_chain_state(sub_q, state)

```

```

40     result = run_kg_chain(chain_state)
41     answer = result.get("kg_answer", "")
42     if not answer:
43         vec_result = run_vector_chain(chain_state)
44         answer = vec_result.get("vector_answer", "No compliance
information found.")
45     return {"compliance_answer": answer}
46
47 def safety_agent(state: SupervisorState) -> dict:
48     """Vector-focused: precautions, first aid, PPE, storage."""
49     sub_q = state["sub_questions"].get("safety", state["question"])
50     chain_state = _make_chain_state(sub_q, state)
51     result = run_vector_chain(chain_state)
52     answer = result.get("vector_answer", "")
53     if not answer:
54         kg_result = run_kg_chain(chain_state)
55         answer = kg_result.get("kg_answer", "No safety information found.")
56     return {"safety_answer": answer}

```

Each specialist wraps the existing chains from Chapter 8 — they are not new implementations, just focused invocations. The HazardAgent and ComplianceAgent use the KG chain first (structured facts are exactly what they need) and fall back to the vector chain if the KG returns nothing. The SafetyAgent uses the vector chain first (narrative procedures live in prose) and falls back to KG.

9.7 The summary agent

```

1 def summary_agent(state: SupervisorState) -> dict:
2     """Synthesise specialist answers into a final coherent response."""
3     parts = []
4     if state.get("hazard_answer"):
5         parts.append(f"""Hazard
Classification:**\n{state['hazard_answer']}""")
6     if state.get("compliance_answer"):
7         parts.append(f"""Regulatory
Compliance:**\n{state['compliance_answer']}""")

```



```

8     if state.get("safety_answer"):
9         parts.append(f"Safety Procedures:\n{state['safety_answer']}")
10
11     if not parts:
12         return {
13             "final_answer": (
14                 f"No information found for material\n{state['material_number']}. "
15                 "Please verify the material number."
16             ),
17             "sources": [],
18         }
19
20     if len(parts) == 1:
21         # Single specialist answered – no synthesis needed
22         return {
23             "final_answer": list(parts)[0].split("\n", 1)[1],
24             "sources": ["Single specialist"],
25         }
26
27     combined = "\n\n".join(parts)
28     prompt = f"""You are a material safety expert. Multiple specialist
29 agents have
30 analysed the question below and provided their answers.
31 {combined}
32
33 Original question: {state['question']}
34
35 Synthesise the above into a single, well-organised, professional answer.
36 - Use headings to separate sections
37 - Do not repeat information that appears in multiple answers
38 - Be specific with codes, values, and units
39 - Write for a safety officer audience
40
41 Answer: """
42
43     response = _llm.invoke([HumanMessage(content=prompt)])
44     return {

```

```

45         "final_answer": response.content,
46         "sources": [f"{len(parts)} specialist agents"],
47     }

```

9.8 Building the supervisor graph

```

1  from concurrent.futures import ThreadPoolExecutor, as_completed
2  from langgraph.graph import StateGraph, END
3
4  from agents.supervisor_state import SupervisorState
5  from agents.supervisor import supervisor_node
6  from agents.specialist_agents import hazard_agent, compliance_agent,
   safety_agent, summary_agent
7
8  def parallel_specialists_node(state: SupervisorState) -> dict:
9      """Run only the needed specialists in parallel."""
10     needed = state.get("specialists_needed", ["hazard", "compliance",
11 "safety"])
12
13     agent_map = {
14         "hazard": hazard_agent,
15         "compliance": compliance_agent,
16         "safety": safety_agent,
17     }
18
19     results = {}
20     agents_to_run = {name: fn for name, fn in agent_map.items() if name in
21 needed}
22
23     with ThreadPoolExecutor(max_workers=len(agents_to_run)) as executor:
24         futures = {executor.submit(fn, state): name for name, fn in
25 agents_to_run.items()}
26
27         for future in as_completed(futures, timeout=60):
28             name = futures[future]
29             try:
30                 results.update(future.result(timeout=30))
31             except Exception as e:

```

```

28         import logging
29         logging.getLogger(__name__).error("Specialist %s failed:
    %s", name, e)
30
31     return results
32
33 def build_supervisor_graph():
34     graph = StateGraph(SupervisorState)
35
36     graph.add_node("supervisor", supervisor_node)
37     graph.add_node("specialists", parallel_specialists_node)
38     graph.add_node("summary", summary_agent)
39
40     graph.set_entry_point("supervisor")
41     graph.add_edge("supervisor", "specialists")
42     graph.add_edge("specialists", "summary")
43     graph.add_edge("summary", END)
44
45     return graph.compile()
46
47 supervisor_app = build_supervisor_graph()

```

The graph is three nodes: supervisor → specialists (parallel) → summary. The `parallel_specialists_node` runs only the specialists that the supervisor selected, using a `ThreadPoolExecutor` sized to the number of active specialists.

9.9 When to use the supervisor vs the direct agent

The supervisor adds latency — one extra LLM call for routing and one for synthesis. For simple questions, this overhead is not justified.

Question type	Recommended agent	Why
Single-domain factual ("what are the hazard codes?")	Direct hybrid RAG (Ch 8)	Faster, no routing overhead
Single-domain narrative ("what is the first aid?")	Direct hybrid RAG (Ch 8)	Faster, single retrieval pass is sufficient

Question type	Recommended agent	Why
Multi-domain factual + narrative (“codes AND precautions”)	Supervisor	Each domain gets focused retrieval
Complex regulatory + safety question	Supervisor	Prevents context dilution
Real-time chat interface	Direct hybrid RAG (Ch 8)	Latency matters in chat
Batch compliance reports	Supervisor	Quality matters more than speed

A practical rule: if the question contains “and” connecting two distinct domains, use the supervisor.

9.10 The /query-advanced endpoint

Add the supervisor endpoint to `agents/main.py`:

```

1  from agents.supervisor import supervisor_app
2  from agents.supervisor_state import SupervisorState
3
4  class AdvancedQueryRequest(BaseModel):
5      question: str
6      material_number: str
7      history: List[dict] = []
8      use_supervisor: bool = False
9
10     @validator("material_number")
11     def validate_material(cls, v):
12         if not re.match(r"^[A-Za-z0-9_-]+$", v):
13             raise ValueError("material_number contains invalid characters")
14         return v
15
16     @app.post("/query-advanced", response_model=QueryResponse)
17     def query_advanced(request: AdvancedQueryRequest):
18         if not request.use_supervisor:
```

```

19         # Fall back to direct hybrid RAG
20         return query(QueryRequest(
21             question=request.question,
22             material_number=request.material_number,
23             history=request.history,
24         ))
25
26     state: SupervisorState = {
27         "question": request.question,
28         "material_number": request.material_number,
29         "history": request.history,
30         "sub_questions": {},
31         "specialists_needed": [],
32         "hazard_answer": "",
33         "compliance_answer": "",
34         "safety_answer": "",
35         "final_answer": "",
36         "sources": [],
37         "error": None,
38     }
39
40     result = supervisor_app.invoke(state)
41
42     return QueryResponse(
43         answer=result.get("final_answer", ""),
44         sources=result.get("sources"),
45     )

```

The `use_supervisor` flag lets the caller choose which agent to use. The Fiori UI sends `use_supervisor: true` when it detects certain patterns in the question (multiple question marks, keywords like “and”, “also”, “as well as”). For simple questions it uses the faster direct endpoint.

9.11 Testing: a question that requires all three specialists

```

1 curl -X POST http://localhost:8000/query-advanced \
2     -H "Content-Type: application/json" \

```

```

3   -d '{
4       "question": "I am setting up a new lab that will store acetone. What
        hazard classifications do I need to post, what are the legal exposure
        limits for my workers, and what PPE should I provide?",
5       "material_number": "ACE001",
6       "history": [],
7       "use_supervisor": true
8   }'
```

Expected supervisor routing (logged to console):

INFO: Supervisor routed to: ['hazard', 'compliance', 'safety']

INFO: Sub-questions:

hazard: "What GHS hazard codes and classifications apply to ACE001?"

compliance: "What are the worker exposure limits for ACE001?"

safety: "What PPE is required when storing and handling ACE001?"

INFO: hazard specialist completed in 2.1s

INFO: compliance specialist completed in 1.9s

INFO: safety specialist completed in 2.3s

INFO: All specialists completed in 2.3s (parallel)

INFO: Summary agent completed in 1.4s

The three specialists run in parallel. The total wall-clock time for specialists is 2.3 seconds (the slowest), not 6.3 seconds (the sum). The summary adds 1.4 seconds, for a total of about 3.7 seconds.

Compare this to the direct hybrid RAG agent with the same question: the single retrieval pass would retrieve 5 passages from across all three domains, giving the answer approximately 2 passages per domain. The supervisor gives each domain 5 full passages, producing a noticeably more detailed and better-organised answer.

9.12 Summary

In this chapter we extended the system from a single agent to a coordinated multi-agent system:

- Identified the **complexity ceiling** of single-agent architectures for multi-domain questions

- Designed four **specialist agents** (HazardAgent, ComplianceAgent, SafetyAgent, SummaryAgent), each focused on one domain
- Built a **supervisor node** that uses Gemini to decompose questions and route sub-questions
- Ran specialists **in parallel** using ThreadPoolExecutor, keeping wall-clock latency equal to the slowest specialist
- Implemented the **SummaryAgent** to synthesise multi-domain answers into coherent final responses
- Added a /query-advanced endpoint with a use_supervisor flag for caller-controlled routing
- Defined clear criteria for **when to use the supervisor** vs the direct hybrid RAG agent

The multi-agent supervisor is the most architecturally sophisticated component we have built. It demonstrates a general pattern — decompose, specialise, synthesise — that applies well beyond MSDS to any multi-domain enterprise knowledge problem.

9.13 Checkpoint

Before continuing to Chapter 10, verify the following:

```
1  # 1. Supervisor graph compiles
2  cd agents
3  python -c "
4  from agents.supervisor import build_supervisor_graph
5  app = build_supervisor_graph()
6  print('Supervisor graph OK, nodes:', list(app.get_graph().nodes.keys()))
7  "
8
9  # 2. Supervisor routes correctly on a simple question
10 python -c "
11 from agents.supervisor import supervisor_node
12 state = {
13     'question': 'What are the hazard codes for acetone?',
14     'material_number': 'ACE001',
15     'history': [],
```

```
16     'sub_questions': {}, 'specialists_needed': [],
17     'hazard_answer': '', 'compliance_answer': '', 'safety_answer': '',
18     'final_answer': '', 'sources': [], 'error': None,
19 }
20 result = supervisor_node(state)
21 print('specialists needed:', result['specialists_needed'])
22 "
23
24 # 3. The advanced endpoint responds
25 curl -s -X POST http://localhost:8000/query-advanced \
26     -H 'Content-Type: application/json' \
27     -d '{
28         "question": "What are the GHS hazard codes for acetone?",
29         "material_number": "ACE001",
30         "history": [],
31         "use_supervisor": false
32     }' | python -m json.tool
```

If the supervisor graph compiles cleanly, the routing test assigns only ["hazard"] or a small subset of specialists to a simple question, and the curl returns a valid JSON response — the multi-agent system is working. Chapter 10 exposes this entire backend through an SAP CAP OData V4 service with a Fiori Elements UI.

End of Chapter 9